蜗行

李延瑞（lyr.m2@live.cn）

2026.09.27

目录

前言

1 新手村

1.1 点 3
1.2 路径 5
1.3 画面 7
1.4 运动 7
1.5 宏 11
1.6 文字 12
1.7 小结 14

2 正则路径

2.1 运算符宏 15
2.2 类型检测 16
2.3 隐匿 18
2.4 变量作用域 19
2.5 变量宏 21
2.6 曲折路径 22
2.7 小结 23

3 文本框

3.1 包围盒 24
3.2 带后缀的名字 26
3.3 类 29
3.4 宏参数 31
3.5 定位 32
3.6 隐患 34
3.7 小结 35

4 刚柔相济

4.1 插值与逼近 36
4.2 圆角 37
4.3 数字后缀 38
4.4 路径自动柔化 39
4.5 节点切向 40
4.6 消除共线点 41
4.7 小结 43

5 骗术

5.1 交点 44
5.2 路径切割 45
5.3 大巧不工 49
5.4 小结 51

6 不可名状

6.1 标准图形 52

6.2 以边框为文本框本体 52
6.3 散射 53
6.4 自由的代价 54
6.5 动态锚点 57
6.6 小结 59

7 路径上的文字

7.1 路径取点 60
7.2 切向 60
7.3 路径标注宏 62
7.4 如影随形 63
7.5 路径等距偏置 64
7.6 路径贴靠标注宏 66
7.7 小结 68

8 样式表

8.1 文字 69
8.2 文本框 69
8.3 非等比例放大 71
8.4 路径 73
8.5 路径箭头 74
8.6 小结 74

9 Lua 语言

9.1 Lua 表 75
9.2 `scantokens` 与 `runscript` 75
9.3 界面 76
9.4 后缀宏 77
9.5 空值 78
9.6 小结 78

10 ConTEXt 模块

10.1 `input` 79
10.2 TEX 目录树 79
10.3 ConTEXt 模块 80
10.4 汉字 81
10.5 小结 81

11 三种蜗牛

11.1 节点类型 82
11.2 节点样式表 83
11.3 锚点 84
11.4 构造无框节点 87
11.5 有框节点 88
11.6 插图节点 91
11.7 节点辅助宏 92
11.8 小结 95

12 蜗牛的路

12.1 路径样式 96
12.2 路 96
12.3 正则曲线 98
12.4 正则封闭曲线 100
12.5 虚线 101
12.6 直连 103
12.7 小结 103

13 路上的 NPC

13.1 样式表 105
13.2 不爱说话的 NPC 105
13.3 话痨 NPC 107
13.4 借 ConTEXt 之力 108
13.5 小结 108

14 Snail 模块

附录：xi 程序的用法

许可证

后记

前言

蜗牛爬的很慢，它爬行的痕迹是一些图案。这份文档写得也很慢，因为它试图用语言的方式画图，不止使用编程语言，还要用人类语言，正如此刻。

这份文档旨在于阐述如何使用 MetaPost 语言绘制流程图，最终目的是为 ConTEXt 排版环境编写一个流程图绘制模块。不过，这份文档以文学编程方式撰写，内容大体由浅入深，故而也可以作为一份 MetaPost 语言的教程。倘若你并不知晓何谓文学编程，那么这份文档的第三个功用便是向你展现文学编程的一些好处——至于其坏处，一开始我便说了。

我用的文学编程工具是我写的 xi 程序[1]。假设你能按照 xi 项目主页上的说明编译和安装 `xi` 程序，且拥有这份文档对应的文学程序文件 `snail.xi`，便可从该文件里抽取你感兴趣的代码用于试验。例如，对于以下名为「`Hello MetaPost`」的代码片段：

```
@ Hello MetaPost #
\startMPpage
draw textext("Hello \METAPOST!") withcolor darkred;
\stopMPpage
```

使用 `xi` 命令可提取该代码片段的全部内容，将其保存于 `hello.tex` 文件：

```
$ xi --tangle --entrance "Hello MetaPost" --output hello.tex snail.xi
```

或

```
$ xi -t -e "Hello MetaPost" -o hello.tex snail.xi
```

上述命令中的 `$` 是命令提示符，当你尝试输入命令时，无需输入它。

熟悉 TEX 的人应该清楚，现代对 MetaPost 最为热情的 TEX 格式是 ConTEXt，其开发者对 MetaPost 作了一些扩展，他们将包含这些扩展的 MetaPost 取名为 MetaFun。倘若你不知 ConTEXt 为何物，更不知如何安装它，则不妨阅读我写的《ConTEXt 蹊径》[2]。有了 ConTEXt 环境，则本文档的全部示例代码皆可落实。在 ConTEXt 环境中，上述含有 MetaPost 绘图代码的 `hello.tex` 文件，可以用 `context` 命令[3]将其编译为 PDF 文件 `hello.pdf`：

```
$ context hello.tex
```

基于 `xi` 程序，亦可将该文件转化为 ConTEXt 源文档，然后由 `context` 命令生成 PDF 格式的文档，即此刻你所阅读的这份文档。例如，使用以下命令可将 `snail.xi` 文件编织为 ConTEXt 源文件 `snail.tex`：

```
$ xi --config ctx.conf --weave --output snail.tex snail.xi
```

或

```
$ xi -c ctx.conf -w -o snail.tex snail.xi
```

`ctx.conf` 为 `xi` 程序的配置文件，其具体内容见本文档附录。使用 `context` 命令便可将 `snail.tex` 编译为 PDF 文档 `snail.pdf`：

```
$ context snail.tex
```

由于本文档的最终目的是为 ConTEXt 编写一个流程图模块，而这个前言是本文档正文完成后所写，所以我可以用这个流程图模块，将文学程序的代码抽取和文档编织过程画为图 1 所示的样子。

[1] 项目地址：https://github.com/liyanrui/xi 或 https://gitee.com/garfileo/xi

[2] 项目地址：https://github.com/liyanrui/ConTeXt-notes 或 https://gitee.com/garfileo/ConTeXt-notes

[3] `context` 命令本是编译 ConTEXt 源文件的命令，但是它内嵌了 MetaPost 解释器，故而可编译 MetaPost 绘图代码。

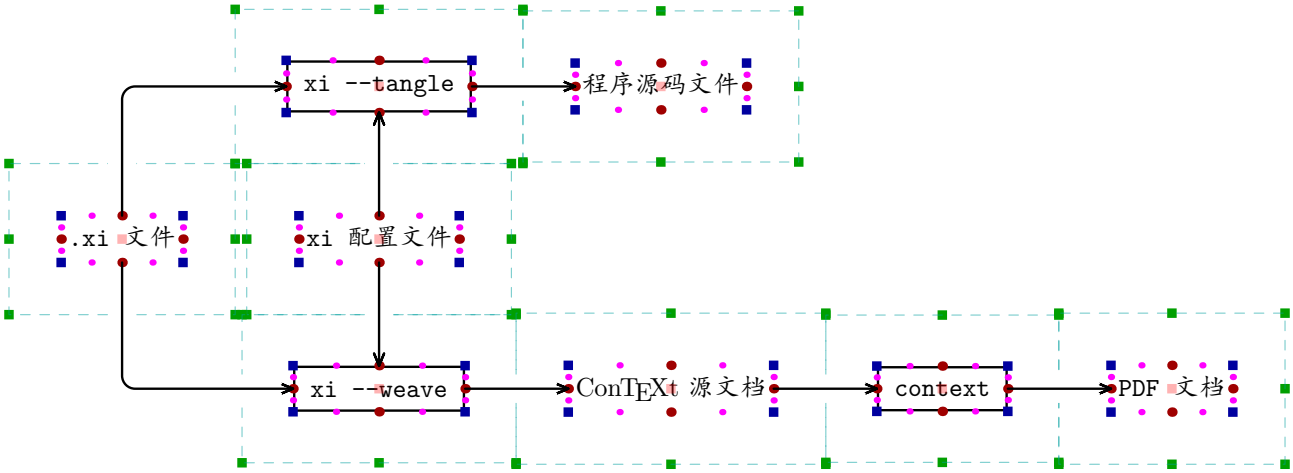


图 1　xi 程序的工作流程

画出图 1 的 MetaPost 代码如下：

```
\usemodule[zhfonts]
\usemodule[snail]
\startMPpage
snail_t xi_file, xi_tangle, src_file, xi_weave, xi_conf, context, tex_file, pdf_file;
% 节点
slug(xi_file, "\type{.xi 文件}"); snail(xi_tangle, "\type{xi --tangle}");
snail(xi_weave, "\type{xi --weave}"); slug(xi_conf, "\type{xi 配置文件}");
snail(context, "\type{context}"); slug(src_file, "\type{程序源码文件}");
slug(tex_file, "\CONTEXT\ \type{源文档}"); slug(pdf_file, "\type{PDF 文档}");
% 节点定位
put (xi_tangle, 巽位) at xi_file.震位; put (src_file, 坎位) at xi_tangle.离位;
put (xi_conf, 坤位) at xi_tangle.乾位; put (xi_weave, 坤位) at xi_conf.乾位;
put (tex_file, 坎位) at xi_weave.离位; put (context, 坎位) at tex_file.离位;
put (pdf_file, 坎位) at context.离位;
% 构造路径
path p[];
p1 := xi_file.子门 yxto xi_tangle.酉门; p2 := xi_conf => xi_tangle;
p3 := xi_tangle => src_file; p4 := xi_file.午门 yxto xi_weave.酉门;
p5 := xi_conf => xi_weave; p6 := xi_weave => tex_file;
p7 := tex_file => context; p8 := context => pdf_file;
% 在调试模式下画出节点及其锚点
snailmod.set("debug", true);
showsnails xi_file, xi_tangle, src_file, xi_conf, xi_weave, tex_file, context, pdf_file;
% 画出路径
for i = 1 upto 8: showroads road(p[i]); endfor;
\stopMPpage
```

1 新手村

我看过的 MetaPost 教程往往颇为严肃，特别是开发者为用户所写的手册，他们似乎颇为担心用户太过于容易学会这门语言，再加上掌握了这门语言的人多为科研领域的学生和老师，于是 MetaPost 变得愈发莫测高深，令人望而生畏。本章可作为一篇面向大众的 MetaPost 入门教程，力求证实该语言甚为平易，顺便证实文学编程亦非阳春白雪。

1.1 点

最简单的涂鸦，是在画布上画一些点。下面的代码画了几个不同颜色的点。

```
@ 一些点 #                                                                    1

\startMPpage
% 这是代码注释
% 在原点画一个深红色的点，它很小，直径为 0.5，以致于你不容易发现它的存在
draw (0, 0) withcolor darkred;
% 用圆头笔，粗度为 5mm，在 (4cm, 0) 处画一个深绿色的点
draw (4cm, 0) withpen pencircle scaled 5mm withcolor darkgreen;
% 用方头笔，粗度为 3mm，在 (3cm, 2cm) 处画一个深蓝色的点
draw (3cm, 2cm) withpen pensquare scaled 3mm withcolor darkblue;
% 三元组 (.4, .5, .6) 表示红色分量 0.4，绿色分量 0.5，蓝色分量 0.6，颜色的分量取值范围为 [0, 1]
draw (1cm, 3cm) withpen pensquare scaled 3mm withcolor (.4, .5, .6);
\stopMPpage
```

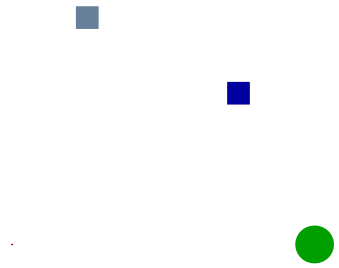


图 1.1　画一些点

在 MetaPost 语言里，数字后面可以紧随着长度单位，例如 `3mm`，这实际上是数字乘以长度单位的简写形式。长度单位本身也是某些固定的数值，例如 `1cm`，实际上是 `1 * cm`，而 `cm` 是数值 28.34645。MetaPost 的数值表达的是尺度，所有数值都是基本尺度单位 `bp` 的倍数，而 `bp` 的值为 1，表示 PostScript 文档标准所规定的大点（Big Point）的直径。1cm 实际上是 `1 * 28.34645 * bp`。

上述示例所用的 `\startMPpage...\stopMPpage` 是 ConTEXt 为 MetaPost 代码提供的一种嵌入环境，该环境可将绘图代码产生的图形嵌入在单页 PDF 文件里，且页面尺寸默认是图形的包围盒尺寸。ConTEXt 也提供了其他 MetaPost 代码嵌入环境，详见前言所述《ConTEXt 蹊径》的第 11 章。

画过了一些点之后，你应该能掌握 MetaPost 基本的画图语句以及画笔和颜色的基本用法。MetaFun 提供了一组预定义颜色并赋予了名字，而且你也可以通过 `(r, g, b)` 三元组的形式设定红、绿和蓝色的分量定义你想要的颜色。

重复使用的画笔，可以用 `pickup` 宏统一设定，从而减少重复代码。例如

```
@ 以粗度为 3mm 的圆头笔画三个点 #                                             2

pickup pencircle scaled 3mm;
draw (1cm, 1cm) withcolor darkgreen;
draw (2cm, 3cm) withcolor darkblue;
```

```
draw (3cm, 3cm) withcolor (.4, .5, .6);
```

如果一些绘图语句所用的画笔和颜色是重复的，可以用 drawoptions 宏作统一设定。例如

```
@ 以粗度为 5mm 的方头笔画三个深红色的点 #                                    3
drawoptions(withpen pensquare scaled 5mm withcolor darkred);
draw (4cm, 0);
draw (5cm, 2cm);
draw (6cm, 3cm);
```

MetaPost 语句以分号结尾，即使多行代码写到一行，也不会有问题，例如：

```
draw (4cm, 0); draw (3cm, 2cm); draw (1cm, 3cm);
```

以上示例代码需放在 \startMPpage...\stopMPpage 环境，亦即

```
@ 再画一些点 #                                                              4
\startMPpage
# 以粗度为 3mm 的圆头笔画三个点 @ <2>
# 以粗度为 5mm 的方头笔画三个深红色的点 @ <3>
\stopMPpage
```

并保存为 ConTEXt 源文档，方可使用 context 命令将其编译为 PDF 单页图形。倘若你用 xi 程序提取上述代码片段的内容，便可得到可编译的 ConTEXt 源文档，而上述示例所演示的便是文学编程中代码片段的引用。xi 程序在提取这个名为「再画一些点」的代码片段时，会将其引用的代码片段内容也提取出来。

需要注意的是，由 xi 程序编织而成的 ConTEXt 源文档，经过 context 命令编译为 PDF 文档后，在每个代码片段的边注区域（单面排版，在右侧边注区域；双面排版，在外侧边注区域）会有一个数字，它是该代码片段的序号。在某个代码片段里引用了另一个代码片段，则引用语句之后会有一些尖括号包含的数字，这些数字是被引用的代码片段的序号。这些序号皆由 xi 程序自动生成，并非文学编程者所写。另外，在支持超级连接的 PDF 文档阅读器里，代码片段引用语句之后的尖括号形式的序号同时也是超级连接，可将页面跳转至被引用的代码片段所在位置。

点实际上是名为 pair 的数据类型。顾名思义，MetaPost 只支持二维空间中的点。本文档沿用主流编程语言的基本概念，将数据类型的实体理解为变量或对象。在 MetaPost 语言里，变量或对象需要先声明，方可为其赋值，例如

```
@ 声明点对象 origin, a, b, c, d #                                           5
pair origin, a, b, c, d;
```

使用:= 符号可以为点对象赋值：

```
@ 为 origin, a, b, c, d 赋值 #                                              6
origin := (0, 0);
a := (4cm, 0); b := (0, 3cm); c := (3cm, 2cm); d := (1cm, 3cm);
```

用 draw 可以画出一个点，倘若不设定画笔和颜色，MetaPost 默认以 0.5bp 直径画出黑色的点，例如

```
draw origin; draw a; draw b; draw c; draw d;
```

MetaPost 提供了 for 语法，基于它可以遍历一些对象。以下代码与上述画 5 个点的代码等效：

```
@ 遍历 origin, a, b, c, d, 逐一画出 #                                       7
```

```
for i in origin, a, b, c, d:
    draw i;
endfor;
```

上述代码也能写成一行：

```
for i in origin, a, b, c, d: draw i; endfor;
```

现在，再次演示文学编程中的代码片段引用技巧，将上述关键代码组织成一个可以实际运行的例子：

```
@ 以遍历的方式，画一组点 #                                                    8
\startMPpage
# 声明点对象 origin, a, b, c, d @ <5>
# 为 origin, a, b, c, d 赋值 @ <6>
# 遍历 origin, a, b, c, d，逐一画出 @ <7>
\stopMPpage
```

从本文档的源文档 snail.xi 中提取上述代码片段的命令如下：

```
$ xi -t -e "以遍历的方式，画一组点" -o foo.tex snail.xi
```

所得 foo.tex 文件的内容如下：

```
\startMPpage
pair origin, a, b, c, d;
origin := (0, 0);
a := (4cm, 0); b := (0, 3cm); c := (3cm, 2cm); d := (1cm, 3cm);
for i in origin, a, b, c, d:
    draw i;
endfor;
\stopMPpage
```

好了，关于 xi 程序的基本用法以及如何看懂本文档的一些示例代码，想必我已经阐述清楚了。

1.2 路径

路径，可将其简单理解为有序点集。在 MetaPost 语言里，路径的类型是 path。与点类型相似，路径对象也需要先声明，再赋值，例如

```
@ 定义路径 foo #                                                              9
path foo;
foo := (4cm, 0) -- (3cm, 2cm) -- (1cm, 3cm);
```

在脑海里，你大致能想象到 foo 的样子，一条由三个点作为节点的折线。不妨使用 draw 语句画出 foo 的精确结果以作验证：

```
@ 画一条路径 #                                                                10
\startMPpage
# 定义路径 foo @ <9>
draw foo withpen pencircle scaled 2pt withcolor darkred;
\stopMPpage
```



图 1.2　一条折线

路径的赋值语句若以 cycle 结尾，可以构造封闭路径。下面代码定义了一个矩形路径并将其画出。

```
@ 画一条封闭路径 #                                                    11
\startMPpage
path rectangle;
rectangle := (0, 0) -- (5cm, 0) -- (5cm, 2.5cm) -- (0, 2.5cm) -- cycle;
draw rectangle withpen pencircle scaled 2pt withcolor darkred;
\stopMPpage
```



图 1.3　一条封闭路径

使用 point ... of ... 语法可基于从 0 开始的序数获取路径的节点坐标。例如，获取路径 foo 的第 1 个、第 2 个以及第 3 个节点，只需

```
pair a, b, c;
a := point 0 of foo;
b := point 1 of foo;
c := point 2 of foo;
```

路径可由一个点构成，例如

```
path a;
a := (0, 0);
```

两条路径可以用 -- 组合为一条路径，例如

```
path a, b, c;
a := (0, 0) --  (3cm, 1cm);
b := (4cm, 1cm) -- (5cm, 2cm);
c := a -- b;
```

路径也可以将自我与其他路径组合起来，例如

```
@ 定义路径 a #                                                        12
path a, b;
a := (0, 0) --  (3cm, 1cm);
b := (4cm, 1cm) -- (5cm, 2cm);
a := a -- b;
```

若使用 drawarrow 画路径，则所画路径便会有方向，亦即带箭头的路径，例如

```
@ 画一条带箭头的路径 #                                                    13
\startMPpage
# 定义路径 a @ <12>
drawarrow a withpen pencircle scaled 2pt withcolor darkred;
\stopMPpage
```

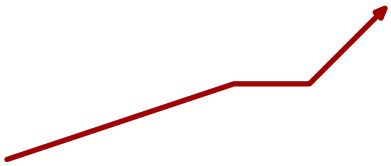


图 1.4　一条带箭头的路径

1.3 画面

MetaPost 可将点和路径组合为静态的画面，并为此提供了 picture 类型。使用 image 宏可以构造 picture 类型的对象，例如

```
@ 构造画面 foo #                                                          14
picture foo;
foo := image(
    drawoptions(withpen pencircle scaled 2pt withcolor darkgray);
    drawarrow (0, 0) -- (5cm, 0);
    drawarrow (0, 0) -- (0, 2cm);
);
```

image 宏可将一些绘图语句（包括画笔和颜色设定语句）添加到 picture 对象，亦即可将后者视为一些绘图语句的组合体，我更建议你将其想象为照片底片，需要用 draw 语句将其冲洗为照片：

```
@ 呈现一个画面 #                                                          15
\startMPpage
# 构造画面 foo @ <14>
draw foo;
\stopMPpage
```



图 1.5　画面

实际上，MetaPost 语言的整个绘图过程都是在向一个叫做 currentpicture 的底片上添加元素，当我们使用 context 命令编译源文档时，便会其冲洗出来。

1.4 运动

无论是点（pair），路径（path），只能用罗列点坐标的方式构造画面。例如，若要画一个圆，你必须事先算出这个圆上所有点的坐标，然后将其连成路径，这种做法非常低效。倘若将一个圆想象为

一个点绕圆心的匀速运动所形成的轨迹，使用 MetaPost 语言描述这个轨迹的生成过程，如此画法，则非常高效。

MetaPost 为点、路径以及画面提供了旋转语法，可基于这些语法描述点或路径的运动轨迹。例如，以下代码演示了一个点绕另一个点的运动轨迹：

```
@ 点 a 绕原点的旋转运动轨迹 #                                                    16
\startMPpage
pair a;
a := (3cm, 0);
for i = 0 upto 180:
    draw a rotated i withpen pencircle scaled 2pt withcolor darkred;
endfor;
\stopMPpage
```

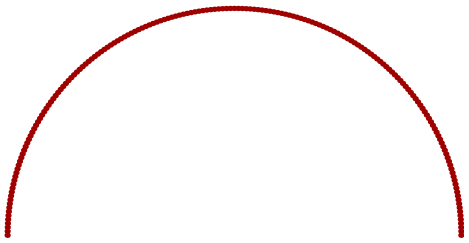


图 1.6　半圆轨迹

你看到了 for 语句的另一种写法，「i = 0 upto 180」表示 i 从 0 开始，每次增 1，至 180 为止。你不妨将其理解为逐一遍历整数集 $\{0, 1, ..., 180\}$ 中的每个数字，i 表示当前数字。不过，这个示例里，最重要的部分是「a rotated i」，它表示将点 a 绕原点，旋转 i 度角。于是，for 语句的每次工作都是在画一个点，画出来的点最终形成了半圆形，亦即图 **1.6** 所示的半圆，实际上是由 181 个直径为 2pt 的点构成。

MetaPost 的 for 语句在遍历整数集时，可以设定步长，基于该特性，略微改动上述示例，如下：

```
@ 点 a 绕原点的旋转运动轨迹之二 #                                                17
\startMPpage
pair a;
a := (3cm, 0);
for i = 0 step 10 until 180:
    draw a rotated i withpen pencircle scaled 4pt withcolor darkred;
endfor;
\stopMPpage
```

改动之后的 for 语句的功能是，依序遍历整数集 $\{0, 1, ..., 180\}$ 的子集，i 每次增 10，于是所画的点是 11 个，结果如图 **1.7** 所示。

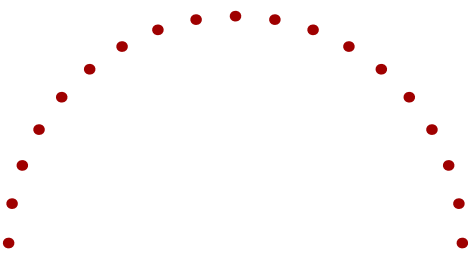


图 1.7　半圆轨迹之二

基于路径组合运算，可将所有旋转了 i 度角的点 a 组合为一条路径，例如：

```
@ 点 a 绕原点的旋转运动轨迹之三 #                                    18
\startMPpage
path p;
pair a;
a := (3cm, 0);
p := a;
for i = 10 step 10 until 180:
    p := p -- a rotated i;
endfor;
drawarrow p withpen pencircle scaled 2pt withcolor darkred;
\stopMPpage
```

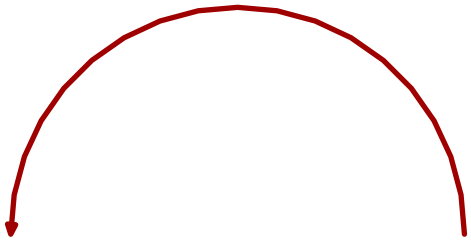


图 1.8　半圆轨迹之三

上述的路径示例揭示了，MetaPost 的旋转运算，是以逆时针为正向的，亦即若将上述代码中的「`a rotated i`」修改为「`a rotated -i`」，结果得到的是圆的下半截，如图 **1.9** 所示。

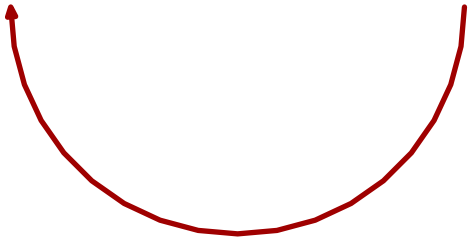


图 1.9　半圆轨迹之四

需要记住，MetaPost 的旋转运算总是以坐标原点为中心。若你希望图形（点、路径或画面）绕某个指定的点旋转，则需要偏移运算的介入。设点 a 为 (4cm, 2cm)，点 b 为 (3cm, 2cm)：

```
@ 定义点 a 和 b #                                                    19
pair a, b;
a := (4cm, 2cm);
b := (3cm, 2cm);
```

考虑如何画出 a 绕 b 旋转 270 度角的轨迹。首先，将原点，a 和 b 画出来：

```
@ 画出原点、a 和 b #                                                 20
draw (0, 0) withpen pensquare scaled 3mm withcolor darkgreen;
draw a withpen pencircle scaled 4pt withcolor darkred;
draw b withpen pencircle scaled 4pt withcolor darkblue;
```

之所以画出原点，是希望能清晰呈现 a 绕 b 旋转这一事实。

现在，请认真思考这样一个事实，令 a 绕 b 旋转 60 度角，等效于以下过程：

- a 和 b 整体挪移，令 b 与原点重合；
- 令偏移后的 a 绕原点旋转 60 度角，构造旋转轨迹 p；
- 将 a，p 以及 b 整体挪移，令 b 回到原位置。

上述过程可以用以下代码实现：

```
@ 令 a 绕 b 旋转 270 度角，并构造旋转轨迹 p # 21
path p;
a := a shifted -b;
p := a;
for i = 1 upto 270:
    p := p -- a rotated i;
endfor;
p := p shifted b;
a := a shifted b;
```

上述代码中的 -b，MetaPost 语言的编译器会将其理解为 -(3cm, 2cm)，亦即 (-3cm, -2cm)。shifted 表示偏移运算。「a shifted -b」意为「将 a 沿向左（x 轴负方向）移动 3cm，向下（y 轴负方向）移动 2cm」。注意，上述过程并没有对 b 作偏移运算，因为将 b 移到某个位置之后，再将其移回原处，这个过程等效于 b 不变。

现在，将上述过程组合起来，并呈现其结果：

```
@ 绕指定点旋转示例 # 22
\startMPpage
# 定义点 a 和 b @ <19>
# 画出原点、a 和 b @ <20>
# 令 a 绕 b 旋转 270 度角，并构造旋转轨迹 p @ <21>
drawarrow p withpen pencircle scaled 2pt withcolor darkred;
\stopMPpage
```

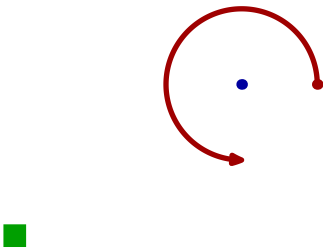


图 1.10　绕指定点旋转

除了旋转和平移，MetaPost 语言也提供了缩放运算，实际上至此为止，我们一直在使用它设置画笔的粗度，亦即 scaled。画笔 pencircle 和 pensquare 的粗度是 1，亦即 1bp，若其后出现 scaled 语句，则会将其放大为指定粗度，该语句不仅能作用于画笔，也能作用于点、路径以及画面，例如

```
% 定义封闭路径 p，一个矩形
path p;
p := (0, 0) -- (2cm, 0) -- (2cm, 1cm) -- (0, 1cm) -- cycle;
% 将 p 放大 2 倍
p := p scaled 2;
% 将 p 缩小 2 倍
p := p scaled 0.5;
```

上述代码对 p 放大 2 倍后又缩小 2 倍，结果 p 回到了它一开始的样子。你不妨亲自将 p 画出来，验证一下。

1.5 宏

若让一个点绕指定的点旋转，即便你已知其理，但想必你并不太愿意将其移过来，旋转，再移过去的。可以将这个过程封装为一个宏，从而让这件事变得容易一些。

也许你从来都没听说过「宏」……最好如此，否则别人对它的很多负面评价多少会让你有所畏惧。实际上，宏是很简单的概念。将一些代码封装起来——我将称之为宏体——，然后为宏体取一个名字，如此便定义了一个宏。调用一个宏，便是展开它所封装的代码。若是宏体里有一些数据是在调用宏时才能确定，需要将其设成宏的参数。在调用宏时，将实际数据传给宏体，宏展开后的代码，其参数便得以确定。

以下代码定义了一个宏 `foo`：

```
def foo(expr a, b) =
    a + b
enddef;
```

「`a + b`」便是宏体，也是宏 `foo` 被调用时所展开的结果，但 `a` 和 `b` 都是参数，它们具体的值需要在调用 `foo` 时方可确定，例如

```
pair s, t, r;
s := (1, 2); t := (-1, -2);
r := foo(s, t);
```

上述代码中，`foo(s, t)` 是在调用宏 `foo`，其展开结果是

```
s + t
```

故而，上述代码本质上是

```
pair s, t, r;
s := (1, 2); t := (-1, -2);
r := s + t;
```

MetaPost 能够直接对两个点坐标求和，故而 `r` 的值便是 `(0, 0)`。

上面这个简单的例子想必能够让你隐约感受到宏的基本原理。现在，我们试着为点 `a` 绕点 `b` 旋转角度 `t` 这个过程创造一个宏，叫作 `rotated_around`：

```
@ 定义 rotated_around 宏 #                                                    23
def rotated_around(expr a, b, t) =
    ((a shifted -b) rotated t) shifted b
enddef;
```

MetaPost 语言允许使用小括号表达运算的优先级。带小括号的表达式，其优先级高于括号的外层部分，故而上述宏体是先将 `a` 偏移 `-b`，然后绕原点旋转角度 `t`，继而将所得结果偏移 `b`。

基于 `rotated_around` 宏，上一节令 `a` 绕 `b` 旋转 270 度角，并构造旋转轨迹 `p` 的过程可简化为

```
@ 基于 rotated_around 宏，令 a 绕 b 旋转 270 度角，并构造旋转轨迹 p #          24
path p;
p := rotated_around(a, b, 0);
for i = 1 upto 270:
    p := p -- rotated_around(a, b, i);
endfor;
```

上述代码中的

```
p := p -- rotated_around(a, b, i);
```

在宏调用语句得以展开后，会变成

```
p := p -- ((a shifted -b) rotated i) shifted b;
```

为了便于验证，下面再构造一个绕指定点旋转过程的示例：

```
@ 绕指定点旋转示例之二 #                                                    25
\startMPpage
# 定义 rotated_around 宏 @ <23>
# 定义点 a 和 b @ <19>
# 画出原点、a 和 b @ <20>
# 基于 rotated_around 宏，令 a 绕 b 旋转 270 度角，并构造旋转轨迹 p @ <24>
drawarrow p withpen pencircle scaled 2pt withcolor darkred;
\stopMPpage
```

该示例所得结果与图 **1.10** 相同。

实际上，MetaPost 语言已经提供了一个类似的运算宏 `rotatedaround`，其用法如下：

```
% 令点 a 绕 b 旋转 30 度角
pair c;
c := a rotatedaround (b, 30);
```

1.6 文字

有时需要画出文字，为此 MetaPost 提供了 `label` 宏，用于在指定位置放置指定的文字。例如

```
@ label 示例 #                                                              26
\startMPpage
pair a, b;
a := (3cm, 1cm);
b := (4cm, 2cm);
label("o", (0, 0));
label("a", a) withcolor darkred;
label("b", b) withcolor darkblue;
\stopMPpage
```



图 1.11　`label` 示例

`label` 宏接受两个参数，第一个参数是要画出来的文字，第二个参数是文字中心所在的位置。通常，在用 MetaPost 绘制几何图形时，需要在指定位置的某一侧放置文字，要达成这一目的，需要使用后缀形式的 `label` 宏，即 `label.top`，`label.rt`，`label.bot` 和 `label.lft`，分别表示在指定位置的顶部、右侧、底部和左侧放置文字。以下代码，可为上一节的示例增加文字标注：

```
@ 绕指定点旋转示例之三 #                                                    27
\startMPpage
# 定义 rotated_around 宏 @ <23>
# 定义点 a 和 b @ <19>
# 画出原点、a 和 b @ <20>
label("o", (0, 0));
label.bot("b", b);
label.rt("a", a);
# 基于 rotated_around 宏，令 a 绕 b 旋转 270 度角，并构造旋转轨迹 p @ <24>
drawarrow p withpen pencircle scaled 2pt withcolor darkred;
\stopMPpage
```

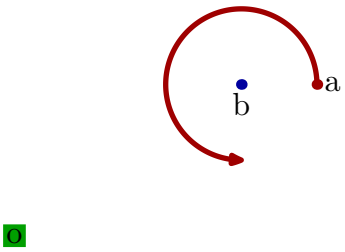


图 1.12　label 示例之二

需要注意的是，label 宏默认不支持汉字，原因是 MetaPost 编译器默认使用的字体是西文字体，未包含汉字字形。在 ConTEXt 环境里使用 MetaPost，可以用 MetaFun 宏 textext 宏代替 label 宏。textext 宏可将文字转化为画面（picture 对象）并使用 draw 语句画出，至于文字的定位问题，可基于 shifted 运算解决。例如

```
@ 绕指定点旋转示例之四 #                                                    28
\usemodule[zhfonts]
\startMPpage
# 定义 rotated_around 宏 @ <23>
# 定义点 a 和 b @ <19>
# 画出原点、a 和 b @ <20>
draw textext("原点") shifted (0, -12pt);
label.bot("b", b);
label.rt("a", a);
# 基于 rotated_around 宏，令 a 绕 b 旋转 270 度角，并构造旋转轨迹 p @ <24>
drawarrow p withpen pencircle scaled 2pt withcolor darkred;
\stopMPpage
```

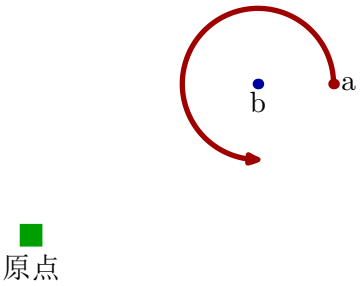


图 1.13　label 示例之三

上述代码为了实现汉字的支持，使用了我为 ConTEXt 所写的 zhfonts 模块，该模块的安装及用法详见本文档前言部分所提及的文档《ConTEXt 蹊径》第 15 章。不过，zhfonts 模块并非必须，你可以参考《ConTEXt 蹊径》第 3 章，使用 ConTEXt 内置的字体功能解决汉字问题。

由于 MetaPost 与 TEX 从一开始便是鱼水之情，故而 MetaPost 能够直接将 TEX 排版语句画出。例如，以下示例能够画出图文框：

```
@ TeX 排版语句示例 #                                                    29
\usemodule[zhfonts]
\startMPpage
draw textext("\TEX\ 数学公式: $E = mc^2$") withcolor darkred;
\stopMPpage
```

TEX 数学公式：$E = mc^2$

图 1.14　画 TEX 排版语句

1.7 小结

本章通过一些不严肃的示例，基于文学编程方式简略介绍了 ConTEXt 环境里的 MetaPost 基本语法，但愿能让你觉得 MetaPost 是一门有趣的图形编程语言。在之后的章节里，这些语法会经常使用，同时也会在适当的时机引入 MetaPost 以及 MetaFun 的更深层次的语法和技巧，如同一个秘境难以避免的一些历险。

2 正则路径

有这样一个世界，它的道路只能沿着东西或南北方向，并不遵守两点之间线段最短这一欧几里得几何公理。这样的世界其实并不奇怪，几乎所有坐落在平原上的现代城市内部的道路通常如此设计。我将这类路径称为正则路径，只不过在用 MetaPost 语言构造这种路径时，采取的方向并非东西南北，而是与 x 轴或 y 轴平行的方向。如此走路，也许很适合刚从新手村里出来的人。

2.1 运算符宏

从点 a 到点 b 的沿 x 轴方向可以构造一条路径，即从 a 出发，沿着 x 轴方向走到点 b 在该方向上的投影点为止。MetaPost 语言提供了 xpart 和 ypart 语句，可分别用于获取点的 x 与 y 坐标分量。我们想要构造的这条路径可以表示为

```
a -- (xpart b, ypart a)
```

现在我们已经具备了定义一些简单的宏的能力，可将上述语句封装为宏 xto：

```
def xto(expr a, b) =
    a -- (xpart b, ypart a)
enddef;
```

xto 的用法如下：

```
path p;
pair a, b;
a := (1cm, 0); b := (5cm, 3cm);
p := xto(a, b);
```

不过，MetaPost 还有一种宏定义机制，能够让宏的调用形式颇似 shifted，rotated 以及 scaled 等运算符，故而基于该机制定义的宏可称为运算符宏。

运算符宏有三个优先级。优先级最高的运算符宏，用 primarydef 定义。优先级次之的运算符宏，用 secondarydef 定义。优先级最低的运算符宏，用 tertiarydef 定义。事实上，我们已经遇到过几次运算符优先级的问题，只是我未将其挑明，例如

```
p := a -- b shifted (2cm, 3cm);
```

显然，-- 的优先级是低于 shifted 的，否则 shifted 便不是作用于点 b，而是作用于路径 a -- b 了。故而，上述代码，若用小括号表达优先级，可表示为

```
p := a -- (b shifted (2cm, 3cm));
```

-- 符号的运算优先级应该是 tertiarydef 级别，可以将 xto 定义为该级别的运算符宏：

```
tertiarydef a xto b =
    a -- (xpart b, ypart a)
enddef;
```

新的 xto 宏的用法如下：

```
path p;
pair a, b;
```

```
a := (1cm, 0); b := (5cm, 3cm);
p := a xto b;
```

虽然新的 xto 宏能够像运算符一样使用，但是它却无法做到 -- 那样连接更多的点。例如，对于以下三个点

```
@ 定义点 a, b 和 c #                                                    30
pair a, b, c;
a := (1cm, 0); b := (5cm, 2cm); c := (6cm, 1cm);
```

用 xto 将上述三点连接为一条路径：

```
path p;
p := a xto b xto c;
```

然而上述代码是非法的，若想知其问题所在，需要将述代码中第一个 xto 宏调用展开，可得

```
p := a -- (xpart b, ypart a) xto c;
```

由于 -- 和 xto 具有相同的运算优先级，而 MetaPost 编译器会从左向右理解表达式，故而会先用 -- 将 a 和 (xpart b, ypart a) 组合为路径，于是第 2 个 xto 宏的左侧所接受的对象是路径，而非点，结果导致宏体里的 ypart 语句非法。倘若你并不理解这番讨论，不妨将上述代码理解为

```
p := (a -- (xpart b, ypart a)) xto c;
```

展开上述代码中的 xto 宏调用语句，可得

```
p := (a -- (xpart b, ypart a)) -- (xpart c, ypart (a -- (xpart b, ypart a)));
```

此刻，也许你会觉得宏语言的面目开始变得有些狰狞了起来。不过，对于上述 xto 宏存在的问题，解决起来并不难，只需提高其运算优先级即可，再重新定义它一次：

```
secondarydef a xto b =
    a -- (xpart b, ypart a)
enddef;
```

现在 xto 宏的优先级为中级，高于 -- 运算符，于是

```
p := a xto b xto c;
```

第一次展开结果是

```
p := a -- (xpart b, ypart a) xto c;
```

由于 xto 的优先级高于 --，故而 xto 左侧的路径尚未形成，(xpart b, ypart a) 便被 xto 捕获为第一个参数了，于是进一步展开为

```
p := a -- (xpart b, ypart a) -- (xpart b, ypart a) -- (xpart c, ypart (xpart b, ypart a));
```

所得路径 p 虽然中间两个节点是重复的，但路径本身是合法的。

2.2 类型检测

上一节虽然最后解决了 xto 组合超过两个点的问题，但所用方法只是投机取巧。假如依然令 xto 处于最低的运算等级，问题的真正解决需要着落在 xto 能否识别出左侧参数的类型以作出与之相符

的处理。显然，如果 xto 左侧参数如果是 pair 类型，上一节给出的宏体足以应对。倘若 xto 的左侧参数是路径，应该取其末端节点，将其与右侧参数（点对象）构成路径。

上述想法，MetaPost 语言完全支持，甚至代码写起来也颇为直观：

```
tertiarydef a xto b =
    if path a:
        pair c;
        c := point (length a) of a;
        c -- (xpart b, ypart c)
    else:
        a -- (xpart b, ypart a)
    fi
enddef;
```

上述宏定义信息量猛增，出现了一些之前未曾介绍过的语法。首先是 `if ... else ... fi`，这是 MetaPost 的条件语句，现在大致了解其形式即可，后续会经常使用条件语句，可慢慢熟悉它的写法。其次，在条件语句里，path 的作用是检测一个对象是否为路径。在上述代码里，若 a 为路径，则 xto 的展开结果便是 if 分支中的内容，即

```
pair c;
c := point (length a) of a;
c -- (xpart b, ypart c)
```

length 语句可用于计算路径的节点数，只是其计算结果是路径节点数量减 1。由于路径阶段的序号是从 0 开始，故而

```
point (length a) of a
```

表示获取路径 a 的最后一个节点，即其末端端点。倘若 a 并非路径，则默认将其视为点对象，于是 xto 的展开结果便是 else 分支中的内容，即

```
a -- (xpart b, ypart a)
```

至此，你应该对新的 xto 宏定义中所涉及的语法有所理解了，只是这个宏在遇到参数 a 为路径类型时，其展开结果虽然是正确的，可是与其他代码的结合却会出现错误，例如

```
path p;
p := (0, 0) --  (1cm, 0);
p := p xto (4cm, 2cm);
```

最后一行的展开结果是

```
p := pair c;
    c := point (length a) of a;
    c -- (xpart b, ypart c);
```

现在，你应该深刻认识到了，所谓的宏展开，基本上是在宏出现的位置上复制出来一段代码，所复制的代码一旦与其相邻的代码出现错误的结合，便会出现无法通过编译的非法语句。上述代码里，显然是不能将 c 的声明语句作为值赋给路径 p 的，置于最后一行代码，原本是希望它能够与 p := 部分结合的，可是关山重重，二者竟无缘得见。

2.3 隐匿

认真观察上一节最后的那段代码，也许你会产生这样的想法，倘若能将

```
pair c;
    c := point (length a) of a;
```

屏蔽掉，则 `p :=` 便可与

```
c -- (xpart b, ypart c);
```

碰面并结合起来。你敢如此想，MetaPost 便敢于支持这个想法，以 `hide` 语句的形式，例如

```
p := hide(pair c;
          c := point (length a) of a;)
    c -- (xpart b, ypart c);
```

`hide` 宏像隐身衣，能够让一些语句变得透明，亦即在保证其效用的前提下，使其失去形体，从而无法与相邻的其他语句产生结合。

倘若在 `xto` 的定义里引入 `hide` 宏，便可得到一个足够健壮的 `xto` 宏了，如下

```
@ xto 宏定义 [试验版] #                                                    31
tertiarydef a xto b =
    if path a:
        hide(pair tmp_loc_hid;
             tmp_loc_hid := point (length a) of a;)
        a -- (xpart b, ypart tmp_loc_hid)
    else:
        a -- (xpart b, ypart a)
    fi
enddef;
```

若想验证 `xto` 宏是否能如我们所愿，先将 **2.1** 节定义的三个点画出来：

```
@ 画出 a, b, c 并予以标注 #                                                32
pickup pensquare scaled 4pt;
draw a; draw b; draw c;
label.llft("$a$", a); % 在 a 点左下角添加标注
label.urt("$b$", b);  % 在 b 点右上角添加标注
label.lrt("$c$", c);  % 在 c 点右下角添加标注
```

上述代码使用了 `label` 宏一些新后缀。然后用 `xto` 连接这三个点，得到路径 `p`：

```
@ 用 xto 连接 a, b 和 c, 得到路径 p 并将其画出来 #                          33
path p;
p := a xto b xto c;
draw p withpen pencircle scaled 2pt withcolor darkred;
```

将上述过程组装起来，便可得到一个可实际运行的示例：

```
@ 测试 xto 宏 #                                                            34
\startMPpage
```

```
# xto 宏定义 [试验版] @ <31>
# 定义点 a, b 和 c @ <30>
# 画出 a, b, c 并予以标注 @ <32>
# 用 xto 连接 a, b 和 c, 得到路径 p 并将其画出来 @ <33>
\stopMPpage
```



图 2.1　xto 构造的路径

2.4 变量作用域

也许你已经注意到了，上一节定义的 xto 宏，在 hide 语句里，以 tmp_loc_hid 作为一个中间点的名字显得有些怪异，是因为 MetaPost 变量作用域所限不得已而为之。

MetaPost 的变量作用域默认是全局的，亦即所有变量默认皆为全局变量，这与绝大多数主流编程语言的理念是相悖的。在后者看来，变量的作用域应当是有限的。例如，你作为某个公司的员工，那么你的作用域就只能局限在公司内部而不会对公司外部的世界产生任何影响，这大概也是孔子毕生追求的社会体系。可是在 MetaPost 语言里，你的一举一动可能会影响整个世界。

tmp_loc_hid 是在宏 xto 的定义中出现的变量，不过一旦 xto 被调用，这个变量也就有了定义，而且可以随处使用。例如

```
@ 变量作用域演示 #                                                    35
\startMPpage
# xto 宏定义 [试验版] @ <31>
# 定义点 a, b 和 c @ <30>
# 画出 a, b, c 并予以标注 @ <32>
# 用 xto 连接 a, b 和 c, 得到路径 p 并将其画出来 @ <33>
draw tmp_loc_hid withpen pensquare scaled 4pt withcolor darkblue;
label.bot("\type{tmp_loc_hid}", tmp_loc_hid);
\stopMPpage
```

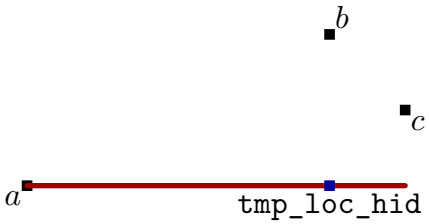


图 2.2　画一个全局变量

通过上述示例，想必你应该知晓 MetaPost 的变量有多么无法无天了吧？主流编程语言也允许一些变量具有全局作用域，只需在所有作用域的外部定义一些变量，对此 MetaPost 的看法是，王侯将相，宁有种乎？为了避免 xto 宏定义中的变量与外部其他变量重名而发生一些冲突，故而取了 tmp_loc_hid 这种怪异的变量名。

不过，MetaPost 也能够限制变量的作用域，方法是使用 begingroup 与 endgroup 构造一个局部作用域，并且以一个值作为该作用域的求值结果。不妨将这个作用域理解为一个表达式，而表达式必须有求值结果，如同一个公司必须有所产出。如果局部作用域没有求值结果，则其求值结果为空。

现在，对 xto 的定义略作修改，也许有助于你理解局部作用域的概念：

```
tertiarydef a xto b =
    if path a:
        hide(begingroup
             save tmp_loc_hid;
             pair tmp_loc_hid;
             tmp_loc_hid := point (length a) of a;
             endgroup)
        a -- (xpart b, ypart tmp_loc_hid)
    else:
        a -- (xpart b, ypart a)
    fi
enddef;
```

在局部作用域里，需要用 save 标定某些名字只能在当前的局部作用域内有效。依上述定义的 xto 实际上是无法用的，因为在 if 分支里，当 ypart 试图取 tmp_loc_hid 的 y 坐标分量时，此时 tmp_loc_hid 位于定义它的局部作用域之外，故而是一个无效的名字。

我希望你能有所醒悟，既然 begingroup 和 endgroup 构造的局部作用域相当于一个表达式，那么 hide 也就没必要存在了。我们只需将 xto 的 if 分支的内容修改为

```
begingroup
save tmp_loc_hid;
pair tmp_loc_hid;
tmp_loc_hid := point (length a) of a;
a -- (xpart b, ypart tmp_loc_hid)
endgroup
```

xto 便会起死回生，而且 tmp_loc_hid 也会老老实实被压在五指山下，变成一个局部变量。

事实上，hide 宏本身便是基于 begingroup 和 endgroup 构造局部作用域的手法实现的，不过现在破解其秘密的时机并不成熟。

下面给出 xto 宏的完整定义：

```
@ 定义 xto 宏 #                                                    36
tertiarydef a xto b =
    if path a:
        begingroup
        save c; pair c; c := point (length a) of a;
        a -- (xpart b, ypart c)
        endgroup
    else:
        a -- (xpart b, ypart a)
    fi
enddef;
```

仿照 xto 宏的最终版本，可以定义 yto 宏：

```
@ 定义 yto 宏 #                                                    37
tertiarydef a yto b =
    if path a:
        begingroup
        save c; pair c; c := point (length a) of a;
```

```
        a -- (xpart c, ypart b)
        endgroup
    else:
        a -- (xpart a, ypart b)
    fi
enddef;
```

yto 可以构造从点 a 到点 b 的沿 y 轴方向的路径。

2.5 变量宏

当你领悟了 begingroup 和 endgroup 所构造的局部作用域本质上是表达式，就有了能够让宏的行为等效于变量的能力，即定义变量宏的能力。在主流编程语言里，与变量宏对应的概念是有返回值的函数。下面的代码定义的宏，像 **1.5** 节定义过的 foo 宏一样，对两个点求和，但是会返回所得结果：

```
@ 定义变量宏 foo #                                                    38
def foo(expr a, b) =
begingroup
save c; pair c; c := a + b;
c
endgroup
enddef;
```

如果像下面这样调用 foo

```
@ 测试变量宏 foo #                                                    39
\startMPpage
# 定义变量宏 foo @ <38>
show foo((1, 2), (3, 4));
\stopMPpage
```

在用 context 命令编译上述示例时，会在终端里输出

```
metapost        > trace > >> (4,6)
```

(4, 6) 便是上述宏 foo 的调用语句返回的结果，当然你顺便又学到了 MetaPost 的 show 原语[1]的基本用法。show 能够向终端输出对象的值，有助于 MetaPost 代码调试。还需要注意的是，foo 宏体里的变量 c 是局部变量，foo 返回其值。在 foo 定义的外部使用 c，是非法的。

MetaPost 为变量宏提供了更为简洁的语法，即 vardef，上述 foo 宏的定义可以写成

```
vardef foo(expr a, b) =
save c; pair c; c := a + b;
c
enddef;
```

亦即 vardef 会为其定义的宏自动构造局部作用域。

[1] 所谓原语，即语言最基本的部分，不能像宏那样展开。

2.6 曲折路径

以 xto 和 yto 为基础，可以构造出曲折的正则路径。例如，从点 a 出发，先沿 x 方向前进，然后再沿 y 方向抵达点 b，该路径可表达为 a xto b yto b。另一种路径，从点 a 出发，先沿 y 方向前进，然后再沿 x 方向抵达点 b，该路径可表达为 a yto b xto b。这两种路径的构造过程，可以表达为以下运算符宏：

```
@ 定义 xyto 和 yxto 宏 #                                                    40
tertiarydef a xyto b =
    a xto b yto b
enddef;
tertiarydef a yxto b =
    a yto b xto b
enddef;
```

以下示例用于演示 xyto 和 yxto 宏的用法并验证其效果：

```
@ 测试 xyto 和 yxto 宏 #                                                    41
\startMPpage
# 定义 xto 宏 @ <36>
# 定义 yto 宏 @ <37>
# 定义 xyto 和 yxto 宏 @ <40>
# 定义点 a, b 和 c @ <30>
# 画出 a, b, c 并予以标注 @ <32>
pickup pencircle scaled 1pt;
drawarrow a yxto b withcolor darkred;
drawarrow b xyto c withcolor darkgreen;
\stopMPpage
```

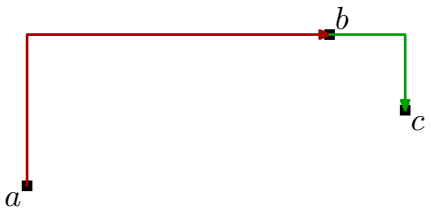


图 2.3　xyto 与 yxto 构造的路径

MetaPost 语法 t[a, b]，可用于从点 a 和 b 构成的直线上取点，当 t 为 0.5 时，所取的点即为 a 和 b 的中点。若取点 a 和 b 的中点 c，则可构造从 a 到 b 的更为曲折的正则路径。例如，从 a 沿 x 和 y 方向到 c，再沿 y 方向和 x 方向到 b，该过程可表示为「a xyto .5[a, b] yxto b」。MetaPost 语法允许小于 1 的正小数忽略整数部分，.5 即 0.5。同理，从 a 沿 y 和 x 方向到 c，在沿 x 和 y 方向到 b，该过程可表示为「a yxto .5[a, b] xyto b」。于是，可以再定义两个宏，xyxto 和 yxyto：

```
@ 定义 xyxto 和 yxyto 宏 #                                                  42
tertiarydef a xyxto b =
    begingroup
    # 获取 a 和 b 的中点 c @ <43>
    a xyto c yxto b
    endgroup
enddef;
tertiarydef a yxyto b =
```

```
    begingroup
    # 获取 a 和 b 的中点 c @ <43>
    a yxto c xyto b
    endgroup
enddef;
```

上述代码中引用的代码片段定义如下：

```
@ 获取 a 和 b 的中点 c #                                                    43
save pa, pb, c; path pa, pb; pair c;
pa := a; pb := b;
c := .5[(point (length pa) of pa), (point 0 of pb)];
```

为了方便后文在示例构造正则路径，我将上述定义的一些宏汇集到一起：

```
@ 正则路径宏 #                                                              44
# 定义 xto 宏 @ <36>
# 定义 yto 宏 @ <37>
# 定义 xyto 和 yxto 宏 @ <40>
# 定义 xyxto 和 yxyto 宏 @ <42>
```

以下示例用于演示 xyxto 和 yxyto 宏的用法并验证其效果：

```
@ 测试 xyxto 和 yxyto 宏 #                                                  45
\startMPpage
# 正则路径宏 @ <44>
# 定义点 a, b 和 c @ <30>
# 画出 a, b, c 并予以标注 @ <32>
pickup pencircle scaled 1pt;
drawarrow a xyxto b withcolor darkred;
drawarrow b yxyto c withcolor darkgreen;
\stopMPpage
```

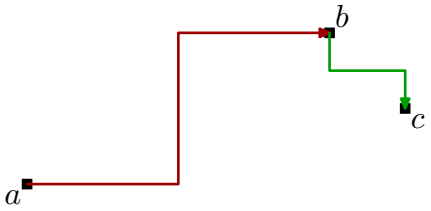


图 2.4　xyxto 与 yxyto 构造的路径

2.7 小结

从最为朴素的定义开始，xto 宏逐步进化，变得稳定，继而嬗变出 yto，xyto，yxto 等宏，在这样的过程里，也许你能隐约感受到，MetaPost 的宏颇似一柄锋利的匕首，一开始接触它，大概会担心不慎会伤到自己的手。然而只要你愿意尝试，愿意失败，也许自然就有机会成为庖丁，目无全牛——我希望以后的中学语文课本不要像侮辱「空穴来风」那样侮辱这个成语。

3 文本框

也许你已经意识到了，像 xyto 和 xyxto 之类的宏，可用于连接流程图中各个文本框所表达的过程。不过，我们还不知道如何用 MetaPost 语言表达文本框，本章致力于解决该问题。

3.1 包围盒

想必你还记得 1.6 节所述的 textext 宏，它能够将文字变为画面——picture 对象。MetaPost 以字符串类型亦即 string 类型表达文字。例如

```
@ 定义字符串 hello 并将其变为画面 hello_pic #                                    46
string hello;
picture hello_pic;
hello := "Hello!";
hello_pic := textext(hello);
```

用 draw 将 hello_pic 画出时，默认会将其中心与原点重合。MetaPost 的所有画面对象，其中心默认皆与原点重合，不过可以用 shifted 将其偏移到指定位置。

要表达流程图中的过程，只需在 textext 构造的文字画面外围画上边框，然后用?? 节所述的 image 宏将边框与文字画面合并为一个画面，便可得到文本框了。可是，如何得知画面的尺寸呢？

MetaFun 提供了 bbwidth 和 bbheight 宏，可用于计算路径或画面的宽度和高度，计算结果是数值——MetaPost 用 numeric 类型表达数值。这两个宏的用法如下：

```
@ 计算 hello_pic 的宽 w 和高 h #                                                 47
numeric w, h;
w := bbwidth hello_pic;
h := bbheight hello_pic;
```

基于 w 和 h 的值，便可为 hello_pic 构造边框：

```
@ 为 hello_pic 构造边框 hello_frame #                                            48
path hello_frame;
hello_frame := (-.5w, -.5h) -- (.5w, -.5h) -- (.5w, .5h) -- (-.5w, .5h) --cycle;
```

最后将 hello_pic 和 hello_frame 合并为一个画面：

```
@ 合并 hello_pic 和 hello_frame 为 framed_hello #                                49
picture framed_hello;
framed_hello := image(
    draw hello_pic;
    draw hello_frame;
);
```

将上述过程合并到一起，然后画出该过程构造的文本框：

```
@ 文本框示例 #                                                                   50
\startMPpage
# 定义字符串 hello 并将其变为画面 hello_pic @ <46>
# 计算 hello_pic 的宽 w 和高 h @ <47>
# 为 hello_pic 构造边框 hello_frame @ <48>
```

```
# 合并 hello_pic 和 hello_frame 为 framed_hello @ <49>
draw framed_hello scaled 7; % 放大 7 倍，看得更为清楚
\stopMPpage
```



图 3.1　文本框示例

实际上 MetaFun 提供了 boundingbox 宏，可以简化路径或画面的边框的构造过程，其用法如下：

```
@ boundingbox 示例 #                                                    51
\startMPpage
# 定义字符串 hello 并将其变为画面 hello_pic @ <46>
# 计算 hello_pic 的宽 w 和高 h @ <47>
path hello_frame;
hello_frame := boundingbox hello_pic;
# 合并 hello_pic 和 hello_frame 为 framed_hello @ <49>
draw framed_hello scaled 7; % 放大 7 倍，看得更为清楚
\stopMPpage
```

上述示例所得结果与图 **3.1** 近乎相同。

虽然有 boundingbox 宏，但基于 bbwidth 和 bbheight 手动构造边框的方式依然是必须的，因为后者有机会为边框四周增加一些留白。例如

```
@ 有留白的文本框示例 #                                                  52
\startMPpage
# 定义字符串 hello 并将其变为画面 hello_pic @ <46>
# 计算 hello_pic 的宽 w 和高 h @ <47>
path hello_frame;
numeric padding;
padding := 4pt;
hello_frame := (-.5w - padding, -.5h - padding)
               -- (.5w + padding, -.5h - padding)
               -- (.5w + padding, .5h + padding)
               -- (-.5w - padding, .5h + padding) --cycle;
# 合并 hello_pic 和 hello_frame 为 framed_hello @ <49>
draw framed_hello scaled 7; % 放大 7 倍，看得更为清楚
\stopMPpage
```

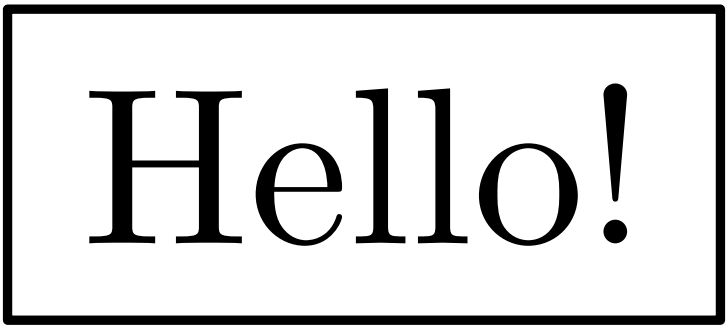


图 3.2　有留白的文本框

一旦理解了文本框对象的构造过程，便可将该过程封装为变量宏：

```
@ 定义 framed_text 宏 #                                                    53
vardef framed_text(expr s, padding) =
    save s_pic, w, h, s_frame;
    picture s_pic; numeric w, h; path s_frame;
    s_pic = textext(s);
    w := bbwidth s_pic;
    h := bbheight s_pic;
    s_frame := (-.5w - padding, -.5h - padding)
                -- (.5w + padding, -.5h - padding)
                -- (.5w + padding, .5h + padding)
                -- (-.5w - padding, .5h + padding) --cycle;
    % 返回一个 picture 对象
    image(draw s_pic; draw s_frame;)
enddef;
```

宏 framed_text 的用法如下：

```
@ 测试 framed_text 宏 #                                                    54
\startMPpage
# 定义 framed_text 宏 @ <53>
draw framed_text("hello", 4pt) withcolor darkred;
draw framed_text("world", 4pt) shifted (2cm, 0) withcolor darkblue;
\stopMPpage
```



图 3.3　文本框宏的测试结果

先不要急于向世界宣告，文本框站起来了！实际上 framed_text 宏还需要更多的参数，例如文字颜色、边框颜色以及边框线的粗度等，不过这些参数皆属于修饰成分，后续统一处理。

3.2 带后缀的名字

若以文本框表达程序流程图中的过程，文本框之间的连线的起点和终点应当位于文本框的边框上，为了方便起见，对于任一文本框，从其边框上可预先构造一些位置，我称之为锚点。

MetaPost 提供了一些基本的宏，可以算出路径或画面的包围盒边框上的一些锚点，诸如中心点、右上角点、右下角点、左下角点、左上角点等，例如

```
picture foo;
pair foo_center, foo_urcorner, foo_lrcorner, foo_llcorner, foo_ulcorner;
foo := framed_text("Foo", 4pt);
foo_center := center foo; % foo 的中心
foo_urcorner := urcorner foo; % foo 的右上 (upper right) 角点
foo_lrcorner := lrcorner foo; % foo 的右下 (lower right) 角点
foo_llcorner := llcorner foo; % foo 的左下 (lower left) 角点
foo_ulcorner := ulcorner foo; % foo 的左上 (upper left) 角点
```

基于上述锚点，用 t[a, b] 语法可以算出更多的锚点，例如

```
pair foo_东, foo_南, foo_西, foo_北;
foo_东:= .5[foo_urcorner, foo_lrcorner];
foo_南:= .5[foo_llcorner, foo_lrcorner];
foo_西:= .5[foo_llcorner, foo_ulcorner];
foo_北:= .5[foo_ulcorner, foo_urcorner];
```

不必吃惊，MetaPost 变量和宏的名字皆允许使用汉字，确切而言，是支持 UTF-8 编码的汉字。

我们一直都未曾讨论变量名和宏名的形式。倘若你学过 C 语言，只需将 MetaPost 的命名规则理解为 C 的命名规则，即名字只能以字母、下划线和数字构成，且不能以数字作为开头。MetaPost 只是比 C 有所宽容，将名字里的汉字也视为字母。

当你试图将上述锚点计算过程封装为宏时，就会发现这是一个不可能实现的任务。例如

```
def make_anchors(expr obj) =
    pair obj_center, obj_urcorner, ...
enddef;
```

当你开始考虑定义锚点变量时，就可以察觉到，在宏内无法直接用参数 `obj` 的名字作为锚点变量名字的前缀了。例如，向 `make_anchors` 传入文本框 `foo` 时，`make_anchors` 定义的锚点变量并非 `foo_center`，`foo_urcorner` 之类的点，而是名为 `obj_center`，`obj_urcorner` 之类的点。这是因为，在 `make_anchors` 的定义里，变量名字里的 `obj_` 前缀里的 `obj` 跟参数 `obj` 毫无关系，MetaPost 并不会用后者的实际名字替换前者，亦即以下宏调用语句

```
make_anchors(foo)
```

展开结果是

```
pair obj_center, obj_urcorner, ...
```

而非

```
pair foo_center, foo_urcorner, ...
```

若想让 `make_anchors` 定义中变量名前缀中的 `obj` 与宏参数 `obj` 建立连系，需要使用 `suffix` 类型的参数，并且将变量名前缀中的下划线 `_` 改为西文 `.` 符号，例如

```
def make_anchors(suffix obj) =
    pair obj.center, obj.urcorner, ...
enddef;
```

如此，在调用 `make_anchors` 并向其传入参数值 `foo` 时，展开结果便是

```
pair foo.center, foo.urcorner, ...
```

在 MetaPost 语言里，变量的名字实际上可由两部分构成，即 `tag.suffix`，亦即带有后缀（suffix）的名字（tag）。例如，`foo.center`，`foo` 是名字，而 `center` 是后缀。其实，我不知道该如何解释 `suffix` 类型的宏参数，只能寄希望于你学过某种主流的编程语言，可以将这种类型的参数想象成指针或者变量的引用，然后将已经用过了许多次的 `expr` 类型的宏参数想象为变量的值。

下面的代码，基于 `suffix` 类型的宏参数，定义宏 `make_anchors`，用于任意路径或画面对象构造 17 个常用的锚点：

```
@ 定义 make_anchors 宏 #                                                    55
def make_anchors(suffix obj) =
    pair obj.中央, obj.东北, obj.西北, obj.西南, obj.东南;
```

```
    pair obj.子门, obj.午门, obj.卯门, obj.酉门;
    pair obj.丑门, obj.寅门, obj.辰门, obj.巳门, obj.未门, obj.申门, obj.戌门, obj.亥门;
    obj.中央:= center obj;
    obj.东北:= urcorner obj; obj.西北:= ulcorner obj;
    obj.西南:= llcorner obj; obj.东南:= lrcorner obj;
    obj.子门:= .5[obj.西北, obj.东北]; obj.午门:= .5[obj.西南, obj.东南];
    obj.卯门:= .5[obj.东南, obj.东北]; obj.酉门:= .5[obj.西南, obj.西北];
    obj.丑门:= .5[obj.子门, obj.东北]; obj.寅门:= .5[obj.卯门, obj.东北];
    obj.辰门:= .5[obj.卯门, obj.东南]; obj.巳门:= .5[obj.午门, obj.东南];
    obj.未门:= .5[obj.午门, obj.西南]; obj.申门:= .5[obj.酉门, obj.西南];
    obj.戌门:= .5[obj.酉门, obj.西北]; obj.亥门:= .5[obj.子门, obj.西北];
enddef;
```

你需要谅解我使用了古奥的十二地支作为锚点的名字，并非我故弄玄虚，而是我实在不如古人善于取名。以下为宏 make_anchors 的测试代码：

```
@ 测试 make_anchors 宏 #                                                    56
\startMPpage
# 定义 framed_text 宏 @ <53>
# 定义 make_anchors 宏 @ <55>
picture foo;
foo := framed_text("Foo", 4pt) scaled 7;
make_anchors(foo);
% 画出 foo 及其锚点
draw foo withcolor darkgray;
pickup pensquare scaled 8pt;
for i = foo.中央, foo.东北, foo.东南, foo.西南, foo.西北:
    draw i withcolor darkblue;
endfor;
pickup pencircle scaled 8pt;
for i = foo.子门, foo.午门, foo.卯门, foo.酉门:
    draw i withcolor darkred;
endfor;
pickup pencircle scaled 6pt;
for i = foo.丑门, foo.寅门, foo.辰门, foo.巳门, foo.未门, foo.申门, foo.戌门, foo.亥门:
    draw i withcolor darkgreen;
endfor;
\stopMPpage
```

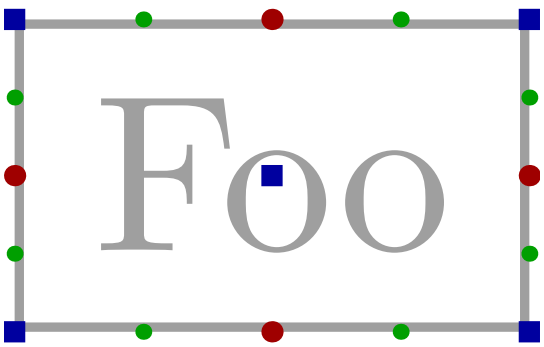


图 3.4　锚点示例

以十二地支命名一个文本框上的锚点，可能是危险的行为，会让一些人觉得不舒服，因为他们一向觉得东方传统文化象征着腐朽，该扫进历史垃圾堆里的。不过，对我而言，我更不能适应用英文的十二个月份来命名，而这些月份的名字也无非是他们的地支的名字罢了。

有文本框及其锚点，结合正则路径，便可画一些简单的流程图了，例如

```
@ foo -> bar #                                                    57
\startMPpage
# 正则路径宏 @ <44>
# 定义 framed_text 宏 @ <53>
# 定义 make_anchors 宏 @ <55>
picture foo, bar;
foo := framed_text("Foo", 4pt);
bar := framed_text("Bar", 4pt) shifted (6cm, 2cm);
make_anchors(foo); make_anchors(bar);
draw foo; draw bar;
drawarrow foo.卯门 xyxto bar.酉门 withpen pencircle withcolor darkred;
\stopMPpage
```

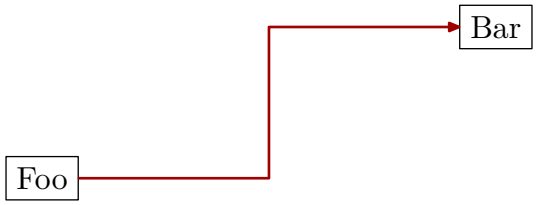


图 3.5　foo → bar

3.3 类

上一节最后的示例虽然是成功的，但绘图代码并不优雅，主要问题在于，文本框与其锚点集合的构造过程是分离的，当文本框发生移动后，锚点需要显式重构。由该问题引申出来的一个问题是，对一个文本框重复调用 `make_anchors` 宏时，所有的锚点会被重复声明。

利用带后缀的名字，模拟一种在面向对象编程语言里称为「类」的机制，可有效解决上述问题。首先，为文本框及其锚点定义一个统一声明的宏：

```
def framed_text_class(suffix name) =
    picture name;
    pair name.中央, name.东北, name.西北, name.西南, name.东南;
    pair name.子门, name.午门, name.卯门, name.酉门;
    pair name.丑门, name.寅门, name.辰门, name.巳门, name.未门, name.申门, name.戌门, name.亥门;
enddef;
```

MetaPost 语言为带后缀的名字提供了专门的 `for` 语法，即 `forsuffixes`，基于该语法可以遍历一些带后缀的名字甚至后缀本身，故而上述代码可略微简化为

```
@ 定义 framed_text_class 宏 #                                     58
def framed_text_class(suffix name) =
    picture name;
    forsuffixes i = 中央, 东北, 西北, 西南, 东南,
                    子门, 午门, 卯门, 酉门, 丑门, 寅门, 辰门, 巳门, 未门, 申门, 戌门, 亥门:
        pair name.i;
    endfor;
```

```
enddef;
```

像下面这样调用 framed_text_class 宏，相当于声明了文本框类的一个对象 foo：

```
framed_text_class(foo);
```

此时，foo 这个名字不仅表达一个文本框的本体，并且它也有了一些属性，即一组锚点。

基于类的概念，便可将文本框对象及其锚点的声明和赋值过程分离，从而可以解决 make_anchors 宏多次调用导致的锚点变量重复声明的问题，故而 make_anchors 宏可简化为

```
@ make_anchors 宏版本之二 #                                                    59
def make_anchors(suffix obj) =
    obj.中央:= center obj;
    obj.东北:= urcorner obj; obj.西北:= ulcorner obj;
    obj.西南:= llcorner obj; obj.东南:= lrcorner obj;
    obj.子门:= .5[obj.西北, obj.东北]; obj.午门:= .5[obj.西南, obj.东南];
    obj.卯门:= .5[obj.东南, obj.东北]; obj.酉门:= .5[obj.西南, obj.西北];
    obj.丑门:= .5[obj.子门, obj.东北]; obj.寅门:= .5[obj.卯门, obj.东北];
    obj.辰门:= .5[obj.卯门, obj.东南]; obj.巳门:= .5[obj.午门, obj.东南];
    obj.未门:= .5[obj.午门, obj.西南]; obj.申门:= .5[obj.酉门, obj.西南];
    obj.戌门:= .5[obj.酉门, obj.西北]; obj.亥门:= .5[obj.子门, obj.西北];
enddef;
```

宏 framed_text 的定义也要作一些修改，除了省去文本框对象的声明之外，在构造出文本框对象后，顺便算出其锚点：

```
@ framed_text 宏版本之二 #                                                     60
def framed_text(suffix obj)(expr s, padding) =
    save s_pic, w, h, s_frame;
    picture s_pic; numeric w, h; path s_frame;
    s_pic = textext(s);
    w := bbwidth s_pic;
    h := bbheight s_pic;
    s_frame := (-.5w - padding, -.5h - padding)
                -- (.5w + padding, -.5h - padding)
                -- (.5w + padding, .5h + padding)
                -- (-.5w - padding, .5h + padding) --cycle;
    obj := image(draw s_pic; draw s_frame;);
    make_anchors(obj);
enddef;
```

注意，上述宏定义出现了两种宏参数并存的情况，这些参数需要按类型分开写，每种类型的参数放在单独的小括号里。当文本框的位置发生变化时，其锚点应当自动更新，要解决该问题，需要为文本框的定位定义一个宏，例如

```
@ 定义 put_framed_text 宏 #                                                    61
def put_framed_text(suffix obj)(expr position) =
   % 先将 obj 复位，即令其中心与原点重合
   obj := obj shifted -obj.中央;
   % 再将 obj 移动到指定位置
   obj := obj shifted position;
```

```
    % 更新锚点集合
    make_anchors(obj);
enddef;
```

基于上述定义的宏，上一节的流程图示例便可以简化为

```
@ foo -> bar 之二 #                                                    62
\startMPpage
# 正则路径宏 @ <44>
# 定义 framed_text_class 宏 @ <58>
# make_anchors 宏版本之二 @ <59>
# framed_text 宏版本之二 @ <60>
# 定义 put_framed_text 宏 @ <61>
framed_text_class(foo); framed_text(foo, "Foo", 4pt);
framed_text_class(bar); framed_text(bar, "Bar", 4pt); put_framed_text(bar, (6cm, 2cm));
draw foo; draw bar;
drawarrow foo.卯门 xyxto bar.酉门 withpen pencircle withcolor darkred;
\stopMPpage
```

上述示例所得结果与图 **3.5** 相同，其代码简化之处不在于代码量变少，实际上代码量反而有所增多了，而是在于语义得到简化，例如，你已经看不到锚点的构造和更新过程了。

3.4 宏参数

现在有必要对 MetaPost 宏参数作一些更深入的探讨了。现在，你已经见识过了三种类型的宏参数了，即 `expr`，`suffix` 以及 `text`。你可能会抗议，因为你似乎从未见过 `text` 类型的宏参数。实际上是见过的，还记得 **2.3** 节用到的 `hide` 宏吗？它的参数类型便是 `text`。

直觉上，你可以将 `expr` 类型理解为可以赋给变量的一切事物，将 `suffix` 类型理解为名字，将 `text` 类型理解为一段 MetaPost 代码。这样说，可能无助于你真正理解它们。更确切一些，向宏传入 `expr` 类型的参数，传的是值。向宏传入 `suffix` 类型的参数，传的是仅仅是名字，无需知道名字所指的实体是什么，如同你听到一个路人的名字，这个名字对你而言，仅仅只是个名字而已。向宏传入 `text` 类型的参数，传的是 MetaPost 语句。

在宏定义里，每种类型的参数需要单独放在至少一对小括号里。同类型的参数可以有多个，以西文逗号间隔，例如

```
def foo(expr a, b)(suffix c, d)(expr e) (text f, g) =
    ... ... ...
enddef;
```

下面是一组 `foo` 宏调用语句，它们都是合法且等效的：

```
foo(a, b, c, d, e)(f)(g);
foo(a, b)(c, d)(e)(f)(g);
foo(a, b, c)(d, e)(f)(g);
```

需要注意的是 `text` 类型的参数，若存在多个该类型的参数值，应当用小括号将其隔离开，若以西文逗号作为参数值的间隔，则逗号会被视为参数值的一部分。实际上，如果一个宏定义需要 `text` 类型的参数，在很多情况下，该类型的参数只有一个，而且放在参数表的末尾，且不需要括号：

```
def foo(expr a) text b =
    draw a b;
```

```
enddef;
```

可以像下面这样调用上述重新定义的 foo 宏：

```
foo((0, 0) -- (3cm, 2cm)) withpen pensquare scaled 1pt;
```

foo 接受的第一个参数值是一条路径，第二个参数值则是

```
withpen pensquare scaled 1pt;
```

当 text 类型的参数没有括号时，宏会抓取后面出现的一切，直至遇到分号为止，但分号不会被纳入。

关于宏参数还需要注意的是，当宏只有 1 个参数时，则其定义和调用语句可以不用括号。例如

```
def foo expr a = ... enddef;
```

且以下调用语句皆合法：

```
foo(a); foo a;
```

借助 text 类型参数，能够让 **3.3** 节中 framed_text_class 在语义上变得更优雅一些，例如令其支持一次声明多个文本框对象：

```
@ framed_text_class 宏版本之二 #                                                    63
def framed_text_class text names =
    forsuffixes i = names:
        picture i;
        forsuffixes j = 中央, 东北, 西北, 西南, 东南,
                        子门, 午门, 卯门, 酉门, 丑门, 寅门, 辰门, 巳门, 未门, 申门, 戌门, 亥门:
            pair i.j;
        endfor;
    endfor;
enddef;
```

现在，若要一次性声明 foo 和 bar 两个文本框对象，只需

```
framed_text_class foo, bar;
```

上述对 MetaPost 宏参数类型的讨论并不精确，也不尽全面，但是对于理解本文档迄今已经出现的所有示例代码是够用的。这些参数类型的用法会在后文一些示例中重复出现，届时我会酌情作更为具体一些的介绍。

3.5 定位

为了在定义文本框时能够直接确定其位置，也为了能随时改变文本框的位置，用一个名字更短的宏 at 代替 shifted 是有必要的，亦即

```
@ 定义 at 宏 #                                                                      64
def at text position =
    shifted (if (path (position)) or (picture (position)): center (position) else: (position) fi)
enddef;
```

注意，在 at 宏体里，参数 position 加了小括号，此举是为了防止 at 宏展开后，position 所指代的语句与周围语句产生错误的结合。此外，at 宏也颇为稳健，若其参数为路径或画面，可取其中心作为定位基准。还需要注意的是，MetaPost 的条件表达式的写法，大概只有宏语言能支持这种写法。

重新定义 framed_text 宏，为其增加一个 text 类型的参数，以接受文本框的位置信息：

```
@ framed_text 宏版本之三 #                                                    65
def framed_text(suffix obj)(expr s, padding) text somewhere =
    begingroup
    save s_pic, w, h, s_frame;
    picture s_pic; numeric w, h; path s_frame;
    s_pic := textext(s);
    w := bbwidth s_pic;
    h := bbheight s_pic;
    s_frame := (-.5w - padding, -.5h - padding)
                -- (.5w + padding, -.5h - padding)
                -- (.5w + padding, .5h + padding)
                -- (-.5w - padding, .5h + padding) --cycle;
    obj := image(draw s_pic; draw s_frame;) somewhere;
    make_anchors(obj);
    endgroup
enddef;
```

本次定义的 framed_text 宏仅仅是用参数 somewhere 取代了上一个版本里 shifted 语句，因为 somewhere 可以是 at 语句，例如：

```
framed_text_class foo;
framed_text(foo, "Foo", 4pt) at (3cm, 2cm);
```

at (3cm, 2cm) 会被 framed_text 宏捕捉到，作为其参数 somewhere 的值。

重新定义 put_framed_text 宏，将文本框的位置信息也表示为 text 类型的参数：

```
@ put_framed_text 宏的新版本 #                                                66
def put_framed_text(suffix obj) text somewhere =
   obj := obj shifted -obj.中央;
   obj := obj somewhere;
   make_anchors(obj);
enddef;
```

改进后的 put_framed_text 宏，用法如下：

```
framed_text_class foo;
framed_text(foo, "Foo", 4pt) at (3cm, 2cm);
% 将 foo 的位置修改为 (5cm, 1cm)
put_framed_text(foo) at (5cm, 1cm);
```

至此 **3.3** 节最后的的示例可进化为

```
@ foo -> bar 之三 #                                                           67
\startMPpage
# 正则路径宏 @ <44>
# make_anchors 宏版本之二 @ <59>
```

```
# framed_text_class 宏版本之二 @ <63>
# 定义 at 宏 @ <64>
# framed_text 宏版本之三 @ <65>
framed_text_class foo, bar;
framed_text(foo, "Foo", 4pt); framed_text(bar, "Bar", 4pt) at (6cm, 2cm);
draw foo; draw bar;
drawarrow foo.卯门 xyxto bar.酉门 withpen pencircle withcolor darkred;
\stopMPpage
```

历经两次进化，文本框的构造过程有了足够简洁的语义了，只是所用的宏名有些冗长，暂时需要忍耐一下。名可名，非恒名，在适当的时机，名字也会发生进化。

3.6 隐患

实际上，framed_text 从第二个版本便存在了隐患，因为它含有 suffix 类型的参数。所有含有 suffix 参数的宏，若其定义中存在定义变量的语句，无论变量是局部的还是全局的，都存在隐患。为了让你看清这种隐患，我杜撰了下面这个宏：

```
def foo(suffix obj) =
    begingroup
    save a; numeric a; a := 3;
    show a, xpart obj, ypart obj;
    endgroup
enddef;
```

倘若像下面这样调用 foo 宏，

```
pair a; a := (2, 3);
foo(a);
```

便会触发 foo 的隐患，该宏定义无法通过 context 的编译，原因是，传给 foo 的 a，会直接进入 foo 的定义，亦即 foo 定义里的 obj 此时便是 a，而它与宏内定义的局部变量 a 同名且被后者覆盖，而后者是数值变量，不支持 xpart 和 ypart 操作。简而言之，上述宏调用语句 foo(a) 的展开结果是

```
begingroup
save a; numeric a; a := 3;
show a;
show a, xpart a, ypart a;
endgroup
```

该如何避免这种隐患呢？比较简单的做法是，将宏内变量的名字取得复杂一些，降低同名冲突概率，例如

```
def foo(suffix obj) =
    begingroup
    save _a_; numeric _a_; _a_ := 3;
    show _a_, xpart obj, ypart obj;
    endgroup
enddef;
```

然后规定以下划线开始和结束的变量名为保留名，调用宏时，禁止向其传递这类名字。但这种办法只是降低了触发隐患的概率，不能从根本上解决问题。更为稳健的做法是，将需要定义变量的部分放在一个不含 `suffix` 参数的宏内，例如

```
def foo(suffix obj) = bar(obj); enddef;
def bar(expr obj) =
    begingroup
    save a; numeric a; a := 3;
    show a, xpart obj, ypart obj;
    endgroup
enddef;
```

用上述第二种方法，可将 `framed_text` 宏修改为更为稳健的版本：

```
def framed_text(suffix obj)(expr s, padding) text somewhere =
    obj := make_framed_text(s, padding) somewhere;
    make_anchors(obj);
enddef;
vardef make_framed_text(expr s, padding) =
    save s_pic, w, h, s_frame;
    picture s_pic; numeric w, h; path s_frame;
    s_pic := textext(s);
    w := bbwidth s_pic;
    h := bbheight s_pic;
    s_frame := (-.5w - padding, -.5h - padding)
                -- (.5w + padding, -.5h - padding)
                -- (.5w + padding, .5h + padding)
                -- (-.5w - padding, .5h + padding) --cycle;
    image(draw s_pic; draw s_frame;)
enddef;
```

3.7 小结

本章着力于阐明 `suffix` 和 `text` 类型的宏参数的用途。至本章为止，本文档所述的 MetaPost 语法大概已囊括 MetaPost 语言核心部分的六七成，足以让你一窥 MetaPost 语言之风采。倘若你想与我继续前行，路途依然曲折，不过可以试着让曲折变得柔和一些。

4 刚柔相济

使用第 2 章定义的 xyto，yxto，xyxto 以及 yxyto 等宏构造的路径，在转折处皆是直角，成于简单，失于硬朗。在构造这些路径的过程中，可以考虑用弧状曲线段构造出圆角转折的效果，从而在刚硬的氛围里掺入一些柔和的气息。最重要的是，在尝试实现这个想法的过程中，可以熟悉 MetaPost 曲线路径的构造语法。

4.1 插值与逼近

用 MetaPost 语言构造曲线路径有三种方法。第一种在 1.4 节中已经用过了，对于已知数学解析式的曲线，例如圆、抛物线、双曲线、多项式曲线等，只需基于其解析式，对曲线离散采样，用直线段连接采样点便可模拟出曲线了。

第二种曲线路径构造方法是构造穿过某些指定点的曲线，这种曲线称为插值曲线。MetaPost 的.. 语法可构造插值曲线，例如

```
@ 曲线插值示例 #                                                    68
\startMPpage
# 定义点 a, b 和 c @ <30>
path p;
p := a .. b .. c;
draw p withpen pencircle withcolor darkred;
# 画出 a, b, c 并予以标注 @ <32>
\stopMPpage
```

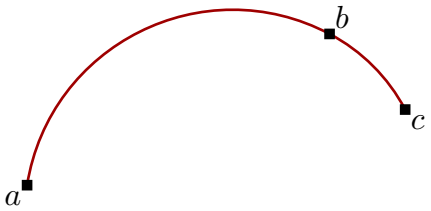


图 4.1　插值三个点的曲线段

第三种曲线路径构造方法是让曲线逼近指定的点，即控制点，而非穿过它们。例如

```
@ 曲线逼近示例 #                                                    69
\startMPpage
# 定义点 a, b 和 c @ <30>
path p;
p := a .. controls b .. c;
draw p withpen pencircle withcolor darkred;
# 画出 a, b, c 并予以标注 @ <32>
\stopMPpage
```



图 4.2　插值于 a 和 c 且逼近 c 的曲线段

含有一个控制点的曲线段是二次曲线，即抛物线。MetaPost 允许一段曲线最多只能有两个控制点，用于构造三次曲线，例如

```
@ 三次曲线逼近示例 #                                                    70
\startMPpage
# 定义点 a, b 和 c @ <30>
pair d; d := (8cm, 3cm);
path p; p := a .. controls b and c .. d;
draw p withpen pencircle withcolor darkred;
# 画出 a, b, c 并予以标注 @ <32>
draw d; label.bot("$d$", d);
\stopMPpage
```

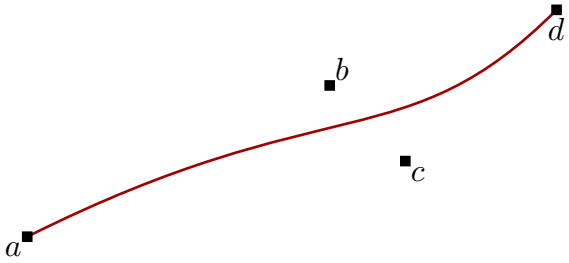


图 4.3　插值于 a 和 d 且逼近 b 和 c 的曲线段

倘若你对 MetaPost 构造插值和逼近曲线的原理感兴趣，可以搜索有关 Bézier 曲线的资料。这种曲线发现于上个世纪 60 年代，并催生出一门新的学科——计算机辅助几何设计。不过，对于本文档要解决的问题而言，只需掌握 MetaPost 的曲线路径连接符.. 和 `controls ... and ...` 语法即可。

4.2 圆角

若是仅仅想让正则路径变得柔和一些，仅用插值曲线便可达成目的。下面逐步手工构造一条带有圆角过渡的路径，以此揭示正则路径的柔化原理。首先，用 `xyto` 宏构造一条直角路径 `p`：

```
@ 构造直角路径 p #                                                      71
pair a, b;
a := (0, 0); b := (6cm, 3cm);
path p; p := a xyto b;
```

`p` 由三个节点构成，取其第 2 个节点，亦即序号为 1 的节点：

```
@ 取 p 的第 2 个节点，表示 c，并将其画出来 #                            72
pair c;
c := point 1 of p;
draw c withpen pensquare scaled 4pt withcolor darkred;
```

然后在路径 `a -- c` 的末端 `5mm` 处取点 `d`，在路径 `c -- b` 的始端 `5mm` 处取点 `e`：

```
@ 在路径 p 上 c 点前后各取一点，分别为 d 和 e #                        73
pair d, e;
d := point -5mm on (a -- c);
e := point  5mm on (c -- b);
```

上述代码出现了之前未遇到的语句 `point ... on ...`，这是 MetaFun 宏，可以在给定的路径上按方向和长度取点。当 `point` 之后的距离值为正值时，表示沿着路径始端向末端方向按所走距离取点，而若

距离值为负值，则表示沿路径末端向始端方向按所走距离取点。例如「point -5mm on (a -- c)」，所取的点是从 c 出发，沿 c -- a 方向上位于 5mm 处的点。

现在基于点 a，d，c，e 以及 b，可以构造有圆角过渡的路径 q，并将其画出：

```
@ 构造含有圆角过渡的路径 #                                                    74
\startMPpage
# 正则路径宏 @ <44>
# 构造直角路径 p @ <71>
# 取 p 的第 2 个节点，表示 c，并将其画出来 @ <72>
# 在路径 p 上 c 点前后各取一点，分别为 d 和 e @ <73>
path q;
q := (a -- d) .. (e -- b);
drawpath q;
drawpoints q;
% 作一些标注
label.bot("$a$", a); label.rt("$b$", b);
label.rt("$c$", c); label.bot("$d$", d); label.rt("$e$", e);
\stopMPpage
```



图 4.4　含有圆角过渡的路径

上述的代码中使用的 drawpath 与 drawpoints 皆为 MetaFun 宏，它们有助于呈现路径的细节，前者默认以浅灰色的粗线画出路径，后者画出路径上的所有节点。

注意，「(a -- d) .. (e -- b)」与「a -- d .. e -- b」是两条不同的路径，前者如图 4.4 所示，而后者等效于「a -- d -- e -- b」。曲线路径连接符 .. 在连接两个点时，所构造的曲线路径会退化为直线段，而当它连接两条路径时，所构造的曲线路径与所连接的路径在连接点处相切。

4.3 数字后缀

在上一节的示例中，最让我不愉快的事情是给变量取名字，诸如 d 和 e 这样的名字，原本我想用的名字是 c1 和 c2，然而在 MetaPost 语言里，数字后缀有特殊含义。例如 foo1，foo2，foo32 等，这些名字可等效写成 foo[1]，foo[2]，foo[32] 这类名字，在主流编程语言里对应的概念是数组或序列。再例如 foo1.a，foo32bar，可等效写为 foo[1].a，foo[32]bar。

这类以数字为后缀的名字在声明时，需要用 [] 符号，例如

```
pair c[];
```

一个名字可以有多个后缀。对于带有多个数字后缀的名字，有几个数字后缀，就要用几对方括号。例如带有两个数字后缀的名字，可以声明为二维数组的形式：

```
picture c[][];
```

同理，也能构造更高维的数组：

```
picture c[][][][][][][];
```

像 c[1][2] 这样名字，你也可以将其等效写为 c1[2] 或 c[1]2。注意，c，c[] 以及 c[][] 并不重名，亦即以下语句是合法的：

```
pair c;
pair c[];
pair c[][];
```

但是，你不能直接以数字为后缀声明变量，例如以下语句是非法的：

```
pair c1;
```

基于数字后缀，在上一节的示例里，在路径 p 上 c 点前后各取一点，可用 c[1] 和 c[2] 表示，亦可简写为 c1 和 c2，例如

```
pair c[];
c1 := point -5mm on (a -- c);
c2 := point  5mm on (c -- b);
```

如果你希望将一个数组作为参数传给宏，可以用 suffix 类型的宏参数，例如

```
def foo(suffix objs)(expr n) =
    for i = 1 upto n:
        show objs[i];
    endfor;
enddef;
% 将数组传给 foo
pair a[];
a1 := (0, 1); a2 := (3, 4); a3 := (-1, 5);
foo(a);
```

需要注意的是，MetaPost 无法得知一个名字后面究竟有多少后缀，在将其作为数组传给宏时，至少需要显式传入数组的长度，此外也应当约定，数组元素的下标是从 0 还是从 1 开始。

4.4 路径自动柔化

基于数字后缀，可以将路径所有节点保存为一个序列，然后除始末节点外，每个中间节点可以构造由两个点构成的子序列，即节点前后所取的辅助点，它们会作为圆角过渡区域的起点和终点。该过程如下：

```
@ 设圆角半径为 r，为路径 p 构造节点序列 c[] 和辅助点序列 h[][] #                    75
numeric m, n;
pair c[], h[][];
n := length p; m := n - 1;
for i = 0 upto n:
    c[i] := point i of p;
endfor;
h[0][0] := c[0]; h[0][1] := c[0];
for i = 1 upto m:
    h[i][0] := point -r on (c[i - 1] -- c[i]);
```

```
    h[i][1] := point  r on (c[i] -- c[i + 1]);
endfor;
h[n][0] := c[n]; h[n][1] := c[n];
```

然后基于 **4.2** 节所述的方法，构造含有圆角的路径 `q`：

```
@ 构造含有圆角的路径 q #                                                    76
path q;
q := c[0] -- h[1][0];
for i = 1 upto m:
    q := q .. (h[i][1] -- h[i + 1][0]);
endfor;
```

将上述过程合并起来，便可得到可将正则路径柔化的宏：

```
@ 定义 smooth_regular_path 宏 #                                             77
vardef smooth_regular_path(expr p, r) =
    save c, h, q;
    # 设圆角半径为 r，为路径 p 构造节点序列 c[] 和辅助点序列 h[][] @ <75>
    # 构造含有圆角的路径 q @ <76>
    q
enddef;
```

以下为 `smooth_regular_path` 宏的测试代码：

```
@ 测试 smooth_regular_path 宏 #                                             78
\startMPpage
# 正则路径宏 @ <44>
# 定义 smooth_regular_path 宏 @ <77>
pair a, b; a := (0, 0); b := (6cm, 3cm); label.lft("$a$", a); label.rt("$b$", b);
path p, q; p := a xyxto b; q := smooth_regular_path(p, 5mm);
drawpath q; drawpoints q;
\stopMPpage
```

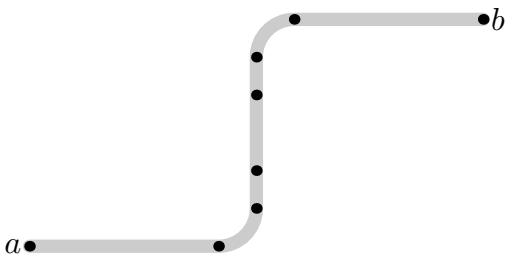


图 4.5　自动构造含有圆角过渡的路径

4.5 节点切向

在构造曲线时，MetaPost 允许以方向修饰曲线段起点和终点，使得曲线段在节点处与指定方向相切，从而控制曲线形状。

方向是单位向量，数据类型是 `pair`。MetaPost 默认提供了 4 个标准方向，`up`，`right`，`down` 和 `left`，对应的向量分别是 `(0, 1)`，`(1, 0)`，`(0, 1)` 和 `(-1, 0)`。此外，可以构造任意向量，然后用 MetaPost 宏 `unitvector` 将其转化为单位向量。

以下示例构造了一条插值曲线，每个节点处皆设定了切向：

```
@ 以方向控制曲线形状 #                                                    79
\startMPpage
# 定义点 a, b 和 c @ <30>
path p;
p := a{up} .. b{right} .. c{unitvector(1, 1)};
draw p; drawpoints p;
# 画出 a, b, c 并予以标注 @ <32>
\stopMPpage
```

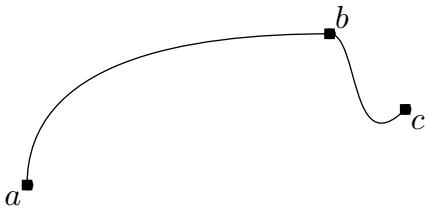


图 4.6　以节点切向控制曲线形状

利用节点切向，可无需构造节点序列 `c[]` 和辅助点序列 `h[][]`，从而简化路径自动柔化过程：

```
@ 改进的 smooth_regular_path 宏 #                                         80
vardef smooth_regular_path(expr p, r) =
    save a, b, c, n, m, q; pair a, b, c, c[]; numeric n, m; path q;
    n := length p; m := n - 1;
    q := point 0 of p;
    for i = 1 upto m:
        a := point (i - 1) of p;
        c := point i of p;
        b := point (i + 1) of p;
        c1 := point -r on (a -- c);
        c2 := point  r on (c -- b);
        q := (q -- c1) .. c2{unitvector(b - c)};
    endfor;
    q := q -- point n of p;
    q
enddef;
```

以下代码用于测试新定义的 `smooth_regular_path` 宏：

```
@ 测试新的 smooth_regular_path 宏 #                                       81
\startMPpage
# 正则路径宏 @ <44>
# 改进的 smooth_regular_path 宏 @ <80>
pair a, b; a := (0, 0); b := (6cm, 3cm); label.lft("$a$", a); label.rt("$b$", b);
path p, q; p := a xyxto b; q := smooth_regular_path(p, 5mm);
drawpath q; drawpoints q;
\stopMPpage
```

结果与图 **4.5** 相同。

4.6 消除共线点

也许你已经注意到了，`xyxto` 和 `yxyto` 构造的正则路径，路径的中段存在一个冗余节点，例如

```
@ 含有冗余节点的正则路径示例 #                                                82
\startMPpage
# 正则路径宏 @ <44>
pair a, b;
a := (0, 0); b := (6cm, 3cm); label.lft("$a$", a); label.rt("$b$", b);
path p; p := a xyxto b;
drawpath p; drawpoints p;
label.rt("$c$", .5[a, b]);
\stopMPpage
```

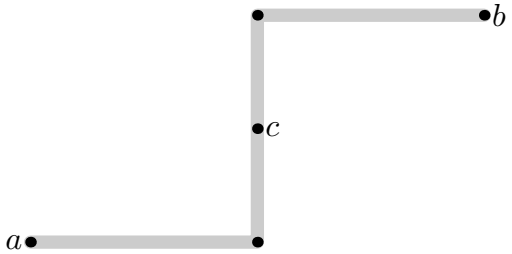


图 4.7　含有冗余节点的路径

图 **4.7** 中的点 c 便是冗余节点，因为对于它所在的路径子段是直线段，该直线段由其两个端点便可定义，无需 c 的存在。

含有冗余节点的正则路径，在其自动柔化处理过程中，每个冗余节点皆会被替换为两个新节点，从而造成更多冗余节点的出现。虽然这些冗余节点不会影响路径的形状，但是尽力让世界能得以简化是我们的本分，此外设法消除冗余节点，在学习 MetaPost 语言的角度上也颇有意义。

在处理正则路径柔化时，我们已经熟悉了在路径节点的遍历过程中构造新路径的方法，只需故技重演，便可消除正则路径上的冗余节点，唯一的难点在于，给定三个点，如何判断它们共线。解决这个问题，需要一点向量代数的知识，即两个单位向量的点积为 1 表示两个向量平行，为 0 则表示两个向量垂直。MetaPost 提供了 `dotprod` 运算符宏，可以计算两个向量的点积，故而三点共线检测过程可定义为

```
@ 定义 is_colinear 宏 #                                                       83
vardef is_colinear(expr a, b, c) =
    save result, v; boolean result; pair v[];
    v[1] := unitvector(b - a); v[2] := unitvector(c - b);
    if abs((v[1] dotprod v[2]) - 1) < 1e-6:
        result := true;
    else:
        result := false;
    fi;
    result
enddef;
```

上述代码使用了之前未曾用的类型 `boolean`，即布尔类型。该类型只有两个值 `true` 和 `false`。另外需要注意的是，因为计算机存在浮点误差，故而判断两个向量的点积是否为 1，不能直接与 1 作相等比较，而是取点积结果与 1 的差值，然后与一个微小的量作比较，若前者小于后者，则可认为两个向量的点积近似为 1。在上述代码中，若 `v[1]` 和 `v[2]` 的点积近似为 1，则判定 `a`，`b` 和 `c` 共线。

基于 `is_colinear` 宏，可将正则路径冗余节点消除过程定义为：

```
@ 定义 simplify_regular_path 宏 #                                             84
vardef simplify_regular_path(expr p) =
    save q, n, m, a, b, c;
```

```
    path q; numeric n, m; pair a, b, c;
    n := length p;
    m := n - 1;
    q := point 0 of p;
    for i = 1 upto m:
        a := point (length q) of q;
        b := point i of p;
        c := point (i + 1) of p;
        if not is_colinear(a, b, c):
            q := q -- b;
        fi;
    endfor;
    q := q -- point n of p;
    q
enddef;
```

上述代码的核心内容是，遍历 p 上除了端点之外的所有节点，在该过程中，若当前节点与 q 的末端节点以及当前节点的下一个节点不共线，则将当前节点添加的路径 q。注意上述代码使用了布尔值取反运算符 not。MetaPost 还提供了 and 和 or，分别表示「与」和「或」运算，后文用到它们时再加以解释。

以下代码用于测试 simplify_regular_path 宏：

```
@ 测试 simplify_regular_path 宏 #                                    85
\startMPpage
# 正则路径宏 @ <44>
# 定义 is_colinear 宏 @ <83>
# 定义 simplify_regular_path 宏 @ <84>
pair a, b; a := (0, 0); b := (6cm, 3cm);
path p, q; p := a xyxto b; q := simplify_regular_path(p);
% 将 p 向左偏移，然后将其与 q 并列画出
p := p shifted (-6cm, 0);
drawpath p; drawpoints p; drawpath q; drawpoints q;
\stopMPpage
```

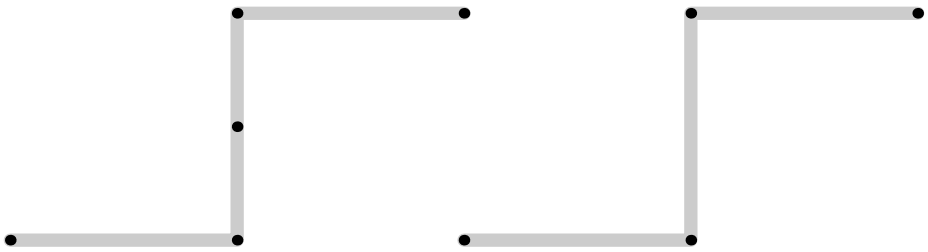


图 4.8　消除冗余节点

4.7 小结

本章引入的 MetaPost 语法有些繁杂，所构造的示例也颇具跳跃性，值得多花一些时间详加揣摩。路不妨走得正直一些，但是遇到拐角时，想要变得圆滑一些，本非易事，除了需要善于总结过去，展望未来，还需要让自己的身体时刻保持轻盈。

5 骗术

流程图中各个文本框之间的连接路径可能会发生交叉，如同现实中道路的交叉，存在应该建成十字路口，还是建造立交桥的选择，我选后者。但是，MetaPost 只支持二维绘图，无法实现三维立交，故而只能用示意风格，即位于底层的路径在交点处断开，位于上层的路径则保持畅通。如何用 MetaPost 实现这种路径交叉风格呢？

5.1 交点

最聪明的方法是算出两条路径的交点，然后在交点处作文章。MetaPost 语言提供了运算符宏 `intersectionpoint`，可用于计算两条路径的交点。例如，对以下路径

```
@ 定义插值曲线路径 p 和 q #                                                    86
path p, q;
p := (0, 0) .. (3cm, 1cm) .. (5cm, 2cm) .. (7cm, 0);
q := (1cm, 3cm) .. (3cm, 0) .. (3cm, -1cm);
```

计算交点并将其画出：

```
@ 计算两条路径的交点 #                                                        87
\startMPpage
# 定义插值曲线路径 p 和 q @ <86>
pair a; a := p intersectionpoint q;
drawpath p; drawpath q;
draw a withpen pencircle scaled 4pt withcolor darkred;
\stopMPpage
```


图 5.1　路径求交

若两条路径有多个交点，`intersectionpoint` 宏只能返回其中一个。若两条路径无交点，则调用 `intersectionpoint` 宏时，会导致 `context` 命令报错，错误信息如下：

```
error: The paths don't intersect
```

MetaPost 还提供了一个运算符原语 `intersectiontimes`，用于算出路径交点在两条路径上的「时间」。其实你已经见识过路径上的时间，即「`point t of p`」语法，`t` 是时间，只是之前我一直称之为节点序号，而且用的一直都是整数，实际上 `t` 的值可以是小数，例如

```
path p; p := (0, 0) -- (1cm, 1cm) -- (2cm, 1cm);
pair a; a := point 1.25 of p;
```

上述代码中的点 `a` 位于第 2、3 个节点构成的路径子段上，且位于该段前 25% 处，即 `a` 到第 2 个节点的距离是路径子段长度的 25%。

基于 intersectiontimes 返回结果是 pair 类型，其第 x 分量是交点在宏左侧路径上的时间，y 分量是交点在宏右侧路径上的时间，例如

```
@ 计算两条路径的相交时间 #                                                    88
\startMPpage
# 定义插值曲线路径 p 和 q @ <86>
pair a; a := p intersectiontimes q;
drawpath p; drawpath q;
draw point (xpart a) of p withpen pencircle scaled 8pt;
draw point (ypart a) of q withpen pencircle scaled 4pt withcolor white;
\stopMPpage
```

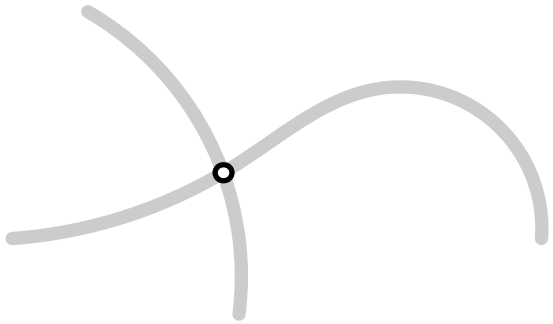


图 5.2　路径相交时间

若两条路径有多个交点，intersectiontimes 只会返回其中一个交点的时间。若遇到两条路径无交点的情况，不同于 intersectionpoint 宏以报错收场，intersectiontimes 会返回 (-1, -1)。

5.2 路径切割

如果两条路径存在多个交点，例如以下两条路径存在 2 个交点：

```
@ 定义存在两个交点的路径 p 和 q #                                             89
path p, q;
p := (0, -2cm) .. (2cm, 0cm) .. (4cm, 2cm) .. (6cm, -2cm);
q := (-1cm, 0) -- (8cm, 0);
```

该如何用 intersectionpoint 或 intersectiontimes 获得它们呢？有一个方法可以做到，只是有些暴力，需要将其中一条路径不断在所求出的交点处打断，再将其两段分别与另一条路径继续求交。

MetaPost 宏 cutbefore 和 cutafter 可以在路径上的一点处切断路径，前者会保留切割点之前的部分，后者会保留切割点之后的部分。例如

```
@ 以 p 和 q 的一个交点 a 切割 p，得到 p1 和 p2 #                              90
path p[]; pair a;
a := p intersectionpoint q;
p1 := p cutafter a;
p1 := p1 cutafter (point -3mm on p1);
p2 := p cutbefore a;
p2 := p2 cutbefore (point 3mm on p2);
```

注意，基于 a 分割 p，得到 p1 和 p2 后，又分别对 p1 的尾部和 p2 的首部又作了 3mm 的切割，从而保证两条路径皆不再包含点 a，这样在下一阶段继续分别用 p1 和 p2 与 q 求交时，能够保证所得交点不再是 a。

以下代码实现了 p1 和 p2 分别与 q 求交的过程：

```
@ p1 和 p2 分别与 q 求交，对 p1 和 p2 继续切割 #                                    91
path p[][]; pair b;
b := p1 intersectiontimes q;
if b <> (-1, -1):
    p[1][1] := p1 cutafter (point (xpart b) of p1);
    p[1][1] := p[1][1] cutafter (point -3mm on p[1][1]);
    p[1][2] := p2 cutbefore (point (xpart b) of p1);
    p[1][2] := p[1][2] cutbefore (point 3mm on p[1][2]);
fi;
b := p2 intersectiontimes q;
if b <> (-1, -1):
    p[2][1] := p2 cutafter (point (xpart b) of p2);
    p[2][1] := p[2][1] cutafter (point -3mm on p[2][1]);
    p[2][2] := p2 cutbefore (point (xpart b) of p2);
    p[2][2] := p[2][2] cutbefore (point 3mm on p[2][2]);
fi;
```

上述代码使用了二维数组，用于表达对 p1 和 p2 的分割结果，从而免去取名字的烦恼。路径求交运算用了更为稳健的 intersectiontimes 以应对路径无交点情况。符号 <> 在条件语句里，表示「不等于」。当 b 不等于 (-1, -1) 时，意味着路径存在交点。注意，由于并不能确定 p1 和 p2 分别与 q 有交点，故而上述过程对 p1 和 p2 的分割结果是否存在也不可知，在使用这些分割结果时，需要判定其存在性。

以下代码将上述过程合并起来，并将所得结果画出：

```
@ 以路径 p 和 q 的两个交点分割 p #                                                 92
\startMPpage
# 定义存在两个交点的路径 p 和 q @ <89>
# 以 p 和 q 的一个交点 a 切割 p，得到 p1 和 p2 @ <90>
# p1 和 p2 分别与 q 求交，对 p1 和 p2 继续切割 @ <91>
if (known p[1][1]) and (known p[1][2]):
    drawpath p[1][1]; drawpath p[1][2];
else:
    drawpath p1;
fi;
if (known p[2][1]) and (known p[2][2]):
    drawpath p[2][1]; drawpath p[2][2];
else:
    drawpath p2;
fi;
drawpath q;
label.top("$p$", point 2 of p); label.top("$q$", point .5 of q);
\stopMPpage
```

known 是 MetaPost 原语，用于判断一个变量是否有值，与之反义的原语是 unknown。布尔运算符 and 可对两个布尔表达式作「并」运算。

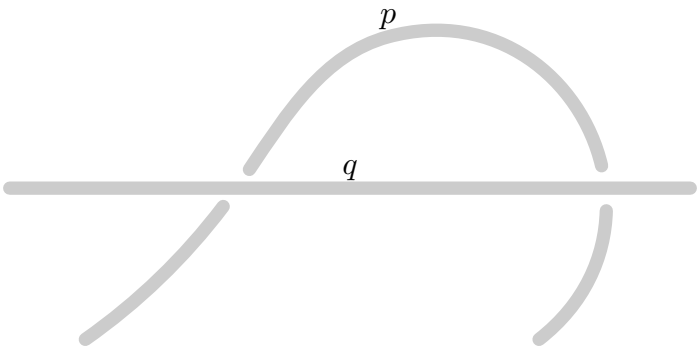


图 5.3　被两个交点分割的路径

图 **5.3** 所示的路径分割效果正是我想实现的路径立交桥的效果，被分割的路径 p 视为位于下层，而路径 q 则位于上层，只是我不希望 p 为此而被斩为三截。

在上述分割 p 的过程中，可以用一个数组收集每次获取的交点，当 p 的所有分割结果与 q 再无交点时，便可得到 p 与 q 的所有交点，这个过程可表示为递归形式，而 MetaPost 的普通宏和变量宏皆支持递归展开，故而可定义获得两条路径所有交点的 all_breakpoints 宏：

```
@ 定义 all_breakpoints 宏 #                                                    93
def all_breakpoints(expr p, q, r)(suffix breakpoints, n) =
    begingroup
    save a, b, s; pair a, b; path s[];
    a := p intersectiontimes q;
    if a <> (-1, -1):
        b := point (xpart a) of p;
        breakpoints[n] := b; n := n + 1;
        s1 := p cutafter b;  s1 := s1 cutafter (point -r on s1);
        all_breakpoints(s1, q, r, breakpoints, n);
        s2 := p cutbefore b; s2 := s2 cutbefore (point r on s2);
        all_breakpoints(s2, q, r, breakpoints, n);
    fi;
    endgroup
enddef;
```

倘若上述递归宏的定义让你有些发蒙，我希望你记住的是，suffix 类型的参数可以引用宏定义之外的变量，类似于主流编程语言里的指针或引用。

以存在三个交点的路径

```
@ 定义存在三个交点的路径 p 和 q #                                              94
path p, q;
p := (0, -2cm) .. (2cm, 0cm) .. (4cm, 2cm) .. (6cm, -2cm) .. (8cm, 1cm);
q := (-1cm, 1cm) -- (10cm, -1cm);
```

测试 all_breakpoints 宏：

```
@ 测试 all_breakpoints 宏 #                                                    95
\startMPpage
# 定义 all_breakpoints 宏 @ <93>
# 定义存在三个交点的路径 p 和 q @ <94>
pair breakpoints[];
numeric n;
n := 0;
```

```
all_breakpoints(p, q, 3mm, breakpoints, n);
drawpath p;
drawpath q;
for i = 0 upto n - 1:
    draw breakpoints[i] withpen pencircle scaled 4pt;
endfor;
\stopMPpage
```

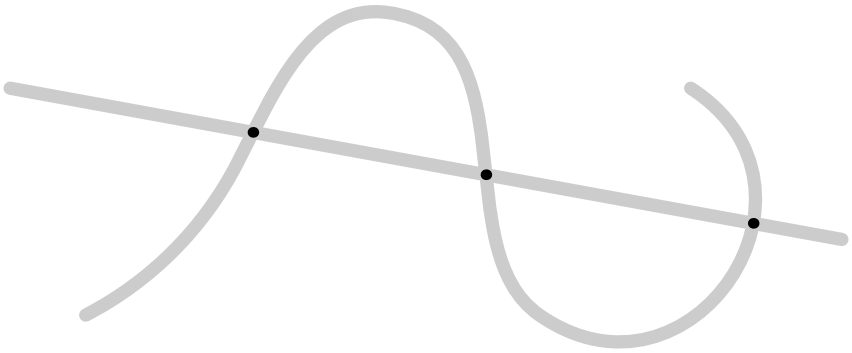


图 5.4　计算两条路径的所有交点

一旦能够算出两条路径的所有交点，在每个交点处画上一团背景色，覆盖在位于底层的路径上，只要合理安排路径和交点的绘制顺序，便可得到类似图 **5.3** 的效果。MetaPost 内置了一个 `color` 类型的全局变量 `background`，通过它可以获得背景色，默认是白色。以下代码便是这一设想的实现：

```
@ 具有欺骗性的路径断开效果 #                                                    96
\startMPpage
# 定义 all_breakpoints 宏 @ <93>
# 定义存在三个交点的路径 p 和 q @ <94>
pair breakpoints[]; numeric n; n := 0;
all_breakpoints(p, q, 3mm, breakpoints, n);
drawpath p;
for i = 0 upto n - 1:
    draw breakpoints[i] withpen pensquare scaled 5mm withcolor background;
endfor;
drawpath q;
\stopMPpage
```


图 5.5　路径断开特效

5.3 大巧不工

求出路径所有交点后，无论是分割路径，还是在交点处以背景色覆盖，都是聪明的办法，因为仅仅实现 all_breakpoints 宏就已经很考验编程能力了。实际上有一种更简单的路径断开效果模拟方法，即在画每条路径之前，先以较粗的画笔和背景色将路径先画一遍，例如

```
@ 更具欺骗性的路径断开效果 #                                                    97
\startMPpage
# 定义存在三个交点的路径 p 和 q @ <94>
draw p withpen pencircle scaled 5mm withcolor background; drawpath p;
draw q withpen pencircle scaled 5mm withcolor background; drawpath q;
\stopMPpage
```


图 5.6　更具备欺骗性的路径断开特效

认真对比图 **5.5** 和 **5.6**，应当能发现二者在路径断茬形态上有所不同，前者是与 x 轴平行，后者则是与位于上层的路径 q 平行，后者更为自然一些。

虽然 MetaPost 提供了背景色变量，但实际上这个变量并不会用于填充图形的背景，确切而言，MetaPost 并未给所绘图形赋予背景。MetaFun 实现的 addbackground 宏可以为已绘制的图形增加背景色，可以用它揭露上述路径断开骗术的真相：

```
@ 揭开路径断开骗术 #                                                           98
\startMPpage
# 定义存在三个交点的路径 p 和 q @ <94>
draw p withpen pencircle scaled 5mm withcolor background; drawpath p;
draw q withpen pencircle scaled 5mm withcolor background; drawpath q;
addbackground withcolor darkyellow;
\stopMPpage
```



图 5.7　路径断开骗术曝光

当我们用这种带有背景色的路径连接流程图中的文本框时，图 **5.7** 预示着一个问题，即背景色会对文本框有所侵蚀。要解决该问题，在绘制路径背景之前，先对路径的首尾切除少许，例如

```
@ 消除背景路径两端侵蚀区 #                                                    99
\startMPpage
# 定义存在三个交点的路径 p 和 q @ <94>
numeric thickness; thickness := 5mm;
path p_bg, q_bg;
p_bg := p cutbefore (point .5thickness on p);
p_bg := p_bg cutafter (point -.5thickness on p_bg);
draw p_bg withpen pencircle scaled thickness withcolor background; drawpath p;
q_bg := q cutbefore (point .5thickness on q);
q_bg := q_bg cutafter (point -.5thickness on q_bg);
draw q_bg withpen pencircle scaled thickness withcolor background; drawpath q;
addbackground withcolor darkyellow;
\stopMPpage
```



图 5.8　收缩背景路径两端

基于上述示例，将这种最简单的带背景路径的绘制过程定义为宏：

```
@ 定义 draw_path_with_background 宏 #                                         100
def draw_path_with_background(expr thickness) text paths =
    begingroup
    save p_bg; path p_bg;
    forsuffixes i = paths:
        p_bg := i cutbefore (point .5thickness on i);
        p_bg := p_bg cutafter (point -.5thickness on p_bg);
        draw p_bg withpen pencircle scaled thickness withcolor background;
        draw i;
    endfor;
    endgroup
enddef;
```

由于 draw_path_with_background 宏的第 2 个参数是 text 类型，可以传入 MetaPost 语句，而 forsuffixes 可遍历一系列名字，故而可使用 draw_path_with_background 宏一次绘制多条路径。注意，上述代码在画背景路径时，是在绘图语句里用了指定的画笔和颜色，而在画路径本体时，则是用默认的画笔和颜色，在实际使用这个宏时，可事先用 drawoptions 宏设定画笔和颜色。

以下代码用于测试 draw_path_with_background 宏并展示其用法：

```
@ 测试 draw_path_with_background 宏 #                                         101
\startMPpage
# 定义 draw_path_with_background 宏 @ <100>
# 定义存在三个交点的路径 p 和 q @ <94>
```

```
drawoptions(withpen pencircle scaled 4pt withcolor darkred);
draw_path_with_background(5mm) p, q;
addbackground withcolor darkyellow;
\stopMPpage
```

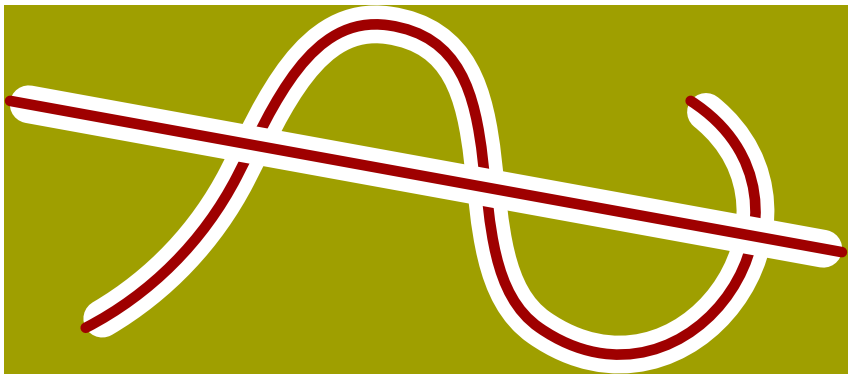


图 5.9　用 draw_path_with_background 宏画的路径

5.4 小结

直接为每条路径赋予背景路径的方法在模拟路径断开效果方面简单有效，后续会采用该方法构造流程图中的正则路径，但是基于交点的路径断开方法也并非没有意义，它为我们提供了一些实例，有助于我们理解两条路径交点的计算方法，后续会有一些问题需要基于路径交点方能解决。

6 不可名状

在第 3 章中，为文本框构造了一些锚点，可作为正则路径的起点和终点，然而这些锚点存在的前提是文本框为矩形。如果文本框是其他图形，例如平行四边形、菱形，甚至某种不规则图形，那么文本框及其锚点该如何定义呢？

6.1 标准图形

在研究非矩形文本框锚点如何构造之前，需要先探讨一个问题，什么形式的路径可作为文本边框？我的答案是它需要满足以下条件：

- 其中心应当为原点，便于用 `shifted` 定位；
- 其所有节点的 x 和 y 坐标的绝对值不大于 0.5，便于缩放。

MetaFun 提供了一些全局路径变量，诸如 `fullsquare`，`fulldiamond`，`fullcircle` 等，皆满足上述两个条件。例如，`fullsquare` 的值为

```
(.5, -.5) -- (.5, .5) -- (-.5, .5) -- (-.5, -.5) -- cycle;
```

`fulldiamond` 的值为

```
(.5, 0) -- (0, .5) -- (-.5, 0) -- (0, -.5) -- cycle;
```

从现在开始，我将所有满足上述两个条件的封闭路径称为标准图形，由此引申出一个问题，对于一个非标准的图形，如何通过平移和缩放运算将其转化为标准图形？该问题的一个答案是

```
@ 定义 convert_to_standard_shape 宏 #                                    102
vardef convert_to_standard_shape(expr p) =
    save f, w, h, s, q;
    numeric f, w, h; path q;
    w := bbwidth p; h := bbheight p;
    q := p shifted -(center p);
    f := if w > h: w else: h fi;
    if f > 0: q := q scaled (1/f); fi;
    q
enddef;
```

上述代码所实现的算法很简单。首先平移 `p`，使其中心对准原点，得到路径 `q`，然后以 `p` 的宽度和高度值的较大者的倒数对 `q` 予以缩放。

6.2 以边框为文本框本体

如果我们将文本框的边框作为文本框对象的本体，将文字画面作为其属性，再加上边框上的锚点属性，那么 `framed_text_class` 宏需要修改为

```
@ framed_text_class 宏版本之三 #                                         103
def framed_text_class text names =
    forsuffixes i = names:
        path i; % 以边框作为文本框的本体
        forsuffixes j = 中央, 东北, 西北, 西南, 东南,
```

```
                子门, 午门, 卯门, 酉门, 丑门, 寅门, 辰门, 巳门, 未门, 申门, 戌门, 亥门:
            pair i.j; % 声明文本框锚点
        endfor;
        picture i.content; % 声明文字画面
    endfor;
enddef;
```

用上述定义的 framed_text_class 声明的文本框对象会自动拥有边框属性，例如声明 foo 对象，

```
framed_text_class foo;
```

foo 是 path 对象，即文本框的边框，同时会有 foo.content，类型为 picture，表示文本画面。

6.3 散射

假设文本框未必为矩形，如何修改 make_anchors 方能使之能为文本框构造出合理的锚点呢？我想出的方法是先基于文本框边界先行构造出一套锚点，使之与文本框中心形成一系列射线，这些射线必然会与边框相交，可取其交点作为文本框的锚点。该过程的实现如下

```
@ make_anchors 宏版本之三 #                                                    104
def make_anchors(suffix obj) =
    % 基于 obj 的边界构造锚点
    obj.中央:= center obj;
    obj.东北:= urcorner obj; obj.西北:= ulcorner obj;
    obj.西南:= llcorner obj; obj.东南:= lrcorner obj;
    obj.子门:= .5[obj.西北, obj.东北]; obj.午门:= .5[obj.西南, obj.东南];
    obj.卯门:= .5[obj.东南, obj.东北]; obj.酉门:= .5[obj.西南, obj.西北];
    obj.丑门:= .5[obj.子门, obj.东北]; obj.寅门:= .5[obj.卯门, obj.东北];
    obj.辰门:= .5[obj.卯门, obj.东南]; obj.巳门:= .5[obj.午门, obj.东南];
    obj.未门:= .5[obj.午门, obj.西南]; obj.申门:= .5[obj.酉门, obj.西南];
    obj.戌门:= .5[obj.酉门, obj.西北]; obj.亥门:= .5[obj.子门, obj.西北];
    % 基于散射路径与 obj 求交，令锚点落在 obj 上
    forsuffixes i = 东北, 西北, 西南, 东南,
                    子门, 卯门, 午门, 酉门, 丑门, 寅门, 辰门, 巳门, 未门, 申门, 戌门, 亥门:
        obj.i := (obj.中央 -- obj.i) intersectionpoint obj;
    endfor;
enddef;
```

可以通过手工模拟一个菱形文本框，验证 make_anchors 是否能够算出菱形上的锚点。首先，声明文本框对象 foo，并将其边框设置为菱形：

```
@ 定义菱形文本框 foo 并为其构造锚点 #                                          105
framed_text_class foo;
foo := fulldiamond xyscaled (6cm, 3cm);
make_anchors(foo);
```

xyscaled 是 MetaPost 原语，可以对画面和路径在 x 和 y 方向上设定缩放倍数。上述语句构造了一个有些压扁的菱形，将其作为 foo 的边框。

为了能直观呈现 foo 锚点的计算过程，再手工构造一个矩形文本框 bar，使其边框刚好能包含 foo 的菱形边框，然后也为 bar 构造锚点：

```
@ 定义矩形文本框 bar，并为其构造锚点 #                                    106
framed_text_class bar;
bar := fullsquare xyscaled (6cm, 3cm);
make_anchors(bar);
```

将上述代码片段合并起来，便可用于测试 make_anchors 宏，并呈现其工作原理：

```
@ 测试 make_anchors 宏版本之三 #                                          107
\startMPpage
# framed_text_class 宏版本之三 @ <103>
# make_anchors 宏版本之三 @ <104>
# 定义菱形文本框 foo 并为其构造锚点 @ <105>
# 定义矩形文本框 bar，并为其构造锚点 @ <106>
draw foo; draw bar;
pickup pencircle scaled 4pt;
forsuffixes i = 东北，西北，西南，东南，
                子门，卯门，午门，酉门，丑门，寅门，辰门，巳门，未门，申门，戌门，亥门:
    draw (0, 0) -- bar.i withpen pencircle withcolor darkgray;
    draw bar.i withcolor darkgreen;
    draw foo.i withcolor darkred;
endfor;
\stopMPpage
```

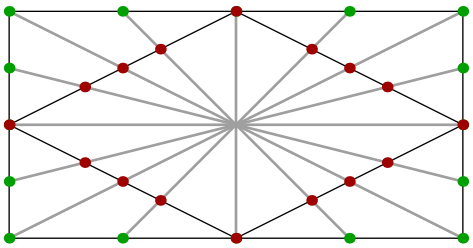


图 6.1　锚点散射

6.4 自由的代价

现在考虑改进 **3.5** 节定义的第三个版本的 framed_text 宏，令其能以参数的形式接受文本框的边框形状，从而将文本框形状的定义权交给用户。framed_text 宏的第四个版本定义如下：

```
@ framed_text 宏版本之四 #                                                108
def framed_text(suffix obj)(expr s, shape, padding) text somewhere =
    obj.content := textext(s);
    obj := make_frame(obj.content, shape, padding) somewhere;
    obj.content := obj.content somewhere;
    make_anchors(obj);
enddef;
vardef make_frame(expr content, shape, padding) =
    save frame; path frame;
    frame := convert_to_standard_shape(shape);
    # 放大 frame，使之足以囊括 content @ <111>
    frame
enddef;
```

上述代码为了克服 framed_text 宏因 suffix 类型参数而带来的隐患，将构造边框的过程交由一个只支持 expr 类型参数的变量宏 make_frame 完成，后者关键之处在于如何放大标准图形 frame，使之能够包含文本画面 content，并在文字周围留出 padding 大小的空白。这个问题颇有难度，因为我们不知道 frame 形状未知，它可能是四周凸起，也可能四周凹陷，也可能四周有凸有凹，我们只知道应该放大它，因为它是标准图形，不足以包含任何一个字母。可以试着用一种粗暴的办法来解决这个问题。

首先，为 content 构造含有留白的包围盒：

```
@ 为 content 构造含有留白的包围盒 content_box #                                    109
save w, h, content_box;
numeric w, h; path content_box;
w := bbwidth content; h := bbheight content;
content_box := (-.5w - padding, -.5h - padding) -- (.5w + padding, -.5h - padding)
               -- (.5w + padding, .5h + padding) -- (-.5w - padding, .5h + padding) -- cycle;
```

然后以一定的速度放大 frame，在此过程中不断探测 frame 的中心到 content_box 角点的射线是否与 frame 相交，若相交则停止探测，并以当前放大倍数的 frame 所求结果。

```
@ 以一定的速度放大 frame，直至它能包含 content_box #                              110
save p, q, a, c, f, t;
path p, q; pair a, c[]; boolean f; numeric t;
c[0] := (0, 0);
c[1] := urcorner content_box; c[2] := lrcorner content_box;
c[3] := llcorner content_box; c[4] := ulcorner content_box;
for i = 1mm step 1mm until infinity:
    p := frame scaled i;
    f := true;
    for j = 0 upto 4:
        q := c[0] -- c[j];
        a := q intersectiontimes p;
        if not (a = (-1, -1)):
          f := false;
          exitif true;
        fi;
    endfor;
    t := i;
    exitif f;
endfor;
frame := frame scaled t;
```

上述代码出现了一些之前未曾见过的语法。infinity 表示无穷大的数，故而第一层 for 语句相当于一个死循环，每次令 i 每次增加 1mm，其末尾的 exitif 语句的作用是，若给定的条件为真时退出循环。与 exitif 语义相反的是 exitunless，后者是在给定条件为假时，退出循环。这两层嵌套的 for 语句能够帮助我们确定 frame 的放大倍数 t。

合并上述两个片段，便可得到 frame 的放大过程，只是最终所得边框会有 1mm 的误差，若要追求精确，可以缩小 frame 的放大速度的量级，但误差总是存在的。

```
@ 放大 frame，使之足以囊括 content #                                              111
# 为 content 构造含有留白的包围盒 content_box @ <109>
# 以一定的速度放大 frame，直至它能包含 content_box @ <110>
```

以下代码用于测试本节所定义的的 framed_text 宏：

```
@ 椭圆文本框示例 #                                                    112
\startMPpage
# 定义 convert_to_standard_shape 宏 @ <102>
# framed_text_class 宏版本之三 @ <103>
# make_anchors 宏版本之三 @ <104>
# framed_text 宏版本之四 @ <108>
framed_text_class foo;
framed_text(foo, "Foo", fullcircle xyscaled (4cm, 2cm), 4pt);
draw foo; draw foo.content;
forsuffixes i = 子门, 卯门, 午门, 酉门,
                丑门, 寅门, 辰门, 巳门, 未门, 申门, 戌门, 亥门:
    draw foo.i withpen pensquare scaled 2pt withcolor magenta;
endfor;
\stopMPpage
```

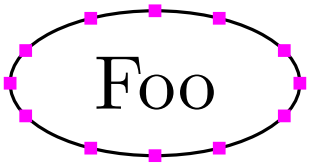


图 6.2　椭圆文本框

实际上 frame 的放大倍数实际上不必从 1mm 开始，它可以从 content_box 的宽度和高度中的较大值开始，原因是 frame 放大后必须包含 content_box。

```
@ 更快的 frame 放大过程 #                                             113
save p, c, f, t;
path p; pair c[]; boolean f; numeric t;
c[0] := (0, 0);
c[1] := urcorner content_box; c[2] := lrcorner content_box;
c[3] := llcorner content_box; c[4] := ulcorner content_box;
t := 1;
% w 和 h 已在构造 content_box 过程中声明了，现在需要更新它们的值
w := bbwidth content_box; h := bbheight content_box;
for i = (if w > h: w else: h fi) step 1mm until infinity:
    p := frame scaled i;
    f := true;
    for j = 1 upto 4:
        if ((c[0] -- c[j]) intersectiontimes p) <> (-1, -1):
          f := false;
          exitif true;
        fi;
    endfor;
    t := i;
    exitif f;
endfor;
frame := frame scaled t;
```

于是便有了 framed_text 的第五个版本：

```
@ framed_text 宏版本之五 #                                                    114
def framed_text(suffix obj)(expr s, shape, padding) text somewhere =
    obj.content := textext(s);
    obj := make_frame(obj.content, shape, padding) somewhere;
    obj.content := obj.content somewhere;
    make_anchors(obj);
enddef;
vardef make_frame(expr content, shape, padding) =
    save frame; path frame;
    frame := convert_to_standard_shape(shape);
    # 为 content 构造含有留白的包围盒 content_box @ <109>
    # 更快的 frame 放大过程 @ <113>
    frame
enddef;
```

6.5 动态锚点

文本框之间的连接必须基于位于文本框边框上预定义的锚点吗？为何不可以直接基于文本框中心构造正则路径，然后将所构造的路径落入边框之内的部分砍掉，剩余部分不正是文本框之间的连接路径吗？不妨动手一试。

首先，构造两个文本框：

```
@ 构造菱形文本框 foo 和椭圆文本框 bar #                                      115
framed_text_class foo, bar;
framed_text(foo, "Foo", fulldiamond xyscaled (4, 2), 4pt);
framed_text(bar, "Bar", fullcircle xyscaled(3, 1), 4pt) at (4cm, 3cm);
```

基于文本框的中心构造正则路径：

```
@ 以 foo 和 bar 的中心构造正则路径 p #                                       116
path p;
p := foo.中央 xyxto bar.中央;
```

以 foo 和 bar 的边框裁剪路径 p 的两端落入边框之内的部分：

```
@ 裁剪 p 的两端落入 foo 和 bar 内的部分 #                                     117
pair a, b;
a := p intersectionpoint foo; b := p intersectionpoint bar;
p := (p cutbefore a) cutafter b;
```

上述代码中构造的点 a 和 b 可分别称为 foo 和 bar 边框上的动态锚点，亦即它们是由具体正则路径临时确定的锚点。

将上述片段合并起来，便可得到连接 foo 和 bar 的正则路径 p，然后将它们都画出来：

```
@ 自动锚点构造过程示例 #                                                     118
\startMPpage
# 定义 at 宏 @ <64>
# 正则路径宏 @ <44>
# 定义 smooth_regular_path 宏 @ <77>
# 定义 convert_to_standard_shape 宏 @ <102>
```

```
# framed_text_class 宏版本之三 @ <103>
# make_anchors 宏版本之三 @ <104>
# framed_text 宏版本之五 @ <114>
# 构造菱形文本框 foo 和椭圆文本框 bar @ <115>
# 以 foo 和 bar 的中心构造正则路径 p @ <116>
# 裁剪 p 的两端落入 foo 和 bar 内的部分 @ <117>
p := smooth_regular_path(p, 2mm);
draw foo; draw foo.content; draw bar; draw bar.content;
drawarrow p withpen pencircle scaled 1pt withcolor darkred;
\stopMPpage
```

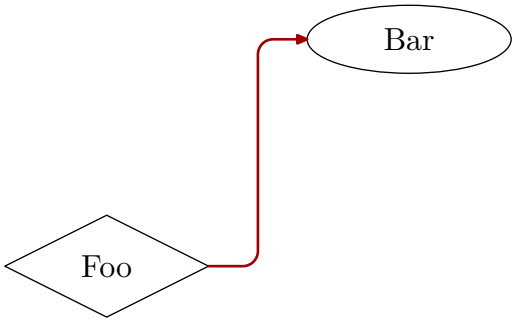


图 6.3　动态锚点示例

实际上 cutbefore 和 cutafter 宏的第 2 个参数可以是路径，上述示例中路径 p 的构造过程可简化为

```
p := (p cutbefore foo) cutafter bar;
```

这意味着动态锚点也是不必要存在的。上述构造正则路径的过程可以定义为运算符宏：

```
tertiarydef a fxyxto b =
    begingroup
    save p; path p;;
    p := a.中央 xyxto b.中央;
    p := (p cutbefore a) cutafter b;
    p
    endgroup
enddef;
```

不过，这个 => 宏无法工作，例如

```
path p; p := foo => bar;
```

会导致 context 命令报错并终止运行，原因在于 MetaPost 运算符宏所接受的参数只能是 expr 类型，在宏定义里，该类型的参数不支持后缀。不过，由于 => 所需要的文本框中心，可以临时算出，无需锚点，将 => 定义写为

```
tertiarydef a => b =
    begingroup
    save p; path p;;
    p := (center a) xyxto (center b);
    p := (p cutbefore a) cutafter b;
    p
```

```
    endgroup
enddef;
```

便可工作了。

6.6 小结

本章基于 MetaPost 路径求交语法，不仅解放了文本框的形状，还建立了更为自由的文本框锚点机制，当然付出的代价是代码变得愈发复杂了。好消息是，后续章节中的代码应该不会更复杂了，原因是绘制流程图这项工作所需的 MetaPost 知识，我们已经接触了大概七成，而且是主干部分，倘若你已经掌握了它们，那么已经能够用 MetaPost 完成许多画图工作了。不过，对于本文档而言，还需要继续前进，而且依然任重道远，希望你能和我继续走下去。

7 路径上的文字

我们已经知道如何使用 label 或 textext 宏在图形里插入文字，也知道如何在路径上取任何一个点，将这两个知识综合运用，便可以为流程图中的正则路径添加文字标注。

7.1 路径取点

我们已经学过了两种在路径上取点的语法，首先是 point ... of ...。point 大概仅仅是语法修饰作用，表示后面的语句是在路径上取点。of 是 MetaPost 原语，可以基于「时间」值从路径上取点，时间从 0 开始，之后在每个节点处增 1，还记得 intersectiontimes 吗？它返回的便是交点在两条路径上的时间值。路径上的节点，亦即在构造路径时所用的点，若按时间排序，第 1 个节点的时间值是 0，第 2 个节点的时间值是 1，依次类推，第 n 个节点的时间值是 $n-1$，这也是 MetaPost 的 length 原语作用于路径时的所得结果。

其次是 point ... on ... 语法，on 是 MetaFun 所作的扩展，它是运算符宏。若给定的长度为正数，on 会从路径的起点开始按长度取点，反之从路径的终点开始按长度取点。

MetaFun 提供了第三种路径取点语法，即 point ... along ...，它可以用路径长度分数的方式从路径的起点开始取点。例如，在路径总长度的 1/3 处取点。

以下代码演示了上述三种取点语法的作用，

```
@ 路径的三种取点方法 #                                                    119
\startMPpage
path p; pair a, b[], c;
p := (0, 0) -- (1cm, 1cm) .. (3cm, -1cm) .. (5cm, 1cm) -- (7cm , 0);
a := point 2.5 of p;
b1 := point 2.5cm on p;
b2 := point -0.75cm on p;
c := point 0.7 along p;
drawpath p; drawpoints p; drawpointlabels p;
pickup pencircle scaled 6pt;
draw a withcolor darkred; label.ulft("$a$", a);
draw b1 withcolor darkgreen; label.urt("$b_1$", b1);
draw b2 withcolor darkblue; label.urt("$b_2$", b2);
draw c withcolor magenta; label.ulft("$c$", c);
\stopMPpage
```

可对比其效果，在实际中根据需要选择合适的语法。图 7.1 中的 0，1，2，3，4 是 drawpointlabels 所绘，表示路径 p 各个节点的时间值。

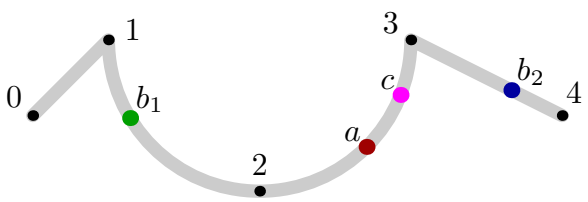


图 7.1　路径的三种取点方法示例

7.2 切向

要在路径上放置文字，除了知道如何从路径取点还不够，还需要知道路径在所取点处的方向，亦即切向。基于切向，便可确定文字标注应当旋转多少角度方能顺应路径的走向。

MetaPost 原语 direction 可与路径取点语法中的 of，on 以及 along 配合，从而在路径上指定位置获得路径切向。例如

```
@ 路径切向示例 #                                                    120
\startMPpage
path p; pair a, v;
p := fullcircle xyscaled (6cm, 3cm);
drawpath p;
for i = 0 step .1 until 1:
    a := point i along p;
    v := unitvector(direction i along p);
    draw a withpen pencircle scaled 4pt withcolor darkred;
    drawarrow a -- (a + 1cm * v) withpen pencircle;
endfor;
\stopMPpage
```

由于 direction 语法计算的路径切向未必是单位向量，可用 unitvector 将其转化为单位向量，然后再以数乘的方式设定切向量的长度。

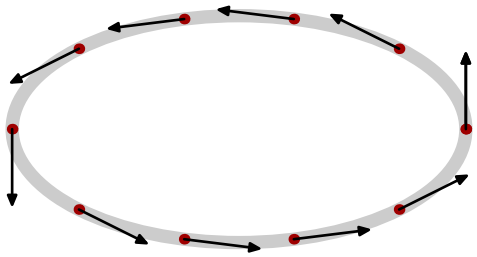


图 7.2　路径切向示例

算出路径上某点的切向后，用 MetaPost 的 angle 原语可以算出该切向的角度值，用该角度值旋转标注文字，便可使之顺应路径走向。例如

```
@ 确定文字标注在路径上的方位示例 #                                  121
\startMPpage
path p; picture a; pair c, t; numeric f;
p := (0, -2cm) .. controls (3cm, 2cm) and (5cm, -4cm) .. (7cm, -2cm);
f := .625;
a := textext("foo");
c := point f along p;
t := direction f along p;
draw p withpen pencircle scaled 2pt withcolor darkgray;
draw a rotated angle(t) shifted c;
draw c withpen pencircle scaled 4pt withcolor darkred;
\stopMPpage
```

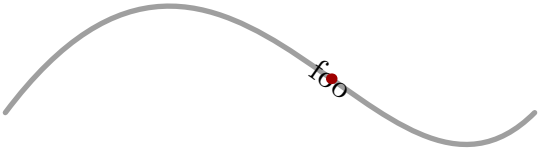


图 7.3　确定文字标注方位

需要注意，图 **7.3** 中，a 的中心是与点 c 重合的，导致路径从文字中部穿过。若想让文字位于曲线的上方或下方，可以在定义文字标注时增加一个偏移量，例如若上述示例中的

```
a := textext("foo");
```

修改为

```
% 将 a 向上偏移 2.5mm
a := textext("foo") shifted (2.5mm * up);
```

up 是单位向量 (0, 1)，是 MetaPost 语言的全局变量。后续在画 a 时，经旋转和平移，从 p 在 c 点的法向看，文字标注便位于 p 的上方了，结果如图 7.4 所示。

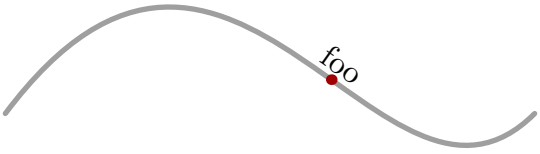


图 7.4　文字标注偏移示例

在上述示例中，至于在定义 a 时，偏移量设为多大合适，可基于 a 的包围盒高度以及路径线宽确定，例如

```
a := textext("foo");
a := a shifted ((.5(bbheight a) + 2pt) * up);
```

2pt 即为上述示例画路径 p 所用的线宽。上述修改所得结果与图 7.4 相近。

7.3 路径标注宏

经过上一节的演练，应当熟悉了如何在路径指定位置处确定文字标注方位过程，我将该过程封装为宏 tag：

```
@ 定义 tag 宏 #                                                    122
vardef tag(expr p, f, s) text somewhere =
    save a, c, v; picture a; pair c, v;
    a := textext(s) somewhere;
    c := point f along p;
    v := direction f along p;
    (a rotated angle(v) shifted c)
enddef;
```

tag 宏的第 4 个参数是 text 类型，可以吸入 MetaPost 语句并将其施加在 textext 构造的文字画面之后，以便于实现文字标注的微量偏移。注意，tag 只支持 along 路径取点方式。

以下代码用于测试 tag 宏：

```
@ 测试 tag 宏 #                                                    123
\startMPpage
# 定义 at 宏 @ <64>
# 定义 tag 宏 @ <122>
path p; numeric f;
p := (0, -2cm) .. controls (3cm, 2cm) and (5cm, -4cm) .. (7cm, -2cm);
f := .625;
draw p withpen pencircle scaled 2pt withcolor darkgray;
draw tag(p, f, "foo") at (2.5mm * up);
```

```
\stopMPpage
```

上述代码所画图形与图 7.4 相同，只是未将文字标注对应的路径位置画出。

7.4 如影随形

MetaFun 是 MetaPost 语言的宏包，至此为止，我们已经见识了许多 MetaFun 宏了。从某种意义上来说，MetaFun 是 MetaPost 语言层面的扩展，而现在的 MetaFun 已经进化至 LuaMetaFun 时代了，后者不仅提供了更多的语法，而且也允许在 MetaPost 代码里潜入 Lua 代码。与 MetaPost 这样的宏语言相比，Lua 更像是一种正经的编程语言。

LuaMetaFun 提供了 `lmt_followtext` 宏，能够让标注文字紧密贴合路径的形状。例如若路径是弯曲的，则标注文字也会变得同样弯曲。为了演示这个宏的用法，先构造一条曲线路径：

```
@ 构造曲线路径 p #                                                    124
path p;
p := (0, 0) .. (1cm, 1cm) .. (3cm, 0) .. (5cm, 2cm);
```

然后调用 `lmt_followtext` 宏：

```
@ 弯曲的标注文字 #                                                    125
\startMPpage
# 构造曲线路径 p @ <124>
draw p withpen pencircle;
draw lmt_followtext [text = "This is a demo.", path = p];
\stopMPpage
```

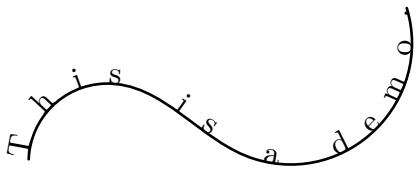


图 7.5　弯曲的文字标注

若你不希望文字扩张充满整条路径,可以用 `spread` 参数加以抑制：

```
@ 弯曲的标注文字之二 #                                                126
\startMPpage
# 构造曲线路径 p @ <124>
draw p withpen pencircle;
draw lmt_followtext [text = "This is a demo.", path = p, spread = false];
\stopMPpage
```

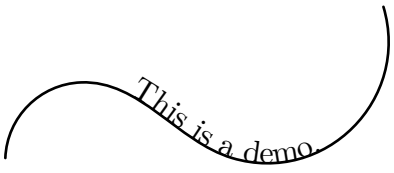


图 7.6　弯曲的文字标注之二

使用 `reverse` 参数还可以控制标注文字出现在路径的对面：

```
@ 弯曲的标注文字之三 #                                                127
\startMPpage
```

```
# 构造曲线路径 p @ <124>
draw p withpen pencircle;
draw lmt_followtext [text = "This is a demo.", path = p, spread = false, reverse = true];
\stopMPpage
```

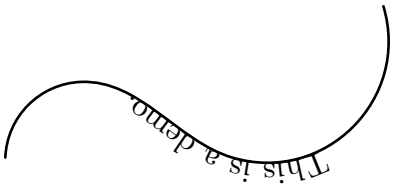


图 7.7　弯曲的文字标注之三

看上去，lmt_followtext 的功能比我们定义的 tag 宏更为强大，然而美中不知之处是，前者构造的文字标注过于贴近路径，倘若路径的粗度再大一些，会与文字产生一些重叠，例如

```
@ 弯曲的标注文字之四 #                                                    128
\startMPpage
# 构造曲线路径 p @ <124>
draw p withpen pencircle scaled 6pt withcolor darkgray;
draw lmt_followtext [text = "This is a demo.", path = p, spread = false];
\stopMPpage
```

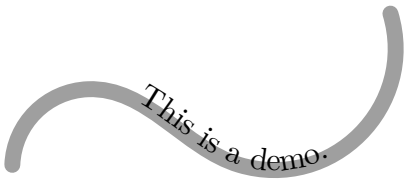


图 7.8　弯曲的文字标注之四

之所以会出现图 7.8 的情况，是因为 lmt_followtext 宏在设计上并非是为了路径标注，而是为了实现风格更为自由的文字排版。

7.5 路径等距偏置

如果对路径上的每个点按路径法向偏置一定的距离，然后构造一条新路径并将其作为 lmt_followtext 宏所用的路径，则上一节路径与文字重叠的问题是否能得以解决呢？无论能与不能，对路径作等距偏置，这本身也是一件有趣的事，不妨一试。

对于给定的路径 p，一个简单粗暴的路径偏置算法可由以下代码实现：

```
@ 定义 offset_path 宏 #                                                   129
vardef offset_path(expr p, d) =
    save c, t, v, q;
    pair c, t, v; path q;
    % 基于 p 的 100 个采样点构造 p
    for i = 0 step .01 until 1:
        c := point i along p;
        t := unitvector(direction i along p);
        v := t rotated 90;
        if i = 0:
            q := c + d * v;
        else:
```

```
            q := q -- (c + d * v);
        fi;
    endfor;
    q
enddef;
```

以下代码可以测 offset_path 宏，并将所得结果作为文字标注路径：

```
@ 测试 offset_path 宏 #                                                    130
\startMPpage
# 定义 offset_path 宏 @ <129>
# 构造曲线路径 p @ <124>
path q;
q := offset_path(p, 4pt);
draw p withpen pencircle scaled 6pt withcolor darkgray;
draw q withpen pencircle withcolor darkred;
draw lmt_followtext[path = q, text = "This is a demo.", spread = false];
\stopMPpage
```

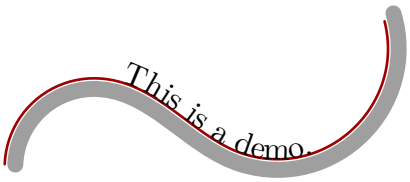


图 7.9　路径偏置

上述示例是成功的，文字标注随着路径的偏置而被抬升到合理的位置，倘若不将 q 画出，则图 **7.9** 已经是路径 p 更好的文字标注示例了。

对上述示例略作修改：

```
@ 测试 offset_path 宏之二 #                                                131
\startMPpage
# 定义 offset_path 宏 @ <129>
# 构造曲线路径 p @ <124>
path q;
q := offset_path(p, -4pt);
draw p withpen pencircle scaled 6pt withcolor darkgray;
draw q withpen pencircle withcolor darkred;
draw lmt_followtext[path = q, text = "This is a demo.", spread = false, reverse = true];
\stopMPpage
```

便可将标注文字定位到路径底部，结果如图 **7.10** 所示。

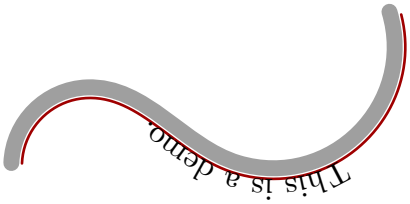


图 7.10　路径偏置之二

offset_path 是基于原路径的 100 个采样点构造出新的路径。一般情况下，无需得到精确的偏置路径，可以根据原路径的节点数量确定采样点数。例如，若原路径由 4 个节点构成，这意味着路径分为

三段，若每段取 5 个采样点，一共可以取 15 个采样点，从中去掉 2 个重复的，结果只需要 13 个采样点便可完成新路径的构造了。基于这一设想，改进上述的 offset_path 宏：

```
@ offset_path 宏版本之二 #                                                    132
vardef offset_path(expr p, d) =
    save c, t, v, f, k, n, delta, q;
    pair c, t, v; numeric f, k, n, delta; path q;
    k := length p; n := 5 * k - (k - 1); delta := 1 / n;
    for i = 0 step delta until 1:
        if i + delta > 1: f := 1; else: f := i; fi;
        c := point f along p;
        t := unitvector(direction f along p);
        v := t rotated 90;
        if i = 0:
            q := c + d * v;
        else:
            q := q -- (c + d * v);
        fi;
    endfor;
    q
enddef;
```

以下代码用于测试上述重新定义的 offset_path 宏：

```
@ 测试 offset_path 宏之三 #                                                   133
\startMPpage
# offset_path 宏版本之二 @ <132>
# 构造曲线路径 p @ <124>
path q;
q := offset_path(p, 4pt);
draw p withpen pencircle scaled 6pt withcolor darkgray;
draw q withpen pencircle withcolor darkred;
draw lmt_followtext[path = q, text = "This is a demo.", spread = false];
\stopMPpage
```

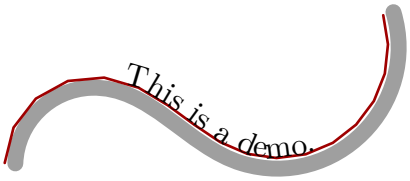


图 7.11　路径偏置之三

从图 7.11 可以看到，降低了采样点数之后，等距偏置生成的路径虽然变得有些粗略，但是用于定位标注文字依然是可行的。

7.6 路径贴靠标注宏

基于上一节对路径等距偏置方法的探讨，定义 followtag 宏，以实现可弯曲的路径标注文字：

```
@ 定义 followtag 宏 #                                                         134
vardef followtag(expr p, s, d) =
```

```
    save q; path q;
    q := offset_path(p, d);
    if d < 0:
        lmt_followtext[path = q, text = s, spread = false, reverse = true]
    else:
        lmt_followtext[path = q, text = s, spread = false]
    fi
enddef;
```

以下代码用于测试 followtag 宏：

```
@ 测试 followtag 宏 #                                                    135
\startMPpage
# offset_path 宏版本之二 @ <132>
# 定义 followtag 宏 @ <134>
# 构造曲线路径 p @ <124>
draw p withpen pencircle scaled 6pt withcolor darkgray;
draw followtag(p, "This is a demo.", 4pt);
\stopMPpage
```

路径贴靠标注宏 followtag 宏虽然能够让标注文字位于路径的上方或下方，但是标注文字总是处于路径的中段，无法像 tag 宏那样能够在路径上定位放置。不过，这个问题并不难解决，只需从待标注路径上截取指定的一段作为标注文字的贴靠路径，例如

```
@ 路径定位贴靠标注示例 #                                                 136
\startMPpage
# offset_path 宏版本之二 @ <132>
# 定义 followtag 宏 @ <134>
# 构造曲线路径 p @ <124>
draw p withpen pencircle scaled 6pt withcolor darkgray;
pair a, b; path q;
a := point .7 along p; b := point .9 along p;
q := (p cutbefore a) cutafter b;
draw q withpen pencircle withcolor darkred;
draw followtag(q, "demo", 4pt);
\stopMPpage
```



图 7.12　路径定位贴靠标注

为方便从路径上截取一段作为文字标注贴靠路径，可将该过程定义为宏：

```
@ 定义 snap 宏 #                                                         137
vardef snap(expr p, a, b) =
    ((p cutbefore (point a along p)) cutafter (point b along p))
enddef;
```

于是上述路径定位贴靠标注示例可简化为

```
@ 路径定位贴靠标注示例之二 #                                                    138
\startMPpage
# offset_path 宏版本之二 @ <132>
# 定义 followtag 宏 @ <134>
# 定义 snap 宏 @ <137>
# 构造曲线路径 p @ <124>
draw p withpen pencircle scaled 6pt withcolor darkgray;
draw followtag(snap(p, .7, .9), "demo", 4pt);
\stopMPpage
```

7.7 小结

本章以路径切向语法为基础，为路径实现了两种文字标注方法，tag 宏所实现的方法适用于标注文字较少的情况，若标注文字较多，则以能够使之贴靠路径的 followtag 宏所实现的方法为宜，其亮点在于以折线段近似构造路径的等距偏置结果。

8 样式表

之前的章节所讲述的 MetaPost 语法以及所实现的一些宏，已经足以胜任各种形式的流程图绘制工作了，只是由于尚且缺乏全局统筹，即便能画一些流程图，所写代码也必定因充斥太多与样式相关的细节而趋于繁杂。倘若我们希望用足够简洁的代码画出流程图，就很有必要定义一组全局变量，通过它们控制流程图的整体风格，从而让画图代码尽量与样式无关。

8.1 文字

我们是在 ConTEXt 语言里使用 MetaPost 语言，亦即后者嵌入前者。ConTEXt 语言负责文字排版，MetaPost 负责绘制图形。

在 MetaPost 语言里，可以通过 `textext` 宏，将 ConTEXt 的文字排版功能引入至 MetaPost 语言，使得后者也具备丰富的文字排版能力。在使用 MetaPost 语言画流程图时，流程图中文本框以及路径上的标注文字，皆可由 ConTEXt 语言制作，所用渠道是 MetaFun 提供的 `textext` 宏。

若使用 `textext` 宏排版流程图中的所有文字，则字号由 ConTEXt 排版环境决定。MetaFun 提供了全局变量 `BodyFontSize`，其值为 ConTEXt 排版环境设定的正文字号。文字的颜色可以在 `textext` 宏里使用 ConTEXt 语言设定，也可以在 MetaPost 的 `draw` 语句里设定，我们选择后者。

流程图中文字的字号和默认颜色，用以下全局变量表达：

```
@ 流程图样式变量 #                                                    139
numeric style.text_fontsize; style.text_fontsize := BodyFontSize;
color style.text_color; style.text_color := black;
```

8.2 文本框

文本框样式可由以下全局变量表示：

```
@ 流程图样式变量 # +                                                  140
path style.frame_shape; % 边框形状，默认不作设定
color style.frame_background; style.frame_background := white; % 文本框背景色
numeric style.frame_padding; style.frame_padding := .5BodyFontSize; % 边框留白
numeric style.frame_thickness; style.frame_thickness := 1pt; % 边框线粗度
color style.frame_color; style.frame_color := black; % 边框颜色
```

上述片段使用了 `xi` 程序支持的同名代码片段合并语法，可以用 `+` 符号为既有的代码片段补充内容。基于文字和文本框的样式变量，`framed_text` 宏可以改进为

```
@ framed_text 宏版本之六 #                                            141
def framed_text(suffix obj)(expr s) text somewhere =
    obj.content := textext(s);
    obj := make_frame(obj.content) somewhere;
    obj.content := obj.content somewhere;
    make_anchors(obj);
enddef;
vardef make_frame(expr content) =
    save frame, padding;
    path frame; numeric padding;
    padding := style.frame_padding;
    # 为 content 构造含有留白的包围盒 content_box @ <109>
```

```
    if unknown style.frame_shape:
        frame := content_box;
    else:
        frame := convert_to_standard_shape(style.frame_shape);
        # 更快的 frame 放大过程 @ <113>
    fi;
    frame
enddef;
```

以下代码用于测试 framed_text 的第六个版本：

```
@ 以默认样式构造文本框 foo 和 bar # 142
\startMPpage
# 流程图样式变量 @ <139>
# 一些稳定下来的宏 @ <143>
# framed_text 宏版本之六 @ <141>
framed_text_class foo, bar;
framed_text(foo, "Foo"); framed_text(bar, "Bar") at (5cm, 2cm);
draw foo; draw foo.content; draw bar; draw bar.content;
\stopMPpage
```

上述代码中引用的片段「一些稳定下来的宏」是一些不再发生演变的宏的汇总：

```
@ 一些稳定下来的宏 # 143
# 定义 at 宏 @ <64>
# 定义 convert_to_standard_shape 宏 @ <102>
# framed_text_class 宏版本之三 @ <103>
# make_anchors 宏版本之三 @ <104>
```

后续这个片段还会加入一些新稳定下来的宏。上述示例所得结果如图 **8.1** 所示。



图 8.1　文本框的默认样式

若 style.frame_shape 未设定，则以包含留白的文本包围盒为边框，那么倘若该变量有定义呢？如以下示例：

```
@ 根据设定的边框样式构造文本框 foo 和 bar # 144
\startMPpage
# 流程图样式变量 @ <139>
# 一些稳定下来的宏 @ <143>
# framed_text 宏版本之六 @ <141>
style.frame_shape := fullcircle;
framed_text_class foo, bar;
framed_text(foo, "Foo"); framed_text(bar, "Bar") at (5cm, 2cm);
draw foo; draw bar;
\stopMPpage
```



图 8.2　有设定的边框样式构造的文本框

对于已定义的变量，亦即有值的变量，倘若需要将其恢复为未定义状态，只需重新将其声明一次。以下代码可将在上述示例中获得定义的 style.frame_shape 再次恢复为未定义状态：

```
path style.frame_shape;
```

8.3 非等比例放大

新修改的 framed_text 宏及其上一个版本，对于给定的边框作标准化处理后再放大的过程中，宽度和高度方向是呈等比例放大的，然而很多时候，我们未必希望如此。例如，图 8.2 对应的示例，给定的边框是 fullcircle，即单位圆，当它套住含有留白的文字包围盒之后，如果我们希望它变成椭圆呢？那需要增加一个样式变量：

```
@ 流程图样式变量 # +                                                    145
boolean style.frame_isotropic; style.frame_isotropic := false; % 边框等比例缩放开关
```

上述变量的默认值表示不对边框的标准图形进行等比例缩放。为了实现边框的标准图形非等比例放大，framed_text 宏需修改为

```
@ framed_text 宏版本之七 #                                              146
def framed_text(suffix obj)(expr s) text somewhere =
    obj.content := textext(s);
    obj := make_frame(obj.content) somewhere;
    obj.content := obj.content somewhere;
    make_anchors(obj);
enddef;
vardef make_frame(expr content) =
    save frame, padding;
    path frame; numeric padding;
    padding := style.frame_padding;
    # 为 content 构造含有留白的包围盒 content_box @ <109>
    if unknown style.frame_shape:
        frame := content_box;
    else:
        frame := convert_to_standard_shape(style.frame_shape);
        # 更快的 frame 放大过程 @ <113>
        if not style.frame_isotropic:
            # 调整 frame 的宽高比 @ <147>
        fi;
    fi;
    frame
enddef;
```

在 s_frame 等比例放大后，只需对其宽高比加以控制，便可得到非等比例放大效果，该过程的实现如下：

```
@ 调整 frame 的宽高比 #                                                    147
save fw, fh; numeric fw, fh;
fw := bbwidth frame; fh := bbheight frame;
if w > h:
    frame := frame xysized (fw, fw * h / w);
else:
    frame := frame xysized (fh * w / h, fh);
fi;
```

非等比例边框宽度和高度基于标注文字包围盒（含有留白）的宽高比确定。xysized 是 MetaFun 运算符宏，用于修改指定路径或画面的宽度和高度。

以下代码用于测试非等比例边框的构造效果，如愿以偿，得到的是两个椭圆边框：

```
@ 以非等比例的方式构造文本框 foo 和 bar #                                 148
\startMPpage
# 流程图样式变量 @ <139>
# 一些稳定下来的宏 @ <143>
# framed_text 宏版本之七 @ <146>
style.frame_shape := fullcircle;
framed_text_class foo, bar;
framed_text(foo, "Foo"); framed_text(bar, "Bar") at (5cm, 2cm);
draw foo; draw foo.content; draw bar; draw bar.content;
\stopMPpage
```



图 8.3 fullcircle 非等比例放大结果

现在，将第七个版本的 framed_text 加入稳定宏集：

```
@ 一些稳定下来的宏 # +                                                     149
# framed_text 宏版本之七 @ <146>
```

不过，如此一来，倘若使用 xi 提取示例代码，上一节的示例调用的 framed_text 宏会出现两个版本，为避免这种情况，需要使用 xi 的 --beginning 和 --end 选项设定代码抽取范围。对于 framed_text 最新版本导致的问题，只需使用 --end 选项即可，例如

```
$ xi -t --end "为了实现边框的标准图形非等比例放大" \
    -e "根据设定的边框样式构造文本框 foo 和 bar" -o demo.tex snail.xi
```

上述命令中的反斜线 \ 是续行符，当命令行较长时，可以用它断为多行。xi 的 --begining 选项的用法和 --end 相同，二者是从文档正文里指定一些文本作为代码抽取范围的开始和结束位置。

8.4 路径

路径的默认样式由以下变量定义：

```
@ 流程图样式变量 # +                                                                    150
color style.path_color; style.path_color := black; % 路径颜色
numeric style.path_thickness; style.path_thickness := 1pt; % 路径粗度
numeric style.path_fillet; style.path_fillet := 6pt; % 路径圆角半径
boolean style.path_background; style.path_background := true; % 是否开启路径背景
```

基于上述变量，为正则路径定义专门的绘制宏：

```
@ 定义 draw_regular_paths 宏 #                                                          151
def draw_regular_paths text paths =
    begingroup
    save tmp_path, bg_path, bg_thickness;
    path tmp_path, bg_path; numeric bg_thickness;
    bg_thickness := 9style.path_thickness;
    for i = paths:
        tmp_path := i;
        if known style.path_fillet:
            tmp_path := simplify_regular_path(tmp_path);
            tmp_path := smooth_regular_path(tmp_path, style.path_fillet);
        fi;
        if style.path_background:
            bg_path := tmp_path cutbefore (point (.5bg_thickness + style.path_thickness) on tmp_path);
            bg_path := bg_path cutafter (point -(.5bg_thickness + style.path_thickness) on bg_path);
            draw bg_path withpen pencircle scaled bg_thickness withcolor background;
        fi;
        drawarrow tmp_path withpen pencircle scaled style.path_thickness withcolor style.path_color;
    endfor;
    endgroup
enddef;
```

以下代码用于测试 `draw_regular_path` 宏：

```
@ 测试 draw_regular_path 宏 #                                                           152
\startMPpage
# 流程图样式变量 @ <139>
# 正则路径宏 @ <44>
# 改进的 smooth_regular_path 宏 @ <80>
# 定义 is_colinear 宏 @ <83>
# 定义 simplify_regular_path 宏 @ <84>
# 一些稳定下来的宏 @ <143>
# 定义 draw_regular_paths 宏 @ <151>
style.frame_shape := fullcircle;
framed_text_class foo, bar;
framed_text(foo, "Foo"); framed_text(bar, "Bar") at (5cm, 2cm);
draw foo; draw bar;
draw_regular_paths foo.卯门 xyxto bar.酉门, foo.子门 yxyto bar.午门;
\stopMPpage
```

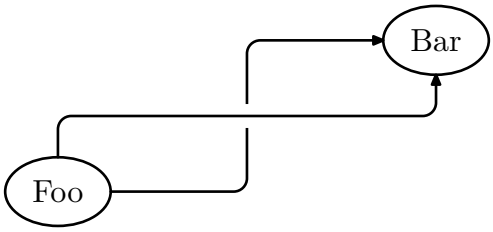


图 8.4　默认路径样式

将上述示例相关的宏也汇入稳定宏集：

```
@ 一些稳定下来的宏 # +                                                    153
# 正则路径宏 @ <44>
# 改进的 smooth_regular_path 宏 @ <80>
# 定义 is_colinear 宏 @ <83>
# 定义 simplify_regular_path 宏 @ <84>
# 定义 draw_regular_paths 宏 @ <151>
```

8.5 路径箭头

MetaPost 提供了全局变量 ahlength 和 ahangle 用于控制路径箭头的形状。ahlength 设定箭头的长度，默认值是 4bp。ahangle 设定箭头的角度，默认值是 45 度。

MetaFun 扩展了路径箭头形状的控制，提供了 ahvariant 和 ahdimple 变量。ahvariant 可以设定箭头的型号，默认有三种型号，以 0，1 和 2 表示。ahdimple 用于控制箭头的窝深，其值为 0，表示箭头是三角形；为 1，表示纯粹的箭头，即窝深最大；为大于 1 的值，可构造菱形箭头。

在流程图样式里，选用以下变量控制箭头形状：

```
@ 流程图样式变量 # +                                                      154
ahvariant := 1;
ahdimple := 1;
ahlength := 5style.path_thickness;
```

基于上述设定构造的路径箭头如图 8.5 所示。

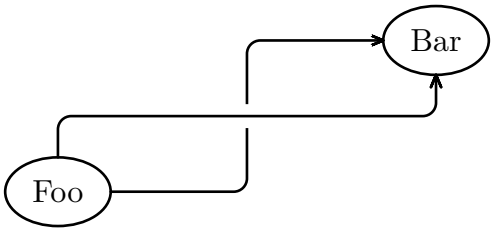


图 8.5　默认箭头样式

8.6 小结

全局变量如同控制木偶的线。虽然在主流编程语言里，使用全局变量通常视为罪行，然而 MetaPost，MetaFun以及 TEX 系统内置的全局变量，却可视为语法的一部分，我们为流程图设定的全局变量也是如此。

9 Lua 语言

现代的 MetaPost 语言，由 LuaTEX 程序内置的 mplib 库翻译为 PDF 语言，后者由 PDF 阅读器渲染为图形。LuaTEX 是一种现代的 TEX 引擎……我需要假设你知道何谓 TEX 引擎。在 LuaTEX 里，TEX 语言，MetaPost 语言和 Lua 语言，是可以沟通的，前两者皆为宏编程语言，而 Lua 是一种颇为正式且小巧的脚本编程语言。LuaTEX 的继任者是 LuaMetaTEX，后者是专为 ConTEXt 开发，进一步强化了 Lua 语言与 TEX 和 MetaPost 语言的联系。本章尝试基于 Lua 语言的表语法，简化流程图样式变量的设定代码。你不必担心自己对 Lua 语言一无所知，因为我们只需要知道极少的 Lua 语法便可达成这一目标。

9.1 Lua 表

在上一章里，为了实现对流程图样式的整体控制，定义了一组全局变量：

```
numeric style.text_fontsize; style.text_fontsize := BodyFontSize;
color style.text_color; style.text_color := black;
path style.frame_shape; % 默认不作设定，边框形状
color style.frame_background; style.frame_background := white; % 文本框背景色
numeric style.frame_padding; style.frame_padding := .5BodyFontSize; % 边框留白
numeric style.frame_thickness; style.frame_thickness := 1pt; % 边框线粗度
color style.frame_color; style.frame_color := black; % 边框颜色
boolean style.frame_isotropic; style.frame_isotropic := false; % 边框等比例缩放开关
color style.path_color; style.path_color := black; % 路径颜色
numeric style.path_thickness; style.path_thickness := 1pt; % 路径粗度
numeric style.path_fillet; style.path_fillet := 6pt; % 路径圆角半径
boolean style.path_background; style.path_background := true; % 是否开启路径背景
```

由于 MetaPost 语言中的变量必须先声明，然后方能赋值，这两个过程不能合而为一，导致上述代码显得颇为冗余。然而，若是用 Lua 表语法，则与上述代码对应的 Lua 表可写为

```
@ 以 Lua 表定义的流程图样式 #                                                    155
\startluacode
style = {
    text = {fontsize = 'BodyFontSize', color = 'black'},
    frame = {
        shape = '', background = 'white', padding = '.5BodyFontSize',
        thickness = '1pt', color = 'black', isotropic = 'false'
    },
    path = {color = 'black', thickness = '1pt', fillet = '6pt', background = 'true'}
}
\stopluacode
```

想必无需我多作解释，上述代码定义的 `style` 表的含义也是一目了然的。只是需要注意，一对单引号包含的内容，在 Lua 语言里表示字符串。此外，Lua 代码需要放在 `luacode` 环境里，不可与 MetaPost 代码混居。需要解决的问题是，如何在 MetaPost 代码里访问 Lua 表中的元素。

9.2 `scantokens` 与 `runscript`

之前我们已经多次用过字符串，亦即用双引号包含的文本，实际上它是 `string` 类型的值。MetaPost 有一个非常奇怪的原语 `scantokens`，它可以将字符串转化为 MetaPost 代码，例如

```
@ scantokens 示例 #                                                    156
\startMPpage
string a;
a := "draw fullsquare scaled 2cm withpen pencircle withcolor darkred";
scantokens(a);
\stopMPpage
```

不妨将 scantokens 视为一个嵌入在 MetaPost 代码里的 MetaPost 语言翻译器。

MetaPost 提供的另一个奇怪的原语是 runscript，它可以执行一段 Lua 代码，并将所得结果交由 scantokens 原语翻译为 MetaPost 代码，这意味着通过 runscript，我们可以获得 Lua 表中元素的值，例如

```
@ 从 Lua 样式表中取值示例 #                                            157
# 以 Lua 表定义的流程图样式 @ <155>
\startMPpage
numeric path_fillet;
path_fillet := runscript("return style.path.fillet");
show "path fillet: " & decimal (path_fillet/pt) & "pt";
\stopMPpage
```

用 context 编译上述示例，在终端里应当会输出「path fillet: 6pt」。上述的代码里的 & 符号用于拼接字符串，而 decimal 可将数值转化为字符串。

同理，基于 runscript 也能在 MetaPost 代码里修改 Lua 表中元素的值，例如

```
@ 从 MetaPost 代码里修改 Lua 样式表中元素的值 #                        158
# 以 Lua 表定义的流程图样式 @ <155>
\startMPpage
runscript("style.path.fillet = '10pt'");
numeric path_fillet;
path_fillet := runscript("return style.path.fillet");
show "path fillet: " & decimal (path_fillet/pt) & "pt";
\stopMPpage
```

上述示例的输出应当是「path fillet: 10pt」。

9.3 界面

runscript 就像是 MetaPost 和 Lua 这两个世界之间的通道，但是直接使用它访问和修改 Lua 表，代码不够直观，需要定义几个宏，简化这个过程。

流程图样式的 Lua 表中元素的访问过程可定义为

```
@ Lua 样式表中元素访问宏 #                                             159
def get_style(expr anything) =
  runscript("style" & anything)
enddef;
```

get_style 宏的用法如下：

```
numeric path_fillet;
path_fillet := get_style("path.fillet");
```

修改流程图样式的 Lua 表中元素的过程可定义为

```
@ Lua 样式表中元素修改宏 #                                                    160
def set_style(expr k, v) =
    begingroup
    save s; string s;
    if string v: s := "'" & v & "'"; fi;
    if boolean v: s := "'" & tostring v & "'"; fi;
    if numeric v: s := decimal(v); fi;
    if color v: s := "'(" & decimal (redpart v) & ","
                          & decimal (greenpart v) & ","
                          & decimal (bluepart v) & ")'";
    runscript("style." & k & "=" & s);
    endgroup
enddef;
```

上述代码展示了以下 MetaPost 语法：

- 数据类型可以作为类型检测运算符使用；
- `tostring` 原语，可将一些类型的数据转化为字符串，作用于数值时，与 `decimal` 等效；
- `redpart`，`greenpart` 和 `bluepart` 可获取颜色值的分量。

`set_style` 宏的用法如下：

```
color a;
a := (.2, .5, .7);
set_style("frame.color", a);
```

9.4 后缀宏

现在，我要告诉你 MetaPost 除了普通宏、变量宏和运算符宏之外，还有一种宏，其行为像变量宏，但是其名字可以有后缀。这种宏通常用于功能的分发，例如

```
@ 定义后缀宏 style #                                                          161
vardef style@#(text anything) =
    save s; string s; s := str @#;
    if s = "get": get_style(anything) fi
    if s = "set": set_style(anything) fi
enddef;
```

「`str @#`」用于将后缀转化为字符串，倘若像下面这样调用 `style` 宏

```
style.set("frame.color", darkred);
```

则 `style` 宏体里的「`str @#`」就是字符串 `"set"`。

上述定义的 `style` 宏便是后缀宏，它支持两个后缀，`get` 和 `set`，并根据后缀调用 `get_style` 和 `set_style` 宏。之前我们使用过的 `label` 宏也是后缀宏。实际上，MetaPost 语言还定义了后缀宏 `z`：

```
vardef z@# = (x@#, y@#) enddef;
```

由于 MetaPost 还定义了后缀变量 `x` 和 `y`，故而当我们写 `z3` 时，实际上是构造了点 `(x3, y3)`。

9.5 空值

以 Lua 表的形式定义流程图样式，在表达文本框的边框默认为空值时，使用的是 Lua 空字符串。这种方法存在缺陷，会导致 runscript 的结果为空，从而出现赋值时出错。例如以下语句

```
path default_frame;
default_frame := style.get("frame.shape");
```

第 2 行的展开结果是非法的赋值语句：

```
default_frame := ;
```

要解决空值的问题，需要借鉴 8.2 节恢复全局变量 style.frame_shape 未定义状态的方法，但还要借助 begingroup ... endgroup 表达式。可以将 style.frame.shape 的初值设定为

```
'begingroup path unknown_path; unknown_path endgroup'
```

亦即令 begingroup ... endgroup 表达式的结果为一个未定义的变量。然而，若 style.frame.shape 的值一经设定，例如

```
style.set("frame.shape", "fullsquare");
```

再要将其恢复为空值，需要作以下设定：

```
style.set("frame.shape", "begingroup path unknown_path; unknown_path endgroup");
```

为了方便起见，可以定义一个通用的 reset_style 宏：

```
@ 定义 reset_style 宏 #                                                    162
def reset_style(expr k) =
    if k = "frame.shape":
        style.set(k, "begingroup path unknown_path; unknown_path endgroup");
    fi;
enddef;
```

目前，reset_style 仅支持恢复 frame.shape 的未定义状态。以后若 Lua 样式表中还有其他元素需要恢复未定义状态，需对 reset_style 予以扩充。

9.6 小结

以 Lua 表代替 MetaPost 全局变量集合定义流程图样式，在 Lua 语言与 MetaPost 语言组合效用方面仅展现了冰山一角。即便如此，我们也能见识到 Lua 语言在组织复杂数据结构方面的优势，而这正是 MetaPost 语言所缺乏的。事实上，只要对 Lua 语言足够熟悉，完全可以用它编写大型的程序，然后将 MetaPost 语言用于画出该程序输出结果。

10 ConTEXt 模块

你围绕某个目的所写的一些久经考验而且颇为有用的 MetaPost 宏，大概是希望能将其统一存放在一起，形成一个模块，日后需要用这些宏时，就加载该模块，而不必一次又一次将这些宏的定义都复制过来。

10.1 input

MetaPost 的 input 原语用于在当前文件里载入所指定文件的内容，若后者为 MetaPost 代码，它们可以是你定义的宏。假设你定义的宏位于 foo.mp 文件，其内容为

```
@ foo.mp 文件内容 #                                                    163
vardef hello =
    save p; path p; p := fullsquare xyscaled (4cm, 2cm);
    image(
        fill p withcolor lightgray;
        draw p withpen pencircle scaled 4pt withcolor darkgray;
        draw textext("hello module!") withcolor darkred;
    )
enddef;
```

上述代码用了 MetaPost 的 fill 宏，该宏可向封闭路径内部填充指定的颜色。之前我一直没有提及它，现在可趁机学会它。

假设在 foo.mp 文件所在目录里有 bar.tex 文件，其内容为

```
@ bar.tex 文件内容 #                                                   164
\startMPpage
input foo.mp;
draw hello;
\stopMPpage
```

用 context 命令编译 bar.tex，可得到图 10.1 所示的结果。



图 10.1　带背景的文本框

10.2 TEX 目录树

如果将 foo.mp 放置在 ConTEXt 的 TEX 目录树里，它便成为日后可重复使用的 MetaPost 模块了。我的 ConTEXt 安装目录是 $HOME/opt/context，该目录里有个 tex 目录，此即 ConTEXt 的 TEX 目录树之根。下面以我的 Linux 机器上的 ConTEXt 环境为例[1]，讲述如何将 foo.mp 文件置入 TEX 目录树中的指定位置。

首先，用以下命令在 ConTEXt 的 TEX 目录树中构造一个新目录 foo：

[1] ConTEXt 也可以在 Windows 和 macOS 系统里使用，前提是你需要对所用系统的命令行使用有所了解。

```
$ mkdir -p $HOME/opt/context/tex/texmf-local/tex/context/third/foo
```

将 foo.mp 放在上述命令所构造的目录内，然后执行以下命令刷新 ConTEXt 的 TEX 目录树的索引：

```
$ context --generate
```

至此，MetaPost 模块 foo.mp 便安装完毕，可执行以下命令予以确认：

```
$ mtxrun --search foo.mp
```

若上述命令未能输出 foo.mp 在 TEX 目录树中的路径，便意味着在前面的步骤里，你可能出错了，或者你所用的 ConTEXt 过于陈旧。

10.3 ConTEXt 模块

在 ConTEXt 环境里使用 MetaPost，将上述的 foo.mp 改造为 ConTEXt 模块，在使用时可与 ConTEXt 语法风格取得统一。

首先，将 TEX 目录树里的 foo.mp 更名为 t-foo.mkxl，然后刷新目录树索引：

```
$ context --generate
```

新的文件名里，前缀 t- 表示这是第三方（third）模块，后缀 .mkxl 表示该模块是面向最新的 ConTEXt 版本，即 ConTEXt LMTX。

完成上述工作后，将 t-foo.mkxl 的内容修改为

```
@ t-foo.mkxl 文件内容 #                                                    165
\writestatus{loading}{ConTeXt User Module / foo}
\startmodule[foo]
\startMPinclusions
vardef hello  =
    save p; path p; p := fullsquare xyscaled (3cm, 1cm);
    image (
        fill p withcolor lightgray;
        draw p withpen pencircle scaled 4pt withcolor darkgray;
        draw textext("\color[darkred]{hello world!}");
    )
enddef;
\stopMPinclusions
\stopmodule
\endinput
```

上述代码中，\writestatus 可以在 context 命令输出中显示 foo 模块的载入信息。\startmodule[foo] ...\stopmodule 表示模块内容的起止。\endinput 表示模块终止。一般是要在 \endinput 之后给出这个模块的用法示例，这些内容会被 context 命令忽略。这些 TEX 语句构成了 ConTEXt 模块的基本框架。foo 模块的 MetaPost 代码放在 MPinclusions 环境里。

现在假设你的 TEX 目录树存在上述 t-foo.mkxl，则上一节的 bar.tex 需修改为

```
@ 使用 foo 模块的 bar.tex 文件内容 #                                       166
\usemodule[foo]
\startMPpage
draw hello;
```

```
\stopMPpage
```

编译新的 bar.tex，所得结果与图 10.1 相同。

需要注意的是，直接使用 MetaPost 的 label 宏是无法支持汉字的，在 ConTEXt 环境里使用 MetaPost，还是忘记 label 宏吧。

10.4 汉字

本文档前言中提到的《ConTEXt 蹊径》的第 3 章讲述了如何为 ConTEXt 安装汉字字体以及排版中文文档的基本方法，第 15 章介绍了我写的 zhfonts 模块的用法。如果你在 ConTeXt 的 TeX 目录树里安装了 zhfonts 模块，并且为 ConTEXt 安装了 zhfonts 默认使用的字体，那么在 MetaPost 代码里，可以为图形制作中文标注。例如

```
@ MetaPost 图形中文标注示例 #                                                    167
\usemodule[zhfonts]
\startMPpage
% 设置箭头形状
ahlength := 5mm; ahvariant := 1; ahdimple := 1;
% 构造路径，添加中文标注
path p; p := (0, 0) -- (7cm, 0);
drawarrow p withpen pencircle scaled 2pt withcolor darkblue;
draw textext("汉字") shifted ((point .5 along p) shifted (0, 8pt));
\stopMPpage
```



图 10.2　中文标注示例

10.5 小结

倘若你只是单纯学习 MetaPost 而阅读本文档，那么本章内容便是你最接近 ConTEXt 的地方。如果你不打算学习 ConTEXt，现在可以不再前进，区区流程图，不过是个大号的新手村。事实上，你所掌握的 MetaPost 语法和技巧已足以让你泛舟于江湖了。只是我还需要前进，因为我想实现的 ConTEXt 流程图模块尚未实现，我只是解决了编写该模块遇到的几乎所有的关键技术问题。

11 三种蜗牛

从本章开始，专注于实现 ConTEXt 流程图模块。该模块名为 snail，意思是蜗牛，取其慢，象征使用该模块画流程图会比较慢，但是若能足够用心，它可以会给你足够好的结果。

11.1 节点类型

犹如 MetaPost 路径由一组节点的连接形成，流程图中也是由一组文本框的连接形成，故而在流程图的范畴里，我们将文本框称为流程图的节点，并简称为节点，由于同样的名字，其确切含义是由上下文决定的，你不至于会将其与路径节点混淆。

流程图节点分为三种，无框节点、有框节点和插图节点。无框节点，即没有边框图形的文字。有框节点，即之前章节里探讨多次的文本框。插图节点是由外部图形构成的节点，当然至此为止，我尚未言及如何基于外部图形构造 picture 类型对象。这三种节点，无论是否有边框，它们都拥有默认的锚点。这三种节点皆需要以 snail_t 宏声明，该宏的定义如下：

```
@ 流程图节点 #                                                                    168
def snail_t text objs =
    forsuffixes it = objs:
        path it; % 节点本身
        path it.outline;  % 节点的轮廓线，用于放置锚点
        boolean it.frame; % 有框节点，此值为 true；其他节点，此值为 false
        picture it.content; % 节点文本
        anchorname it; % 锚点集集合
     endfor;
enddef;
```

snail_t 宏的行为类似前面章节探讨的 framed_text_class 宏，主要是利用了比喻，缩短了宏名，此外宏名中的 _t 后缀，是 _type 的简写。上述代码中所用的 anchorname 宏，其定义为

```
@ 流程图节点 # ^+                                                                 169
def anchorname text obj =
    pair obj.anchors[][];
    forsuffixes it = 中央，东北，西北，西南，东南，
                     子门，丑门，寅门，卯门，辰门，巳门，午门，未门，
                     申门，酉门，戌门，亥门，
                     乾位，坤位，艮位，兑位，巽位，震位，坎位，离位：
        pair obj.it;
    endfor;
enddef;
```

在 xi 所支持的文学编程语法里，代码片段首部出现的符号 ^+，表示在 xi 程序将本段代码在与其他同名代码片段的代码合并时，会插入其他同名片段代码之前。之前我们用过另一个同名片段合并运算符 +，它表示顺序合并，亦即按照同名代码片段出现的顺序合并代码。anchorname 的定义除了之前出现过的锚点，还额外增加了 8 个锚点，以八卦命名……切莫觉得这是故弄玄虚，因为我实在不想使用又臭又长的名字来给这么多锚点取名。不过，我考虑到你可能真的不喜欢古代这些玄妙的术语，所以我特意为你准备了一个二维数组 obj.anchors，我会按照三层锚点结构填充这个数组，以后你也可以基于这个数组索取所需锚点。

考虑到你也可能存在命名恐惧症，当你不想为每个流程图节点取名字，也可以使用 snailfam_t 定义节点数组，该宏的定义如下：

```
@ 流程图节点 # +                                                    170
def snailfam_t text objs =
    forsuffixes it = objs:
        path it[];
        path it[].outline;
        boolean it[].frame;
        picture it[].content;
        anchorname it[];
    endfor;
enddef;
```

snailfam_t 声明的每个对象都是一个一维数组，可将其作为以下标索引的节点序列使用。也能够更进一步，声明每个对象为二维数组：

```
@ 流程图节点 # +                                                    171
def snailgrid_t text objs =
    forsuffixes it = objs:
        path it[][];
        path it[][].outline;
        boolean it[][].frame;
        picture it[][].content;
        anchorname it[][];
    endfor;
enddef;
```

11.2 节点样式表

在开始定义节点构造宏之前，需要先给出节点的样式定义，我们采用 Lua 表结构表达流程图样式。以下代码先给出节点样式的默认设定：

```
@ Snail 模块样式表 #                                                172
\startluacode
snailmod = {
    text = {fontsize = 'BodyFontSize', color = 'black', offset = '(0, 0)'},
    frame = {
        shape = 'fullsquare', background = {fill = 'false', color = 'white'},
        thickness = '1pt', color = 'black', isotropic = 'false',
        padding = '.5BodyFontSize', margin = '2BodyFontSize'
    },
    # 其他元素的样式 @ <177>
}
\stopluacode
```

其他元素的样式待后续予以补充。

下面定义的两个宏分别用于访问和修改上述 Lua 表：

```
@ 流程图节点 # ^+                                                   173
<模块参数访问和设定>
% 该宏用于获取模块参数
def snail_mod_get(expr anything) =
```

```
    runscript("snailmod." & anything)
enddef;
% 该宏用于设置模块参数
def snail_mod_set(expr k, v) =
    begingroup
    save s; string s;
    if string v: s := "'" & v & "'"; fi;
    if boolean v: s := "'" & tostring v & "'"; fi;
    if numeric v: s := decimal(v); fi;
    if color v: s := "'(" & decimal (redpart v) & ","
                          & decimal (greenpart v) & ","
                          & decimal (bluepart v) & ")'"; fi;
    runscript("snailmod." & k & "=" & s);
    % 箭头样式是 MetaFun 全局变量，需要重新赋值方能使之生效
    if k = "path.ahvariant": ahvariant := snail_mod_get(k); fi;
    if k = "path.ahdimple": ahdimple := snail_mod_get(k); fi;
    if k = "path.ahlength": ahlength := snail_mod_get(k); fi;
    endgroup
enddef;
```

注意，上述代码中出现的「<模块参数访问和设定>」是 xi 文学编程语法中的代码片段标签，在 xi 程序从文学程序里抽取代码时，其他同名的代码片段可基于该标签，以 ^+ 或 + 运算符告诉 xi 程序，自身内容应当追加到含有该标签的代码片段之前或之后。

基于后缀宏机制，为样式表的访问和修改过程定义界面：

```
@ 流程图节点 # <模块参数访问和设定> +                                    174
<snailmod 后缀宏>
vardef snailmod@#(text anything) =
    save s; string s; s := str @#;
    if s = "get": snail_mod_get(anything) fi
    if s = "set": snail_mod_set(anything) fi
    # 基于后缀 s，将参数 anything 分发给其他宏 @ <184>
enddef;
```

上述代码片段使用代码片段标签「<模块参数访问和设定>」和运算符 +，表示该代码片段内容应追加到含有该标签的同名片段之后。另外，上述代码片段自身也含有一个标签「<snailmod 后缀宏>」，以便在定义其他同名代码片段时作为代码合并的又一个基准。还需要注意的是，上述代码片段使用了 xi 文学编程的一个常用技巧，即一个代码片段内部若后续还需要扩充内容，可预设一个代码片段引用，然后在适当的时机再给出其定义。

11.3 锚点

虽然用于构造三种流程图节点的宏尚未定义出来，但是基于前面一些章节的探索，结合 snail_t 以及样式表的定义，可以先将节点的锚点构造过程定义出来，如下：

```
@ 流程图节点 # +                                                        175
def make_anchors suffix obj =
    % 基于 obj 的边界构造锚点
    obj.中央:= center obj;
    obj.东北:= urcorner obj; obj.西北:= ulcorner obj;
```

```
    obj.西南:= llcorner obj; obj.东南:= lrcorner obj;
    obj.子门:= .5[obj.西北, obj.东北]; obj.午门:= .5[obj.西南, obj.东南];
    obj.卯门:= .5[obj.东南, obj.东北]; obj.酉门:= .5[obj.西南, obj.西北];
    obj.丑门:= .5[obj.子门, obj.东北]; obj.寅门:= .5[obj.卯门, obj.东北];
    obj.辰门:= .5[obj.卯门, obj.东南]; obj.巳门:= .5[obj.午门, obj.东南];
    obj.未门:= .5[obj.午门, obj.西南]; obj.申门:= .5[obj.酉门, obj.西南];
    obj.戌门:= .5[obj.酉门, obj.西北]; obj.亥门:= .5[obj.子门, obj.西北];
    % 基于散射路径与 obj.outline 求交，令锚点落在 obj 上
    forsuffixes i = 东北, 西北, 西南, 东南,
                    子门, 卯门, 午门, 酉门, 丑门, 寅门, 辰门, 巳门, 未门, 申门, 戌门, 亥门:
        obj.i := make_anchor_by_ray(obj.中央, obj.i, obj.outline);
    endfor;
    % 构造外围锚点
    make_anchors_in_third_level(obj, (boundingbox obj) enlarged snailmod.get("frame.margin"));
    % 为讨厌古典的人构造锚点二维数组
    make_anchor_array(obj);
enddef;
vardef make_anchor_by_ray(expr a, b, cutter) =
    save thi, p; numeric thi; path p;
    thi := snailmod.get("path.thickness");
    p := a -- (b shifted (2 * thi * unitvector(b - a)));
    p := p cutafter cutter;
    (point 1 along p)
enddef;
def make_anchors_in_third_level(suffix obj)(expr margin) =
    obj.艮位:= (ulcorner margin); obj.震位:= (urcorner margin);
    obj.兑位:= (lrcorner margin); obj.巽位:= (llcorner margin);
    obj.坤位:= .5[obj.艮位, obj.震位]; obj.乾位:= .5[obj.巽位, obj.兑位];
    obj.离位:= .5[obj.兑位, obj.震位]; obj.坎位:= .5[obj.巽位, obj.艮位];
enddef;
def make_anchor_array(suffix obj) =
    begingroup
    % 为讨厌十二地支的人们准备的二维锚点数组
    save i; numeric i; i := 1;
    obj.anchors[0][0] := obj.中央;
    forsuffixes it = 卯门, 寅门, 东北, 丑门, 子门, 亥门, 西北, 戌门,
                     酉门, 申门, 西南, 未门, 午门, 巳门, 东南, 辰门:
        obj.anchors[1][i] := obj.it;
        i := i + 1;
    endfor;
    i  := 1;
    forsuffixes it = 离位, 震位, 坤位, 艮位, 坎位, 巽位, 乾位, 兑位:
        obj.anchors[2][i ] := obj.it;
        i  := i  + 1;
    endfor;
    endgroup
enddef;
```

make_anchors 宏使用的 enlarged 是 MetaFun 运算符宏，该宏可将包围盒按给定尺度扩大。注意，make_anchor_array 虽然含 suffix 类型的参数，但是其内部变量 i 依然是足够安全的，你知道为什么吗？原因很简单，只要宏内没有曲解 i 类型及用途的语句，即使 suffix 类型的参数是 i，也是无妨的，宏内没有那条语句将 i 视为其他类型的变量，所以也不必为宏含有 suffix 类型的参数过于提心吊胆。

需要解释一下 make_anchor_array 宏构造的二维数组的下标含义。假设存在节点 foo，使用 make_anchors 为其构造锚点，倘若你既不能接受以十二地支命名的 foo 边界上的锚点，更不能接受以八卦命名的位于 foo 边界外围的锚点，你可以使用 foo.anchors 来获得这些锚点的位置，前提是你需要理解该数组下标的含义。foo.anchors[0][0] 是 foo 的中心点。foo.anchors[1][1] 至 foo.anchors[1][12] 为十二地支对应的锚点，其中 foo.anchors[1][1] 是 foo 的边框上的正右方锚点，亦即边框与 x 轴的交点，从它开始，按逆时针顺序直至 foo.anchors[1][12]。同理，foo.anchors[2][1] 是 foo 的正右方的外围锚点，从它开始，按逆时针顺序直至 foo.anchors[2][8]。

定义 snail_draw_anchors 宏，用于画出一系列节点的锚点：

```
@ 流程图节点 # +                                                                    176
def snail_draw_anchors text all =
    begingroup
    save s, p; numeric s; path p;
    s := snailmod.get("anchorsize");
    forsuffixes foo = all:
        draw foo.中央 withpen pensquare scaled s withcolor transparent(1, .3, red);
        forsuffixes i = 乾位, 坤位, 艮位, 兑位, 巽位, 震位, 坎位, 离位:
            draw foo.i withpen pensquare scaled s withcolor darkgreen;
        endfor;
        p := foo.离位 -- foo.震位 -- foo.坤位 -- foo.艮位
             -- foo.坎位 -- foo.巽位 -- foo.乾位 -- foo.兑位 -- cycle;
        draw p withdashes s withcolor transparent (1, .5, darkcyan);
        if known foo.content: % 不画空节点的边界锚点
            forsuffixes i = 子门, 卯门, 午门, 酉门:
                draw foo.i withpen pencircle scaled s withcolor darkred;
            endfor;
            forsuffixes i = 丑门, 寅门, 辰门, 巳门, 未门, 申门, 戌门, 亥门:
                draw foo.i withpen pencircle scaled (s/2) withcolor magenta;
            endfor;
            forsuffixes i = 东北, 东南, 西南, 西北:
                draw foo.i withpen pensquare scaled s withcolor darkblue;
            endfor;
        fi;
    endfor;
    endgroup
enddef;
```

上述代码所用的 anchorsize 设定，其定义如下：

```
@ 其他元素的样式 #                                                                  177
anchorsize = '4pt',
```

11.4 构造无框节点

我将构造无框节点的宏取名为 slug，即蛞蝓（读作「阔余」），俗称鼻涕虫，像是没有壳的蜗牛，其定义为

```
@ 流程图节点 # +                                                                    178
def slug (suffix obj) (expr s) text somewhere =
    if s = "": % 当 s 为空文本时，构造一个空节点
        obj := fullsquare scaled snailmod.get("text.fontsize");
    else:
        obj.content := image(draw textext("\slug{" & s & "}") withcolor snailmod.get("text.color"));
        obj := boundingbox obj.content;
    fi;
    obj.outline := boundingbox obj;
    obj.frame := false;
    make_anchors(obj);
    obj := obj somewhere;
    obj.outline := obj.outline somewhere;
    if known obj.content: obj.content := obj.content somewhere; fi;
    make_anchors(obj);
enddef;
```

上述代码里的 nullpicture 表示空画面对象。对于非空的无框节点，虽然无需绘制其边框，但是却使用 ConTEXt 宏 \slug 构造节点文字。\slug 的定义如下：

```
@ 定义 ConTeXt 宏 \slug\ #                                                          179
\defineframed[slug][offset=.25em, frame=off]
```

在此不便展开讲述 \slug 宏的含义，你只需要知道它能够构造一个不显示边框的 ConTEXt 文本框，其四周留白是 1/4 倍的正文字号。若想了解得更为深入一些，可阅读本文档前言提到的《ConTEXt 蹊径》第 10 章。

不要奇怪 slug 宏末尾两次调用了 make_anchors 宏，第一次调用是为了下一行代码里的 somewhere 里使用锚点。第二次调用，是因为 obj 的位置发生变化，需要重新构造一次锚点。

以下示例构造了两个无框节点，并画出它们的锚点：

```
@ 测试无框节点 #                                                                    180
# Snail 模块样式表 @ <172>
# 定义 ConTeXt 宏 \slug\ @ <179>
\startMPpage
# 定义 at 宏 @ <64>
# 流程图节点 @ <168>
snail_t foo, bar;
slug(foo, "Foo"); slug(bar, "Bar") at (4cm, 2cm);
draw foo; draw foo.content; draw bar; draw bar.content;
snail_draw_anchors foo, bar; % 画出 foo 和 bar 的锚点
\stopMPpage
```

若令节点文本为空字符串，用 slug 宏可以构造空节点。空节点能用来做什么呢？事实上，我并未想好它的用法，只是出于某种有实必有虚的念头而不得不实现它。不过，我能想到的一个场景是，空间点可以作为实节点的间隔，使得节点布局无需显式设定尺度，例如

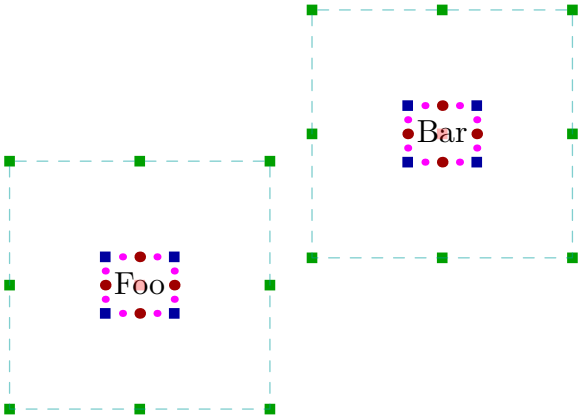


图 11.1　无框节点示例

```
@ 空节点示例 #                                                    181
# Snail 模块样式表 @ <172>
# 定义 ConTeXt 宏 \slug\ @ <179>
\startMPpage
# 定义 at 宏 @ <64>
# 流程图节点 @ <168>
snail_t a, b, empty;
slug(a, "Node a");
slug(empty, "") at a.震位;
slug(b, "Node b") at empty.震位;
draw a; draw empty; draw b;
snail_draw_anchors a, empty, b;
\stopMPpage
```

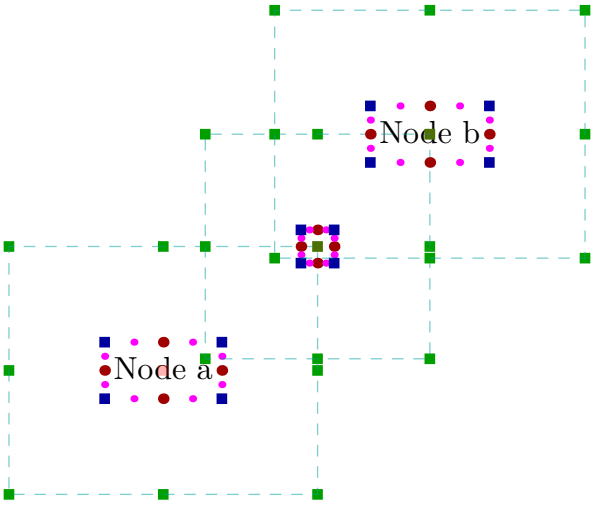


图 11.2　空节点示例

上述示例里的节点 empty 便是空节点，实际上它是直径为 BodyFontSize 的圆形区域，但是它也有 25 个锚点，故而其他节点可利用空节点的一些锚点定位自身。

11.5 有框节点

之前在多个章节里探讨了有框节点的构造，现在虽然是完全重新为其设计一个构造宏，但所用算法未变，仅仅是将节点构造过程与 Lua 样式表关联起来。用于构造有框节点的宏，其名为 snail，意为「蜗牛」，与模块同名，象征流程图中最为普遍的节点。snail 宏的定义如下：

```
@ 流程图节点 # +                                                    182
# 定义 convert_to_standard_shape 宏 @ <102>
def snail(suffix obj)(expr s) text somewhere =
    obj.frame := true;
    obj.content := textext(s) shifted snailmod.get("text.offset");
    obj := make_frame(obj.content);
    obj.outline := make_outline(obj);
    make_anchors(obj);
    obj := obj somewhere;
    obj.content := obj.content somewhere;
    obj.outline := obj.outline somewhere;
    make_anchors(obj);
enddef;
vardef make_frame(expr content) =
    save frame, padding;
    path frame; numeric padding;
    padding := snailmod.get("frame.padding");
    # 为 content 构造含有留白的包围盒 content_box @ <109>
    frame := convert_to_standard_shape(snailmod.get("frame.shape"));
    # 更快的 frame 放大过程 @ <113>
    if not snailmod.get("frame.isotropic"):
        # 调整 frame 的宽高比 @ <147>
    fi;
    frame
enddef;
vardef make_outline(expr obj) =
    save thi, w, h, p; numeric thi, w, h; path p;
    thi := snailmod.get("frame.thickness");
    w := bbwidth obj; h := bbheight obj;
    p := obj xysized (w + thi, h + thi);
    p := p shifted ((center obj) - (center p));
    p
enddef;
```

对于复杂形状的边框，节点文字的位置可能不够理想，需基于 `snailmod.text.offset` 参数对文字位置予以调整，该参数默认值为 `(0, 0)`，表示无需调整。倘若需要调整，则该参数表示文字的偏移向量。

以下代码用于测试有框节点的构造过程及基本效果：

```
@ 测试有框节点及其锚点 #                                            183
# Snail 模块样式表 @ <172>
# 定义 ConTeXt 宏 \slug\ @ <179>
\startMPpage
# 定义 at 宏 @ <64>
# 正则路径宏 @ <44>
# 流程图节点 @ <168>
def ellipse(suffix obj)(expr content) text somewhere =
    snailmod.set("frame.shape", "fullcircle");
    snail(obj, content) somewhere;
    snailmod.set("frame.shape", "fullsquare");
```

```
enddef;
% 构造矩形和椭圆形节点
snail_t foo, bar;
snail(foo, "Foo"); ellipse(bar, "Bar") at (4cm, 2cm);
draw foo; draw foo.content; draw bar; draw bar.content;
drawarrow foo.卯门 xyxto bar.酉门 withpen pencircle;
snail_draw_anchors foo, bar;
\stopMPpage
```

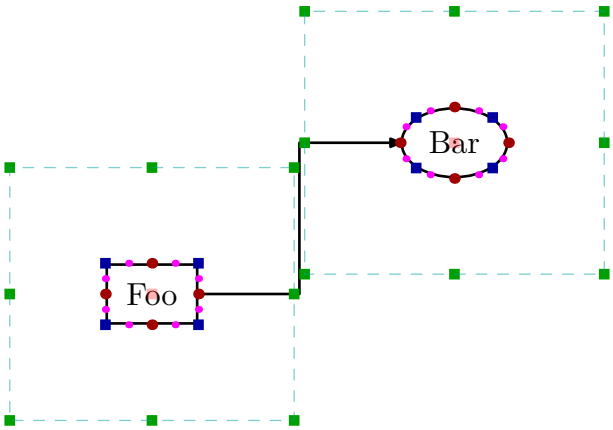


图 11.3　有框节点及其锚点示例

上述示例暴露出一个问题，当我们出于需要而修改样式表中的一些参数之后，若想恢复其原值，前提是需要记住或能够查阅到其原值。该过程对于模块的用户而言颇不友好。要解决该问题，需要为 snailmod 宏增加一个样式重置后缀：

```
@ 基于后缀 s, 将参数 anything 分发给其他宏 # +                                    184
if s = "reset":
    snail_mod_reset(anything)
fi
```

snail_mod_reset 宏的定义如下：

```
@ 流程图节点 # <snailmod 后缀宏>  ^+                                              185
def snail_mod_reset(expr parameter) =
    if parameter = "text.fontsize": snailmod.set(parameter, "BodyFontSize"); fi;
    if parameter = "text.color": snailmod.set(parameter, "black"); fi;
    if parameter = "text.offset": snailmod.set(parameter, "(0, 0)"); fi;
    if parameter = "frame.shape": snailmod.set(parameter, "fullsquare"); fi;
    if parameter = "frame.background.fill": snailmod.set(parameter, false); fi;
    if parameter = "frame.background.color": snailmod.set(parameter, "white"); fi;
    if parameter = "frame.isotropic": snailmod.set(parameter, false); fi;
    if parameter = "frame.padding": snailmod.set(parameter, ".5BodyFontSize"); fi;
    if parameter = "frame.margin": snailmod.set(parameter, "2BodyFontSize"); fi;
    if parameter = "frame.color": snailmod.set(parameter, "black"); fi;
    if parameter = "frame.thickness": snailmod.set(parameter, "1pt"); fi;
    # 扩充 snail_mod_reset 宏 @ <191>
enddef;
```

上述宏定义里预留了扩充区域。基于 snailmod 的 reset 后缀，则上述示例中用于构造椭圆节点的宏可修改为

```
def ellipse(suffix obj)(expr content) text somewhere =
    snailmod.set("frame.shape", "fullcircle");
    snail(obj, content) somewhere;
    snailmod.reset("frame.shape");
enddef;
```

通过上述节点构造示例，想必你应该能理解，我们所实现的流程图模块，节点的形状是开放的。对于该模块上不支持的节点形状，你可以仿照 ellipse 宏的方式定义你需要的形状。

11.6 插图节点

用于构造插图节点的宏叫作 avatar，意为化身，表示蜗牛通过图像的形式幻化成各种形态。在定义 avatar 宏之前，需要了解一下 MetaPost 语言如何引入外部图片，例如

```
@ 引入外部图片示例 #                                                    186
\startMPpage
picture a;
a := externalfigure "foo.png";
draw a scaled 3;
drawoptions(withpen pencircle scaled 2pt withcolor darkred);
drawarrow (0, 0) -- (4.5cm, 0); drawarrow (0, 0) -- (0, 4.5cm);
\stopMPpage
```

假设上述示例保存在 foo.tex 文件里，在该文件所在目录下有位图文件 foo.png，则使用 context 命令编译上述示例后，所得结果如图 11.4 所示。



图 11.4　引入外部图片示例

externalfigure 是 MetaFun 扩展的原语，除了用于引入各种位图之外，也能用于引入 SVG 和 PDF 等格式的矢量图形。所有引入的外部图形，皆表示为 picture 类型的对象。另外，由图 11.4 可以发现，externalfigure 引入的图形，其左下角点是原点。

掌握上述上述知识，便可定义 avatar 宏：

```
@ 流程图节点 # +                                                        187
def avatar (suffix obj) (expr a, w, h) text somewhere =
    obj.content := make_avatar(a, w, h);
    obj.content := obj.content shifted -(center obj.content); % 令插图中心对准原点
    obj := boundingbox obj.content;
    obj.outline := obj;
    obj.frame := false;
    make_anchors(obj);
```

```
    obj := obj somewhere;
    obj.outline := obj.outline somewhere;
    obj.content := obj.content somewhere;
    make_anchors(obj);
enddef;
vardef make_avatar(expr a, w, h) =
    save p, f; picture p; numeric f;
    p := externalfigure a;
    f := (bbwidth p) / (bbheight p);
    if numeric w and numeric h:
        p := p xysized (w, h);
    else:
        if numeric w:
            p := p xysized (w, w / f);
        fi;
        if numeric h:
            p := p xysized (h * f, h);
        fi;
    fi;
    p
enddef;
```

avatar 的参数 a 是字符串，表示外部图片的路径，w 和 h 分别用于构造插图节点时设定图片的宽度和高度，若仅设定其中之一，另一个值可根据图形宽高比自动确定。由 avatar 宏构造的插图节点，其中心默认为原点，可通过定位语句 somewhere 移动到指定位置。

以下代码用于测试 avatar 宏：

```
@ 插图节点示例 #                                                    188
# Snail 模块样式表 @ <172>
# 定义 ConTeXt 宏 \slug\ @ <179>
\startMPpage
# 定义 at 宏 @ <64>
# 正则路径宏 @ <44>
# 流程图节点 @ <168>
snail_t foo, bar;
snail(foo, "Foo");
avatar(bar, "foo.png", 2cm, "auto") at (6cm, 2cm);
draw foo.content; draw bar.content;
drawarrow foo.卯门 xyxto bar.酉门 withpen pencircle;
snail_draw_anchors foo, bar;
\stopMPpage
```

结果见图 11.5。

在使用 avatar 构造插图节点时，插图的宽度和高度，二者可只设其一，而另一个只需设为非 numeric 类型的对象即可，例如上述示例使用的是字符串 "auto"，实际上可以是任意字符串。

11.7 节点辅助宏

定义 showsnails 宏，实现一次画多个蜗牛：

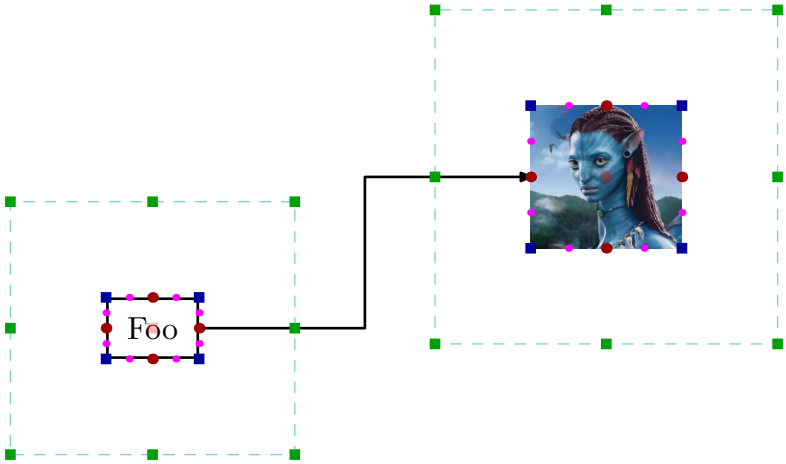


图 11.5　插图节点示例

```
@ 流程图节点 # +                                                                    189
def showsnails text all =
    begingroup
    forsuffixes it = all:
        if snailmod.get("frame.background.fill"):
            fill it withcolor snailmod.get("frame.background.color");
        fi;
        if it.frame:
            draw it withpen pencircle scaled snailmod.get("frame.thickness")
                                            withcolor snailmod.get("frame.color");
        fi;
        if known it.content: draw it.content withcolor snailmod.get("text.color"); fi;
    endfor;
    if snailmod.get("debug"): forsuffixes it = all: snail_draw_anchors it; endfor; fi;
    endgroup
enddef;
```

如果流程图样式表中的参数 `snailmod.debug` 值为 `'true'`，`showsnails` 会将节点的锚点一并画出。现在，将这个参数补充到流程图样式表中，

```
@ 其他元素的样式 # +                                                                190
debug = 'false',
```

并为其定义重置操作：

```
@ 扩充 snail_mod_reset 宏 # +                                                       191
if parameter = "debug":
    snailmod.set(parameter, false);
fi;
```

`showsnails` 的用法如下：

```
snail_t empty, foo, bar;
snail(foo, "i am foo");
slug(empty, "") at foo.震位;
snail(bar, "i am bar") at empty.震位;
showsnails empty, foo;
```

```
snailmod.set("debug", true); % 开启调试模式
showsnails bar; % 画出节点 bar 及其锚点
snailmod.reset("debug");
```

将之前定义的 at 宏也作为节点辅助宏，然后定义 put 宏，用于将节点放置在指定位置：

```
@ 流程图节点 # +                                                    192
# 定义 at 宏 @ <64>
def put (suffix obj, anchor) text somewhere =
  obj := (obj shifted -obj.anchor) somewhere;
  obj.outline := (obj.outline shifted -obj.anchor) somewhere;
  if known obj.content:
      obj.content := (obj.content shifted -obj.anchor) somewhere;
  fi;
  make_anchors obj;
enddef;
```

at 可与 put 配合使用，从而让节点定位操作具备更为直观的语义，例如

```
snail_t foo;
snail(foo, "Foo");
put(foo, 中央) at (3cm, 2cm);
```

下面定义的 makegrid 宏可基于空节点构造网格，showgrid 宏可画出网格：

```
@ 流程图节点 # +                                                    193
def makegrid(suffix name)(expr m, n) =
    for i = 1 upto m:
        slug(name[i][1], "");
        if i > 1:
            put (name[i][1], 坤位) at name[i - 1][1].乾位;
        fi;
        for j = 2 upto n:
            slug(name[i][j], "");
            put (name[i][j], 坎位) at name[i][j - 1].离位;
        endfor;
    endfor;
enddef;
def showgrid(suffix name)(expr m, n) =
    snailmod.set("debug", true);
    for i = 1 upto m:
        for j = 1 upto n:
            showsnails name[i][j];
        endfor;
    endfor;
    snailmod.reset("debug");
enddef;
```

11.8 小结

所谓流程图，由无壳的，带壳的，以及善于幻化的蜗牛构成，它们之间存在的一些连系可以形成网络。在这网络中的每条路径这头的一只蜗牛，大概会以为，只要自己沿着这条路径向另一头慢吞吞地爬过去，到达终点时，就会变成另一只蜗牛……

12 蜗牛的路

流程图中的蜗牛，虽然爬得很慢，但它们依然严肃地走出一条又一条正则路径，甚至在拐弯的时候，会走出一道优美的弧线，颇为学究。有的时候，它们好像是在走一条好看的曲线路径，但实际上它依然严肃地认为，自己是走出的是正则路径。有的时候，它们会跳着爬。

12.1 路径样式

在探讨如何构造流程图节点之间的路径前，需要向流程图样式表中增加路径样式的定义：

```
@ 其他元素的样式 # +                                                    194
path = {
    thickness = '1pt', color = 'black', background = 'true',
    smooth = 'true', fillet = '.5BodyFontSize',
    arrow = 'true', ahvariant = 1, ahdimple = 1, ahlength = '.5BodyFontSize',
    # 扩充路径样式 @ <208>
},
```

上述样式表中设定了路径箭头的样式，先将其转化为 MetaPost 为箭头定义的全局变量的值：

```
@ 流程图路径 #                                                          195
ahvariant := snailmod.get("path.ahvariant");
ahdimple := snailmod.get("path.ahdimple");
ahlength := snailmod.get("path.ahlength");
```

至于样式表中路径的其他参数的用法，待构造具体路径时，再行解释。

12.2 路

定义 road 宏，用于构造蜗牛之间的路径：

```
@ 流程图路径 # +                                                        196
# 正则路径宏 @ <44>
# 改进的 smooth_regular_path 宏 @ <80>
# 定义 is_colinear 宏 @ <83>
# 定义 simplify_regular_path 宏 @ <84>
vardef road(expr p) =
    if snailmod.get("path.smooth"):
        smooth_regular_path(simplify_regular_path(p), snailmod.get("path.fillet"))
    else:
        p
    fi
enddef;
```

road 宏以正则路径为参数，根据 snailmod.path.smooth 参数的值是否为 true 决定是否对正则路径作简化和柔化处理。

snailmod.path.smooth 参数的值默认是 true，倘若将其值修改为 false，则 road 会将正则路径作为结果。下面为该参数定义重置操作：

```
@ 扩充 snail_mod_reset 宏 # +                                           197
```

```
if parameter = "path.smooth":
    snailmod.set(parameter, true);
fi;
```

将 8.4 节定义的 draw_regular_paths 宏改造为 showroads 宏，用于显示一系列路径：

```
@ 流程图路径 # +                                                                    198
def showroads text paths =
    begingroup
    save bg_path, bg, thi, bg_thi, has_arrow, road_color;
    path bg_path; boolean bg; numeric thi, bg_thi;
    boolean has_arrow; color road_color;
    bg := snailmod.get("path.background");
    thi := snailmod.get("path.thickness");
    bg_thi := 7 * thi;
    has_arrow := snailmod.get("path.arrow");
    road_color := snailmod.get("path.color");
    for i = paths:
        if bg:
            bg_path := i cutbefore (point .5bg_thi on i);
            bg_path := bg_path cutafter (point -.5bg_thi on bg_path);
            draw bg_path withpen pencircle scaled bg_thi withcolor background;
        fi;

        if has_arrow:
            drawarrow i withpen pencircle scaled thi withcolor road_color;
        else:
            draw i withpen pencircle scaled thi withcolor road_color;
        fi;
    endfor;
    endgroup
enddef;
```

以下代码用于测试 road 和 showroads 宏，结果如图 12.1 所示。

```
@ 蜗牛之路：示例一 #                                                                199
# Snail 模块样式表 @ <172>
# 定义 ConTeXt 宏 \slug\ @ <179>
\startMPpage
# 流程图节点 @ <168>
# 流程图路径 @ <195>
snail_t foo, bar;
snail(foo, "Foo");
snail(bar, "Bar") at (5cm, 3cm);
showsnails foo, bar;
showroads road(foo.卯门 xyxto bar.酉门), road(foo.子门 yxyto bar.午门);
\stopMPpage
```

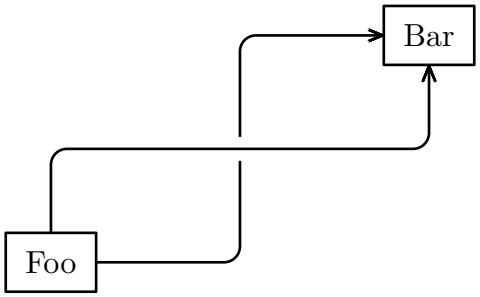


图 12.1　蜗牛之路：示例一

无论你曾经有多么厌恶宏编程语言，也无论你在遇到 MetaPost 代码出错时有多么抓狂，可是你不得不承认，宏编程语言在语法层面上的表现力，特别是 MetaPost 这样的语言，是非宏编程语言难以企及的。有所失，必有所得。

12.3 正则曲线

我们尝试构造一种也许是前所未有的曲线。对于任意一条正则路径 p，设它由节点集合 $\{a_0, a_1, \cdots, a_n\}$ 定义，若将其任一中间节点 a_i 替换为 $b_i = ta_i + (1-t)\dfrac{a_{i-1}+a_{i+1}}{2}$，$0 < i < n$，然后 MetaPost 的曲线路径构造符 .. 以及节点集合 $\{a_0, b_1, \cdots, b_{n-1}, a_n\}$ 定义路径 q，

$$q := a_0 \mathinner{..} b_1 \mathinner{..} b2 \mathinner{..} \cdots \mathinner{..} b_{n-1} \mathinner{..} a_n;$$

q 会是什么样子呢？

为了回答这个问题，先将上述算法定义为宏 tocurve：

```
@ 定义 tocurve 宏 #                                                    200
vardef tocurve(expr p, t) =
    save a, n; pair a[]; numeric n;
    n := length p;
    if n < 2: p
    else:
        for i = 0 upto n: a[i] := point i of p; endfor;
        a[0] for i = 1 upto n - 1: .. t[a[i], .5[a[i - 1], a[i + 1]]] endfor .. a[n]
    fi
enddef;
```

尽管你会觉得上述代码的倒数第 3 行颇为不可思议，可它确能构造出像下面这样的路径：

```
a[0] .. t[a[1], .5[a[0], a[1]]] .. ... .. t[a[n - 1], .5[a[n - 2], a[n]]] .. a[n]
```

以下示例用于测试 tocurve 的效果：

```
@ 蜗牛之路：示例二 #                                                   201
# Snail 模块样式表 @ <172>
# 定义 ConTeXt 宏 \slug\ @ <179>
\startMPpage
# 流程图节点 @ <168>
# 流程图路径 @ <195>
# 定义 tocurve 宏 @ <200>
snail_t foo, bar;
snail(foo, "Foo");
```

```
snail(bar, "Bar") at (5cm, 3cm);
showsnails foo, bar;
showroads tocurve(foo.卯门 xyxto bar.酉门, .75), road(foo.子门 yxyto bar.午门);
\stopMPpage
```

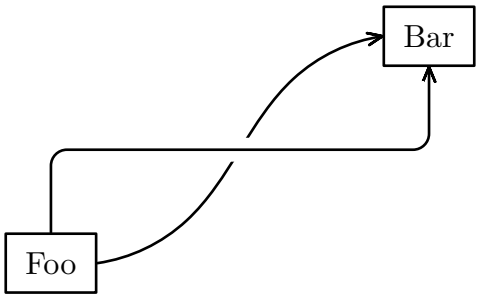


图 12.2　蜗牛之路：示例二

上述示例调用 tocurve 宏时，其第 2 个参数取 0.75，该参数决定 tocurve 所构造曲线的弯曲程度。以下示例可展示该参数对曲线形状的影响：

```
@ tocurve 第二个参数的几何意义 #                                                    202
\startMPpage
# 定义 tag 宏 @ <122>
# 定义 tocurve 宏 @ <200>
pair a, b; path p, q;
a := (0, 0); b := (5cm, 5cm);
p := a -- (5cm, 0) -- b;
draw p withpen pencircle scaled 2pt withcolor darkred;
for t = -.25 step .25 until 1.5:
    q := tocurve(p, t);
    draw q withpen pencircle;
    draw tag(q, .5, "{\tfx$" & decimal t & "$}") shifted (0, 8pt);
endfor;
pickup pencircle scaled 4pt;
draw a withcolor darkred; label.llft("$a$", a);
draw b withcolor darkblue; label.urt("$b$", b);
\stopMPpage
```

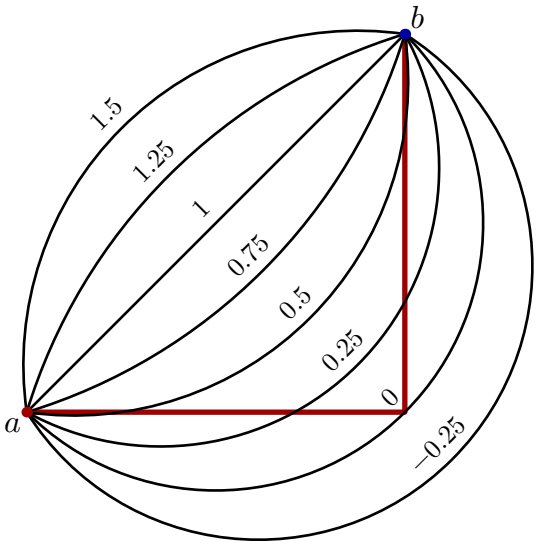


图 12.3　tocurve 宏的第 2 个参数的几何意义

事实上，即使向 tocurve 传入其他形式的非封闭路径，也是可行的。需要注意的是，若 tocurve 接受的路径只有两个节点，即直线路径，则曲线构造结果为该直线路径本身。

12.4 正则封闭曲线

如果让 tocurve 构造的曲线封闭，结果会如何呢？设 p 为封闭路径，则其节点集合 $\{a_0, a_1, \cdots, a_n\}$ 不再区分端点和中间节点了，而是每个节点皆为中间节点，亦即在封闭路径中，a_0 和 a_n 是同一个节点，并且需要由其本身及其前后两个节点确定 b_0，亦即 b_n。a_0 的前一个点是 a_{n-1}。基于这些条件，可以定义一个能基于封闭路径构造封闭曲线的宏：

```
@ 定义 toclosedcurve 宏 #                                                    203
vardef toclosedcurve(expr p, t) =
    save a, n; pair a[]; numeric n;
    n := length p;
    if n < 2: p
    else:
        for i = 0 upto n: a[i] := point i of p; endfor;
        a[-1] := a[n - 1];
        for i = 0 upto n - 1: t[a[i], .5[a[i - 1], a[i + 1]]] .. endfor cycle
    fi
enddef;
```

以下代码可测试 toclosedcurve 宏并揭示其第 2 个参数的几何意义：

```
@ 正则封闭曲线示例一 #                                                      204
\startMPpage
# 定义 tag 宏 @ <122>
# 定义 toclosedcurve 宏 @ <203>
path p, q;
p := fullsquare xyscaled (4cm, 1cm);
draw p withpen pencircle scaled 2pt withcolor darkred;
for t = -.5 step .25 until .75:
    q := toclosedcurve(p, t);
    draw q withpen pencircle;
    draw tag(q, 0, "{\tfx$" & decimal t & "$}") shifted (0, 6pt);
endfor;
\stopMPpage
```

上述示例基于一个矩形构造出来的封闭曲线像是一系列略微有些不规则的圆，如图 **12.4** 所示，也许这并不能让你觉得正则封闭曲线有什么用处。倘若将矩形换成一个有些扁的菱形，也许就会让你觉得 toclosedcurve 有些用处，例如以下示例可基于菱形构造似椭圆而非椭圆的封闭曲线，结果如图 **12.5** 所示。

```
@ 正则封闭曲线示例二 #                                                      205
\startMPpage
# 定义 tag 宏 @ <122>
# 定义 toclosedcurve 宏 @ <203>
path p;
p := fulldiamond xyscaled (7cm, 3cm);
draw p withpen pencircle scaled 2pt withcolor darkgray;
```

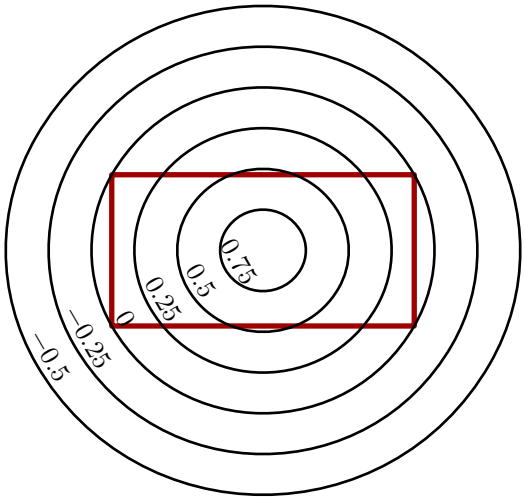


图 12.4　基于矩形构造的封闭曲线

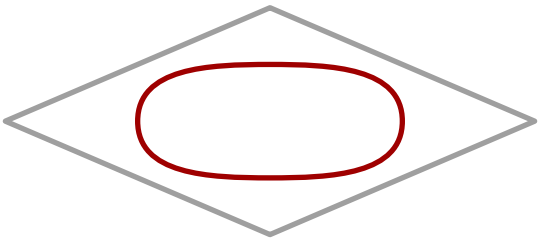


图 12.5　似椭圆而非椭圆

```
draw toclosedcurve(p, .5) withpen pencircle scaled 2pt withcolor darkred;
\stopMPpage
```

将 tocurve 和 toclosedcurve 宏的定义并入流程图路径代码：

```
@ 流程图路径 # +                                                    206
# 定义 tocurve 宏 @ <200>
# 定义 toclosedcurve 宏 @ <203>
```

12.5 虚线

有时，路径需要以虚线的形式画出，而我从未介绍过如何用 MetaPost 画虚线。以下代码将路径画成以 1mm 的空白作为间隔的虚线：

```
@ 虚线示例 #                                                        207
\startMPpage
% 构造虚线模式
picture a;
a := dashpattern(on 3mm off 2mm);
% 构造路径 p
path p;
p := (0, 0) -- (7cm, 0);
% 以虚线画出路径 p
draw p dashed a withpen pencircle withcolor darkred;
% 以实线画出偏移的 p
draw p shifted (0, -5mm) withpen pencircle;
\stopMPpage
```

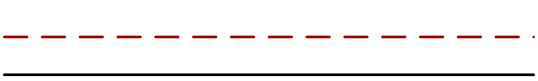


图 12.6　虚线示例

上述示例中的 dashpattern 语句可以构造一个图案，该图案由 on 和 off 部分构成，前者宽度皆为 3mm，后者宽度为 2mm。on 部分表示可见，off 表示不可见。若将该图案当成印章，沿着路径 p 一路重复盖下去，亦即 p dashed a，便可得到 p 的虚线形式，此即 MetaPost 构造虚线的原理，实际上也是一种骗术。

在画流程图时，节点之间的路径也许需要画成虚线。对路径样式加以扩充，设定默认的虚线图案：

```
@ 扩充路径样式 #                                                    208
dashpattern = 'dashpattern(on 3pt off 3pt)',
```

该样式设定的重置操作如下：

```
@ 扩充 snail_mod_reset 宏 # +                                       209
if parameter = "path.dashpattern":
    snailmod.set(parameter, 'dashpattern(on 3pt off 3pt)');
fi;
```

基于上述样式以及 showroads 宏，便可定义一个专用于画虚线路径的宏：

```
@ 流程图路径 # +                                                    210
def showdashedroads text paths =
    begingroup
    save pat; picture pat;
    pat := snailmod.get("path.dashpattern");
    drawoptions(dashed pat);
    showroads paths;
    drawoptions();
    endgroup
enddef;
```

用 drawoptions 宏将虚线模式设定为 draw 语句的默认选项，待画完所有路径后，用空参数形式的 drawoptions 宏将之前的设定清除。不过，需要注意的是，若是在调用 showdashedroads 宏之前用 drawoptions 设定了绘图选项，所作设定会被 showdashedroads 宏内部的 drawoptions 语句替换。

以下代码用于测试 showdashedroads 宏：

```
@ 蜗牛之路：示例三 #                                                211
# Snail 模块样式表 @ <172>
# 定义 ConTeXt 宏 \slug\ @ <179>
\startMPpage
# 流程图节点 @ <168>
# 流程图路径 @ <195>
snail_t foo, bar;
snail(foo, "Foo");
snail(bar, "Bar") at (5cm, 3cm);
showsnails foo, bar;
showdashedroads tocurve(foo.卯门 xyxto bar.酉门, .75);
showroads road(foo.子门 yxyto bar.午门);
\stopMPpage
```

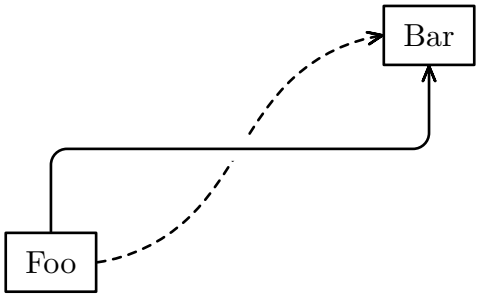


图 12.7　节点虚线路径示例

12.6 直连

有时，你会希望直接在两个节点之间直接构造路径，而不是通过它们的锚点。对于该需求，可基于两个节点的中心构造路径，然后用两个节点的边框裁剪路径两端。我们将该操作定义为运算符宏：

```
@ 流程图路径 # +                                                    212
tertiarydef a => b =
    begingroup
    save p, a_c, b_c, thi, comp;
    path p; pair a_c, b_c, comp; numeric thi;
    a_c := center a; b_c := center b;
    p := a_c -- b_c;
    thi := snailmod.get("path.thickness");
    comp := thi * unitvector(b_c - a_c); % 边框线宽补偿
    p := p cutbefore (a shifted comp);
    p := p cutafter (b shifted -comp);
    p
    endgroup
enddef;
```

以下代码用于测试 => 宏：

```
@ 蜗牛之路：示例四 #                                                213
# Snail 模块样式表 @ <172>
# 定义 ConTeXt 宏 \slug\ @ <179>
\startMPpage
# 流程图节点 @ <168>
# 流程图路径 @ <195>
snail_t a, b, c;
slug(a, "A"); snail(b, "B") at (3cm, 1.5cm); slug(c, "C") at (6cm, 3cm);
snailmod.set("debug", true);
showsnails a, b, c;
showroads a => b, b => c;
\stopMPpage
```

12.7 小结

蜗牛能走出来的路，大概就这几种，以后也许还能走出一些新花样。路径的样式表是保持开放的，本章也提供了曲线和虚线这些路径样式的扩展实例，能否创造出新花样，取决于你的想象力。

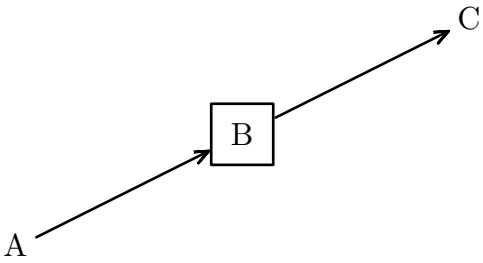


图 12.8　节点直连

13 路上的 NPC

当一只蜗牛奋力在路上向前爬行时，他可能会忘记这条路叫作什么。忘记路的名字，也许并不是什么严重的事情，你还记得自己所走的这条路的名字吗，它还是之前的那条路吗？可是蜗牛认为，他可以忘记这条路的名字，但是这条路有义务报出它的名字。为了满足蜗牛的这个执念，我们有必要在一些有义务报出自己名字的路上设置一些 NPC，倘若你玩过游戏，应该理解这三个字母是 Non-Play Character 的简写，由 NPC 负责报名。

13.1 样式表

你肯定猜到了，并没有什么 NPC，有的只是路径标注，它们就像我们的城市里的路牌。不过，既然已经用了这个比喻，就继续用下去。NPC 需要以下样式：

```
@ 其他元素的样式 # +                                                    214
tag = {
    upright = 'true', reverse = 'false', padding = "1pt"
},
```

upright 表示路径上的标注文字是否为直立，默认为直立，亦即无论路径是横向的，还是纵向的，还是曲线形式的，标注文字保持直立状态，不会顺着路径的方向而倾斜。reverse 控制标注文字是否反向，默认为正向。padding 用于控制标注文字和路径的间距。以下代码实现了这三个样式参数的重置操作：

```
@ 扩充 snail_mod_reset 宏 # +                                           215
if parameter = "tag.upright":
    snailmod.set(parameter, true);
fi;
if parameter = "tag.reverse":
    snailmod.set(parameter, false);
fi;
if parameter = "tag.padding":
    snailmod.set(parameter, "1pt");
fi;
```

13.2 不爱说话的 NPC

第一种 NPC 不太爱说话，其原型是 **7.3** 节定义的 tag 宏。我们依然用这个宏名，原因是它足够短，只是要让它成为名副其实的 NPC，需要费一些心思。

样式表中默认将标注文字设为直立，但无论标注文字是否为直立，要解决的关键问题是，如何让标注文字处于路径上指定位置附近，使之不与路径相交。我依然选择粗暴但简单的算法解决这个问题。设待标注的路径为 p，其上有一点 c，p 在 c 处的切向和法向分别为 v1 和 v2，再设 s 为标注文字，pic_s 为 textext(s) 构造的 picture 对象，其中心与 c 重合。在这些已知条件下，将 pic_s 定位到 c 点附近且不与 p 相交的粗暴算法是，沿着 p 在 c 点的法向不断移动 pic_s，直至其包围盒不与 p 相交为止。以下代码实现了该算法：

```
@ 沿 p 在 c 点的法向移动 pic_s，直至 pic_s 的包围盒与 p 不相交为止 #      216
save u; numeric u;
u := snailmod.get("tag.padding");
for i = 1 upto infinity:
```

```
    pic_s := pic_s shifted (u * v2);
    box_s := box_s shifted (u * v2);
    if (box_s intersectiontimes p) = (-1, -1):
        exitif true;
    fi;
endfor;
pic_s := pic_s shifted (u * v2);
```

上述代码移动 pic_s 的过程中，每次移动的距离 u 是标注文字和路径的间距，这意味着该过程结束后，标注字体到路径的最近距离介于 0 和 u 之间，即可能略微存在一些间距，为了保证间距必须存在，故而最后又补偿了一个间距。

基于上述算法的实现，将 tag 宏重新定义为

```
@ 路径标注 #                                                                    217
vardef tag(expr p, f, s) text somewhere =
    save pic_s, box_s, c, v;
    picture pic_s; path box_s; pair c, v[];
    c := point f along p;
    v1 := unitvector(direction f along p); % 路径切向
    if (v1 dotprod right) < 0: v1 := -v1; fi;
    v2 := v1 rotated 90;
    pic_s := textext(s) somewhere;
    box_s := (boundingbox pic_s) somewhere;
    if snailmod.get("tag.upright"):
        pic_s := pic_s shifted c;
        box_s := box_s shifted c;
    else:
        pic_s := pic_s rotated angle(v1) shifted c;
        box_s := box_s rotated angle(v1) shifted c;
    fi
    # 沿 p 在 c 点的法向移动 pic_s，直至 pic_s 的包围盒与 p 不相交为止 @ <216>
    pic_s
enddef;
```

以下代码用于测试新的 tag 宏：

```
@ NPC 示例一 #                                                                  218
# Snail 模块样式表 @ <172>
# 定义 ConTeXt 宏 \slug\ @ <179>
\startMPpage
# 流程图节点 @ <168>
# 流程图路径 @ <195>
# 路径标注 @ <217>
snail_t foo, bar; snail(foo, "Foo"); snail(bar, "Bar") at (5cm, 2cm);
path p[];
p1 := foo.寅门 xyxto bar.酉门;
p2 := foo.子门 yxyto bar.午门;
p3 := foo.辰门 xyto bar.离位 xto bar.卯门;
showsnails foo, bar;
showroads p1, p2, p3;
```

```
snailmod.set("tag.upright", false); draw tag(p1, .4, "42"); snailmod.reset("tag.upright");
draw tag(p2, .7, "43");
snailmod.set("tag.reverse", true); draw tag(p3, .3, "44"); snailmod.reset("tag.reverse");
\stopMPpage
```

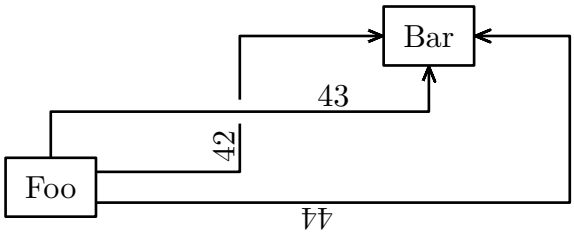


图 13.1　直立、躺平和倒立的 NPC

13.3 话痨 NPC

所谓话痨 NPC，就是路径标注可以用较长的文字，在 **7.6** 节已经实现了它的原型宏 `followtag`，现在对该宏的定义略作修改，使之与样式表关联起来，便可得到话痨 NPC。

```
@ 路径标注 # +                                                                    219
# offset_path 宏版本之二 @ <132>
# 定义 snap 宏 @ <137>
vardef followtag(expr p, s) =
    save d, pic_s, q; numeric d; picture pic_s; path q;
    pic_s := textext(s);
    d := .5snailmod.get("path.thickness") + 2.5 * snailmod.get("tag.padding");
    if snailmod.get("tag.reverse"): d := -d; fi;
    q := offset_path(p, d);
    if d < 0:
        lmt_followtext[path = q, text = s, spread = false, reverse = true]
    else:
        lmt_followtext[path = q, text = s, spread = false]
    fi
enddef;
```

以下示例用于测试新的 `followtag` 宏：

```
@ NPC 示例二 #                                                                    220
# Snail 模块样式表 @ <172>
# 定义 ConTeXt 宏 \slug\ @ <179>
\startMPpage
# 流程图节点 @ <168>
# 流程图路径 @ <195>
# 路径标注 @ <217>
snail_t foo, bar; snail(foo, "Foo"); snail(bar, "Bar") at (5cm, 2cm);
path p[];
p1 := foo.寅门 xyxto bar.酉门;
p2 := tocurve(foo.子门 yxyto bar.午门, .5);
p3 := foo.辰门 xyto bar.离位 xto bar.卯门;
showsnails foo, bar; showroads p1, p2, p3;
snailmod.set("tag.upright", false); draw tag(p1, .6, "42"); snailmod.reset("tag.upright");
```

```
draw followtag(p2, "do you known? i am 43!");
snailmod.set("tag.reverse", true); draw tag(p3, .3, "44"); snailmod.reset("tag.reverse");
\stopMPpage
```

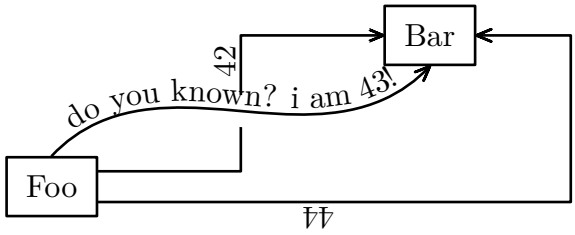


图 13.2　话痨 NPC

注意，上述重新定义的 `followtag` 依然能够与抓取路径子域的宏 `snap` 配合使用，例如

```
draw followtag(snap(p2, .1, .3), "i am 43!");
```

标注文字会出现在路径 `p2` 的 `[.1, .3]` 段。

13.4 借 ConTEXt 之力

无论是节点还是标注，其文字对象皆由 `textext` 宏构造，这意味着可以用 TEX 对文字作一些排版工作。由于我们是在 ConTEXt 环境中使用 MetaPost，而 ConTEXt 是一种 TEX，故而可以用一些 ConTEXt 宏排版流程图中的文字，例如改变其颜色和字号，例如

```
showtags tag(p1, .6, "\color[darkred]{\tfx i am 42!}");
```

可以将标注文字排版为深红色的文字，且其字号比正文小一号。当然，能从 ConTEXt 借多少力，取决于你对它的了解程度。只学 MetaPost，不学 ConTEXt，如同只用一条腿走路，反之亦然。

13.5 小结

至此，流程图模块的核心功能实现完毕，剩下的工作只是如何将这一切组织为 ConTEXt 模块文件。如果你不想推开 ConTEXt 的沉重之门，那么关于 MetaPost 语言的学习过程至此便可圆满，不必感谢我，要感谢蜗牛，因为它爬得够慢。

14 Snail 模块

建立 t-snail.mkxl 文件，其内容如下：

```
@ t-snail.mkxl #                                                    221
\writestatus{loading}{ConTeXt User Module / Snail}
\startmodule[snail]
# Snail 模块样式表 @ <172>
# 定义 ConTeXt 宏 \slug\ @ <179>
\startMPinclusions
# 流程图节点 @ <168>
# 流程图路径 @ <195>
# 路径标注 @ <217>
\stopMPinclusions
\stopmodule
\endinput
```

用 xi 程序从本文档的源文件 snail.xi 中抽取上述代码片段，保存为 t-snail.mkxl 文件，亦即

```
$ xi -t -e "t-snail.mkxl" -o t-snail.mkxl snail.xi
```

然后按照按照第 10 章所讲的 ConTEXt 模块安装方法，将 t-snail.mkxl 文件安装至 TEX 目录树并更新其索引。以我的 Linux 环境为例，只需

```
$ mkdir -p $HOME/opt/context/tex/texmf-local/tex/context/third/snail
$ cp t-snail.mkxl $HOME/opt/context/tex/texmf-local/tex/context/third/snail
$ context --generate
```

然后便可在 ConTEXt 源文档里加载 Snail 模块并是用它画流程图了，例如

```
\usemodule[snail]
\startMPpage
% 画流程图的代码
\stopMPpage
```

至于如何使用 Snail 模块画流程图，前面章节实际上已经介绍了所有的流程图宏的用法，我想大概是没必要再为它专门写一份教程了。

附录：xi 程序的用法

虽然在本文档的一些章节里，介绍了 `xi` 程序及其标记的用法，可是依然有必要为其提供一份独立且全面的介绍，一方面是为了没有耐心的读者能够一步到位理解 `xi` 程序所支持的文学编程机制，另一方面是为了已在本文档正文章节里对 `xi` 有所了解的读者能够随时快速回顾 `xi` 程序的用法。

文学编程

容易让人理解的程序应当以撰写一篇文章甚至一本书的方式去编写。一个程序可以视为是一栋建筑，通过文字去描述它，能够给人快速领会其本质的渠道或机会，而不至于迅速陷入程序源码的具体细节里，盲人摸象。可将程序的源码打碎，使之分散在文字铺设的诸多情节之中，如同一张又一张图纸，辅佐人类的语言，精确描画这栋建筑。这种编程方式便是文学编程，由 Donald Knuth 于 20 世纪 80 年代提出并付诸实践，从而孕育出 TEX 排版系统，且繁衍至今，形成枝叶繁茂的 TEX 家族。

文学程序本身可以使用某种排版语言或标记语言编写，然后转化为可供阅读或印刷的程序文档。若需要程序在计算机里运行，可从文档里以符合机器逻辑的形式完整提取程序代码，交由编译器生成具体程序。因此，文学程序通常是某种排版语言和编程语言的混合体。

Knuth 所写的文学编程工具名曰 WEB，该工具由两个程序构成，即 `tangle` 和 `weave`，前者可从文学程序中抽取程序代码，后者可将文学程序编织为可供排版的文档。WEB 工具仅支持 TEX 语言和 Pascal 语言，前者用于排版文档，后者用于编写程序。后来，Silvio Levy 主导设计了 CWEB，将文学编程的范式引入 C 语言世界。为了扩大排版和编程语言的支持范围，Norman Ramsey 设计并实现了新的文学编程工具 noweb。不过，noweb 默认支持 TeX，LaTeX 和 HTML 等排版语言，但若要支持其他排版语言，需要用户自行编写相应的后端程序添加到 noweb 的工作管线，该过程对于一般用户较为困难。

我觉得现在依然还活着的这些文学编程工具皆已颇为陈旧，于是不自量力，几经反复，以世人觉得已经老掉了牙的 C 语言写了 `xi` 程序。这个程序的名字，在汉语里可对应「兮」，表示该工具简单到只是清淡的语气，它几乎不依赖任何东西，包括排版语言和编程语言。

片段及其引用

文学编程强调的是「文学」，故而文档是宿主，程序则以代码片段的形式寄生于文档之内。`xi` 程序支持的代码片段，其基本形式为

```
@ 程序片段名 #
程序片段的内容
@
```

文档内容充斥于代码片段之间，例如

```
这是文档内容……

@ 程序片段 1 #
程序片段 1 的内容
@

这也是文档内容……

@ 程序片段 2 #
程序片段 2 的内容
@
```

……

在一个程序片段内，可以引用另一个程序片段，例如

```
@ 画一条路径 #
\startMPpage
# 构造路径 p @
draw p withpen pencircle withcolor darkred;
\stopMPpage
@
```

引用代码片段所用的标记恰好与定义代码片段所用的标记相反。上述代码片段引用了名为「构造路径 p」的片段，后者的定义为

```
@ 构造路径 p #
path p;
p := fullsquare xysized (3cm, 2cm);
@
```

在使用 xi 程序抽取上述名为「画一条路径」的代码片段时，其引用的代码片段的内容也会被连带着抽取出来。例如，假设上述两个代码片段在同一个文学程序源文档 foo.xi 里，使用以下命令抽取「画一条路径」的代码片段：

```
$ xi --tangle --entrance "画一条路径" --output foo.tex foo.xi
```

上述命令也可以写成 xi 的短选项形式：

```
$ xi -t -e "画一条路径" -o foo.tex foo.xi
```

输出文件 foo.tex 的内容为

```
\startMPpage
path p;
p := fullsquare xysized (3cm, 2cm);
draw p withpen pencircle withcolor darkred;
\stopMPpage
```

当然，若被引用的代码片段，其定义中又引用了其他代码片段，则 xi 会根据引用链将其依序抽取出来。不过，xi 不允许代码片段自引用或回环引用。倘若你好奇，想尝试一下，我告诉你，结果会导致 xi 陷入递归而不能自拔。

同名片段追加运算

我有足够的理由相信，你和我一样不擅长为各种事物取名字，包括为代码片段取名字，所以当我们好不容易为一个代码片段取了名字，也许会希望新的代码片段可以继续用这个名字，特别是后者的内容往往是对前者内容的补充，于是就会出现同名的代码片段。例如

```
@ foo #
这是第一个名为 foo 的代码片段
@
@ foo #
这是第二个名为 foo 的代码片段
```

```
@
```

在抽取代码片段 foo 的内容时，xi 程序不知道该如何处理这样的同名代码片段。若第 2 个 foo 片段的内容确实是第 1 个 foo 片段内容的补充，则这两个代码片段应当定义为

```
@ foo #
这是第一个名为 foo 的代码片段
@
@ foo # +
这是第二个名为 foo 的代码片段
@
```

运算符 + 表示在之前出现的同名片段之后追加内容。当 xi 抽取 foo 片段内容时，会按照 + 所表达的操作合并同名片段的内容，所得结果为

```
这是第一个名为 foo 的代码片段
这是第二个名为 foo 的代码片段
```

有时，你会希望一个代码片段的内容能够追加到所有与它同名片段之前，这时需要用同名片段首部追加运算符[1] ^+。例如

```
@ bar #
这是第一个名为 bar 的代码片段
@
@ bar # +
这是第二个名为 bar 的代码片段
@
@ bar # ^+
这是第三个名为 bar 的代码片段
@
```

用 xi 程序抽取上述的 bar 片段，所得结果为

```
这是第三个名为 bar 的代码片段
这是第一个名为 bar 的代码片段
这是第二个名为 bar 的代码片段
```

亦即第 3 个 bar 片段的内容窜到了前两个 bar 片段之前了。

片段标签

若同名片段多于三个，倘若你想将一个新的同名片段内容追加到之前某个同名片段之前或之后，则 + 和 ^+ 需要配合代码片段标签才能完成这种操作。所谓代码标签，即位于代码片段名字下一行以尖括号包含的文本，例如

```
@ bar #
<代码片段标签>
这是第一个名为 bar 的代码片段
@
```

[1] 在标准键盘上，^ 符号和数字键 6 在同一键位，用 shift + 6 输入。

在下面的示例里，第 2 个 bar 片段含有标签 <hi>，第 3 个 bar 片段利用该标签将自身内容追加到第 2 个 bar 片段之前：

```
@ bar #
这是第一个名为 bar 的代码片段
@
@ bar # +
<hi>
这是第二个名为 bar 的代码片段
@
@ bar # <hi> ^+
这是第三个名为 bar 的代码片段
@
```

使用 xi 程序抽取上述 bar 片段内容时，所得结果为

```
这是第一个名为 bar 的代码片段
这是第三个名为 bar 的代码片段
这是第二个名为 bar 的代码片段
```

现在，出一道题目，用 xi 程序抽取下面示例中 bar 片段的内容，所得结果是什么？

```
@ bar #
<hi>
这是第一个名为 bar 的代码片段
@
@ bar # +
这是第二个名为 bar 的代码片段
@
@ bar # <hi> +
这是第三个名为 bar 的代码片段
@
```

文档编织

前面几节讲述的是 xi 的代码抽取功能，即与 xi 的 --tangle 选项相关的功能。若想将文学程序源文档排版为程序文档，需要用 xi 的 --weave 选项编织文学程序源文档。

文档编织过程依赖文学程序所用的排版语言。Donald Knuth 开创文学编程先河之时，他使用的排版语言是他所开发的 TEX 系统支持的排版语言，即 TEX 语言。其实在 TEX 语言之前，已有许多计算机排版语言了，然而 TEX 对排版质量的追求，至今仍然独占众多排版语言的鳌头，并且发展出 LATEX 和 ConTEXt 这些更为强大的排版语言。我偏好 ConTEXt，故而要用它作为示例，讲述如何将文学程序编织为 ConTEXt 源文档，再由 context 命令将后者编译为 PDF 文档。

若将文学程序源文档编织为 ConTEXt 源文档，需要先定义转换样式，亦即文学程序中，代码片段所用的一些标记，如何转换为 ConTEXt 排版命令。转换样式是由 xi 配置文件的形式提供，面向 ConTEXt 语言的转换样式的定义如下：

```
# ctx.conf
snippet_start: \start${language}\n
snippet_stop: \stop${language}
snippet_name_start: /BTEX\color[darkred]{@}/ETEX
```

```
snippet_name: /BTEX\type[color=darkred]{${name}}/ETEX
snippet_id: /BTEX\reference[xi-${id}]{${id}}\inoutermargin{\darkred{${id}}}/ETEX
snippet_name_stop: "/BTEX\color[darkred]{\#}/ETEX"
snippet_tag: \n/BTEX\color[darkmagenta]{<${name}>}/ETEX
snippet_tag_reference: /BTEX\color[darkmagenta]{\ <${name}>\ }/ETEX
snippet_reference_start: "/BTEX\color[darkcyan]{\#}/ETEX"
snippet_reference: /BTEX\type[color=darkcyan]{${name}}/ETEX
snippet_reference_id: /BTEX\ <\in[xi-${id}]>/ETEX
snippet_reference_stop: "/BTEX\color[darkcyan]{@}/ETEX"
# snippet_emission: /BTEX=>\color[darkyellow]{${name}}<\in[xi-${id}]>/ETEX\n
snippet_prepending_operator: /BTEX\ \color[darkgreen]{\bf $\wedge+$}\ /ETEX
snippet_appending_operator: /BTEX\ \color[darkgreen]{\bf $+$}\ /ETEX
```

先不要害怕上述这些样式定义，因为它们是与排版语言密切相关的。你若熟悉 ConTEXt，自然就能理解它们。上述代码里，只有形如 `${...}` 的标记独立于排版语言，它们是占位符。`${language}` 是语言占位符，表示代码片段所用的编程语言。`${name}` 表示代码片段的名字。`${id}` 表示代码片段的序号。在使用 `xi` 程序编织文档时，这些占位符会被填充具体的内容。

我需要重点讲述 `${language}` 占位符。`xi` 程序自然无法知道每个代码片段所用的语言，但是可以通过代码片段的语言标记告诉它，例如以下代码片段使用 `[MP]` 标记表示代码片段的语言是 MetaPost：

```
@ foo bar # [MP]
... MetaPost 代码 ...
@
```

基于上述样式定义，`xi` 可将上述代码片段编织为

```
\startMP
... MetaPost 代码 ...
\stopMP
```

无须每个代码片段都带上语言标记，只要片段引用链上任一片段带有语言标记，`xi` 程序会将其自动传播给链上的其他片段。

只要理解上述占位符的含义，结合你所熟悉的排版语言，你也能试着为 `xi` 写出可用的代码片段的转换样式定义，一旦有了它，便可使用 `xi` 程序为文学程序编织文档了。以 ConTEXt 排版语言为例，假设文学程序源文档 `foo.xi` 的内容为

```
\usemodule[zhfonts]
\starttext
这是一个没什么用处的文学程序，它只是以 \METAPOST\ 代码画一条线。
@ 画一条路径 # [MP]
\startMPpage
# 构造路径 p @
draw p withpen pencircle withcolor darkred;
\stopMPpage
@
构造路径 \type{p} 的过程为
@ 构造路径 p #
path p;
p := fullsquare xysized (3cm, 2cm);
```

```
@
\stoptext
```

再假设上述给出的样式定义存放在 ctx.conf 文件里，且该文件与 foo.xi 位于同一目录，执行以下命令便可将 foo.xi 编织为 foo.tex 文件：

```
$ xi --config ctx.conf --weave --output foo.tex foo.xi
```

上述命令也能写成短选项形式：

```
$ xi -c ctx.conf -w -o foo.tex foo.xi
```

所得 foo.tex 文件的内容如图 14.1 所示，然后使用以下命令将 foo.tex 编译为 foo.pdf 文件：

```
$ context foo.tex
```

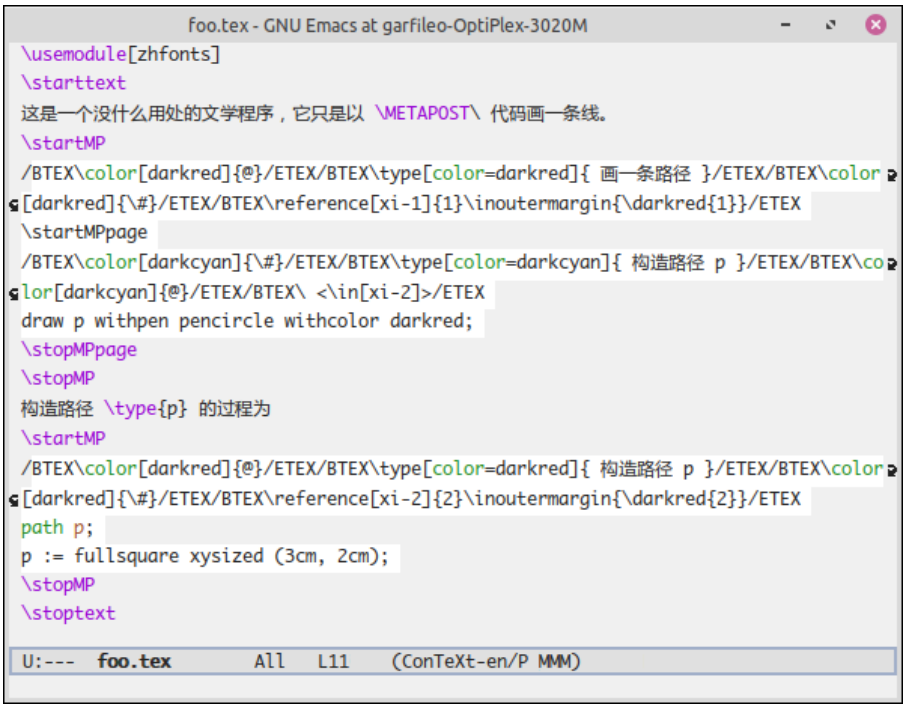


图 14.1　文学程序编织结果

同名片段截断

由于同名片段的存在，会导致在文学程序某个阶段的代码抽取结果或文档编织结果里出现了后面某些同名片段的内容，而这有时未必是你所希望的。例如

```
@ foo #
这是第一个名为 foo 的代码片段
@
@ foo # +
这是第二个名为 foo 的代码片段
@
@ bar #
# foo @
# demo @
@
```

```
@ foo #  +
这是第三个名为 foo 的代码片段
@
@ demo #
... demo ...
@
```

假设上述示例保存在 foo.xi 文件，若使用 xi 程序抽取片段 bar 的内容，则所得是前两个 foo 片段以及 demo 片段内容的累加结果，这是因为在 bar 只能引用位于它之前的同名片段，而bar 却可以向后引用普通片段。

修改片段标记

xi 使用的文学编程标记，实际上都是延续着这十多年来，它的各个前身一直使用的符号，不多，一共有十一个：

- 片段界限符 @；
- 片段名字界限符 #；
- 片段名内的续行符 \；
- 语言标记区域的起始符 [和终止符]；
- 片段标签（或其引用形式）的起始符 < 和终止符 >；
- 片段运算符 + 和 ^+；
- 片段引用区域的起始符 # 和终止符 @。

我已经习惯了这些符号，我心里的 xi 程序就是这些符号和普通文本组成的一种意象。无论我身处天涯，还是海角，倘若在某处摩崖的石刻上看到

```
@ ... ... ... \
  ... ... ... # [...] <...> + 或 ^+
<...>
... ... ...
# ... ... @
... ... ...
@
```

我就会进入一种状态，它的名字是，莫愁前路无知己。但是，这只是我的 xi，却未必是你的。你的 xi 的样子兴许是

```
||| ... ... ... \
    ... ... ... > [...] <...> + 或 ^+
<...>
... ... ...
::: ... ... ;;;
... ... ...
|||
```

前提是你需要在 xi 的配置文件里写入以下内容

```
snippet_delimiter: "|||"
snippet_name_delimiter: ">"
snippet_reference_start_mark: ":::"
```

```
snippet_reference_stop_mark: ";;;"
```

关于 xi 配置文件的用法在上一节已经提及了，即使用 --config 或 -c 选项指定其所在路径。需要注意的是，倘若 xi 配置文件和文学程序源文档并不在同一目录，则需要在 xi 程序的 --config 或 -c 选项中提供其路径，例如

```
xi -c your/path/to/ctx.conf -t -e "代码片段名" -o 程序源码文件 snail.xi
```

xi 所用的十一个符号皆可配置，它们在配置文件里的名目如下：

- 片段界限符 snippet_delimiter；
- 片段名字界限符 snippet_name_delimiter；
- 片段名内的续行符 snippet_name_continuation；
- 语言标记区域的起始符 language_start_mark 和 language_stop_mark；
- 片段标签（或其引用形式）的起始符 tag_start_mark 和终止符 tag_stop_mark；
- 片段运算符：snippet_appending_mark 和 snippet_preppending_mark；
- 片段引用区域的起始符 snippet_reference_start_mark 和终止符 snippet_reference_stop_mark。

小结

现在，也许你能够明白为何 xi 程序的名字在汉语是为何是「兮」了。这个程序实际上什么也没有创造，它只是设法将排版系统和编程语言连接了起来，亦即 xi 的本性为空，它之所以能够存在，是因为排版系统与编程语言的缘起。

许可证

本文学程序由程序代码和文档两部分构成，分别适用不同的自由许可证。

- **程序代码**：本文档及其源文档内的全部程序代码，以 GNU 通用公共许可证（GNU GPL）第 2 版或任何后续版本发布[1]。你可以根据自由软件基金会发布的 GPL 条款重新分发和/或修改这些代码。
- **文档作品**：本文档及其源文档内的说明文字，在 GNU 自由文档许可证（GNU FDL）第 1.3 版或任何后续版本条款下[2]，授予复制、分发和/或修改本文档的许可；无不变章节，无前封面文字，无后封面文字。

[1] GPL 全文见 https://www.gnu.org/licenses/gpl-2.0.html

[2] GFDL 全文见 https://www.gnu.org/licenses/fdl-1.3.html

后记

从 2026 年 9 月 1 日开始，历时 23 天，忙里偷闲，终于将这份文档写完了，而且是以文学编程这种颇为奇怪的方式写完了。接下来，我的任务是，去收拾一下混乱了二十三天的房间和混乱了二十三年的我的人生了。

虽然最终得到的 Snail 模块的全部代码仅有区区 495 行，而这份文档却有 100 多页。还记得本文档前言里所说的文学编程的坏处吗？好的未必真的好，坏的也未必真的坏。世间的事，哪能分得那么清楚呢？虽然一个仅有 495 行的程序，却拥有 100 多页的文档，对于作者而言也许是一种折磨，但对于读者而言，也许是一个宝藏。

事实上，这种折磨于我而言，也是有益的。在写这份文档的过程中，我对 MetaPost 语言的认识自然应该是更深了一层。例如，对于 `begingroup ... endgroup` 语句的理解，以前我从未想过它居然是表达式，导致我在定义正则路径运算符宏时，不得不借助一些全局变量来完成。还有一些以前我以为难以解决的问题，例如路径交叉特效，以前我绞尽脑汁，算出一条路径与其他所有路径的交点将底层路径断开，此法虽然可行，但有很多限制，而在本文档里，用了简单的路径背景骗术，问题便轻松化解了。

我还有一些收获，在撰写这份文档的过程里，因频繁使用 `xi` 程序抽取代码和编织文档，发现并修复了 `xi` 程序的一些 bug。大概只有足够篇幅的文学程序，才能发现这些 bug。

不过，本文档作为 MetaPost 教程还是很大欠缺，这主要是因为我对 MetaPost 的了解也颇为有限。此外，在写这份文档是，我的内心还是将它作为 Snail 模块备忘来写的，以防将来我要重修该模块时，还能记得起来它使用的 MetaPost 语法。没错，我也不是整天在用 MetaPost 画图。无论你多么熟悉的语言或工具，只要时间足够长，你总是会渐渐忘掉它们。文档可以帮我更快地将它们找回来，这大概也是文学编程好的一面。